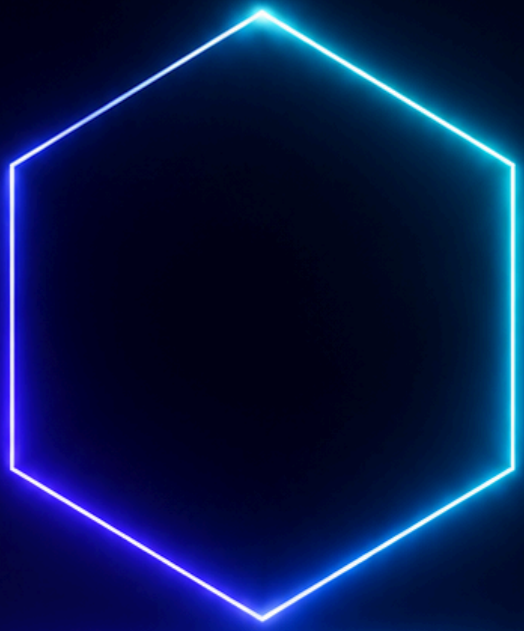
QA INSIDER



PERCA O MEDO DA AUTOMAÇÃO

13 CAPÍTULOS • 255 PÁGINAS

PLAYWRIGHT • APPIUM • API • SQL • CI/CD • IA

300 EXERCÍCIOS RESOLVIDOS COM GABARITO

50 PROMPTS PARA IA • CHECKLIST • GLOSSÁRIO

LILIAN BORBA

QA LEAD SÊNIOR • QA INSIDER

SOBRE A AUTORA

Lilian Borba

QA Lead Sênior | Arquiteta de Qualidade & IA

QAOps · CI/CD · Playwright · Cypress · Appium

Com mais de 20 anos de experiência em Qualidade de Software, Lilian Borba construiu uma carreira sólida como QA Lead e Arquiteta de Qualidade, atuando em projetos de alta complexidade nas áreas de automação de testes, estratégia de qualidade e engenharia de QA. Baseada em Belo Horizonte, ela é referência em QAOps, CI/CD e na aplicação de Inteligência Artificial no contexto de testes de software.

Fundadora do QA Insider, plataforma de conteúdo e educação voltada para profissionais de QA no Brasil, Lilian tem como missão democratizar o conhecimento técnico em automação e qualidade, com uma linguagem direta, prática e livre de abstrações acadêmicas.

Sua abordagem é centrada no mundo real: cada conceito ensinado neste ebook foi vivenciado em projetos reais, com equipes reais, enfrentando os mesmos desafios que você encontra no dia a dia.

20+ anos de QA	QA Lead & Sênior	QA Insider Founder	Automacao & IA

Stack & Especialidades

Automação Web	Playwright · Cypress · Selenium
Automação Mobile	Appium · BrowserStack · Firebase
API Testing	Postman · Rest Assured · Playwright API
BDD/ATDD	Cucumber · Gherkin · Robot Framework
CI/CD	GitHub Actions · Jenkins · GitLab CI
IA para QA	ChatGPT · Copilot · Claude · Jules
Performance	k6 · JMeter · Gatling

Qualidade Estratégica	QAOps · Test Architecture · Shift-Left

linkedin.com/in/lilian-borba | QA Insider: qa-insider-labs.base44.app

SUMÁRIO

PARTE 1 O Medo da Automação 7

- Por que QAs têm medo de automação
- Principais bloqueios mentais
- Mitos e realidade sobre automação
- Caso real de transição QA Manual → Automação

PARTE 2 Fundamentos de Automação 11

- O que automatizar (e o que não)
- Tipos de testes: regressão, smoke, sanidade, integração, API
- Conceitos de programação para QA
- Exercícios práticos resolvidos

PARTE 3 Automação Web 15

- Selenium · Cypress · Playwright — comparativo
- Primeiro projeto do zero
- Localizadores: id, class, XPath, CSS Selector
- Exemplos: login, cadastro, carrinho, checkout, modal…

PARTE 4 API Testing 22

- Request / Response / Headers / Body / Status Codes
- Postman do zero: GET, POST, PUT, DELETE
- Automação de API: Playwright / Supertest / Rest Assured
- Validações automatizadas

PARTE 5 Banco de Dados — SQL para QA 27

- SELECT, WHERE, ORDER BY, GROUP BY
- JOIN (INNER, LEFT), COUNT, SUM, AVG
- Casos reais de validação em testes

PARTE 6 IA Aplicada à Automação 30

- Como usar IA sem depender dela
- ChatGPT · Copilot · Claude — exemplos reais
- Gerar testes, dados, código e explicar erros

PARTE 7 Boas Práticas de Automação 33

■ Page Object Model
■ Clean Code para QA
■ Flaky tests e waits
■ Estrutura de frameworks profissionais

PARTE 8 CI/CD 36
■ GitHub Actions · Jenkins · GitLab CI
■ Pipeline básico de automação

PARTE 9 Mobile Testing 39
■ Appium do zero: instalação, arquitetura, primeiro teste
■ Localizadores mobile: accessibility id, xpath, uiAutomator
■ Cenários: biometria, GPS, câmera, push, swipe

PARTE 10 Robot Framework 42
■ Estrutura de projeto
■ Selenium + Robot · API + Robot · Banco + Robot

PARTE 11 BDD 44
■ Gherkin: Given / When / Then
■ Cucumber do zero + integração web

PARTE 12 Outras Ferramentas 46
■ Cypress avançado · Playwright avançado · Selenium Grid
■ Karate · Rest Assured · JMeter · k6 · OWASP

PARTE 13 Carreira em Automação 49
■ Como escolher sua stack
■ Roadmap: Júnior → Pleno → Sênior
■ Entrevistas técnicas com respostas comentadas

BÔNUS Exercícios & Recursos 53
■ 100 exercícios Web · 100 API · 50 SQL · 50 Mobile
■ 50 prompts para IA · Checklist profissional · Glossário

■ qa-insider-labs.base44.app

PARTE 1

O MEDO DA AUTOMAÇÃO

Por que tantos QAs travam antes mesmo de começar?

Por que QAs têm medo de automação

A maioria dos profissionais de QA que nunca aprendeu automação não é preguiçosa, nem incapaz. É vítima de um conjunto de crenças limitantes que se acumularam ao longo do tempo — vindas de colegas, de recrutadores e até de si mesmos.

Este ebook começa exatamente aqui: desmontando esses bloqueios um por um, com honestidade e dados reais do mercado.

Principais Bloqueios Mentais

"Não sei programar"

Este é o bloco número um. E faz sentido — o termo 'automação' soa como sinônimo de 'desenvolvimento de software'. Mas não é.

Automação de testes exige lógica, não engenharia de software completa. Você não vai construir um sistema bancário. Vai escrever instruções que dizem: 'clique aqui, verifique que o resultado é X, repita.'

> ■ ***Ponto-chave: Ferramentas como Playwright e Cypress foram projetadas para pessoas que testam software, não para quem desenvolve frameworks de produção.***

"Sou ruim de lógica"

Você já escreveu um roteiro de testes? 'Se o usuário informar uma senha incorreta 3 vezes, o sistema deve bloquear a conta.' Isso é lógica. Você já pensa dessa forma todos os dias.

A diferença é só a sintaxe — a forma como você expressa essa lógica para uma máquina. E sintaxe se aprende em semanas.

"Já tentei e falhei"

Provavelmente você tentou da forma errada. Muitos QAs são jogados em um projeto de automação sem base, sem estrutura de aprendizado, sem mentoria — e ficam sozinhos olhando para um terminal vazio.

A falha não foi sua. Foi do método. Este ebook foi estruturado para eliminar exatamente esse problema: cada passo constrói sobre o anterior.

"Sou velho(a) demais para aprender"

Esta crença é particularmente cruel — e completamente falsa. Profissionais com 10, 15, 20 anos de experiência em QA manual têm uma vantagem enorme sobre desenvolvedores aprendendo testes: vocês já entendem o problema que estão resolvendo.

> ■ ***Um desenvolvedor que aprende Selenium precisa aprender QA E programação. Você já sabe QA. Só precisa aprender a sintaxe.***

Mitos da Automação

■ Mito	■ Realidade
Automação substitui o QA manual	Automação executa os testes repetitivos; o QA humano pensa nos cenários que a máquina nunca descobriria
Precisa saber programar como dev	Você precisa de lógica e sintaxe básica — não arquitetura de software
Automação é cara e complexa	Com Playwright + GitHub Actions, você tem CI/CD gratuito em 1 tarde
Só serve para empresas grandes	Uma startup com 2 devs se beneficia ainda mais — menos tempo em regressão manual
Uma vez automatizado, nunca quebra	Testes precisam de manutenção — flaky tests são realidade, não fracasso

O que Automação NÃO É

- ■ ■ Substituição completa do teste manual e exploratório
- ■ ■ Solução para requisitos mal definidos
- ■ ■ Garantia de que bugs não existirão em produção
- ■ ■ Algo que só funciona quando os testes nunca falham

■■ Um projeto paralelo que 'o time de automação' faz separado do QA

O que Automação REALMENTE É

■■ Uma ferramenta para executar testes repetitivos de forma rápida e confiável

■■ Um multiplicador de cobertura — você escreve uma vez, roda mil vezes

■■ Parte de uma estratégia de qualidade (não a estratégia inteira)

■■ Uma habilidade que qualquer QA pode aprender com o método certo

■■ A base para CI/CD e entrega contínua com qualidade

Caso Real: Transição QA Manual → Automação

Ana tinha 8 anos de experiência em QA manual quando sua empresa decidiu migrar para entregas ágeis semanais. O time de regressão levava 3 dias completos a cada sprint. Ana foi designada para 'aprender automação em 30 dias'.

Primeira semana: pânico. Abriu o VS Code, viu um arquivo .js vazio e fechou o notebook. Segunda semana: assistiu a 4 tutoriais aleatórios no YouTube — cada um usando uma ferramenta diferente, nenhum explicando o porquê.

O que funcionou: ela parou de tentar aprender tudo. Focou em um único cenário de teste que conhecia de cor — o login do sistema. Escreveu o primeiro teste em Playwright em 2 horas. Funcionou.

■ ***Lição: o primeiro teste não precisa ser perfeito. Precisa funcionar. A partir daí, você tem evidência de que consegue — e o medo começa a ir embora.***

Em 45 dias, Ana tinha 120 testes rodando em pipeline. A regressão caiu de 3 dias para 40 minutos.

Exercício: Autoavaliação do Seu Nível Atual

Responda honestamente (sem julgamento):

1	Você já abriu um arquivo de código (.js, .py, .java) e entendeu pelo menos 30% do que estava escrito?	SIM / NÃO
2	Você consegue escrever um roteiro de teste manual com condições? (Ex: SE o campo estiver vazio, ENTÃO exibir mensagem X)	SIM / NÃO
3	Você já usou o DevTools do navegador para inspecionar um elemento HTML?	SIM / NÃO

4	Você já fez uma requisição GET no Postman ou em qualquer outra ferramenta?	SIM / NÃO
5	Você já configurou qualquer ferramenta de linha de comando (npm, pip, etc)?	SIM / NÃO

Resultado:

0–1 SIM	Ponto de partida zero. Ótimo — você vai aprender sem vícios.
2–3 SIM	Base iniciante. Você já tem os fundamentos cognitivos necessários.
4–5 SIM	Você está mais próximo do que imagina. Falta só prática dirigida.

PARTE 2

FUNDAMENTOS DE AUTOMAÇÃO

O que você precisa saber antes de escrever seu primeiro teste

O que Automatizar (e o que NÃO Automatizar)

Esta é a decisão mais estratégica em qualquer projeto de automação. Automatizar errado desperdiça tempo, dinheiro e gera flaky tests que destroem a confiança do time na suite.

AUTOMATIZE ISSO	NÃO AUTOMATIZE ISSO
■ Testes de regressão que rodam em todo ciclo	■ Testes exploratórios (por natureza, são manuais)
■ Fluxos críticos: login, checkout, autenticação	■ Cenários de usabilidade e UX
■ Validações de API com dados previsíveis	■ Funcionalidades que mudam toda semana
■ Testes de fumaça (smoke tests) pré-deploy	■ Testes que dependem de dados externos instáveis
■ Cenários que rodam em múltiplos ambientes	■ Casos de uso que são executados raramente (1x por ano)
■ Validações de banco de dados pós-processamento	

Tipos de Testes

Regressão	Valida que funcionalidades existentes não foram quebradas por novas mudanças. É o coração da automação — roda a cada commit ou PR.
Smoke Test	Verificação rápida das funcionalidades mais críticas. Executado antes de qualquer teste mais profundo. Falhou? Pare tudo.
Sanidade	Subset da regressão, focado em validar uma correção ou nova funcionalidade específica. Mais rápido que a regressão completa.
Integração	Verifica que dois ou mais componentes funcionam corretamente juntos. Ex: frontend + API + banco de dados.

API	Testa diretamente os endpoints da aplicação, sem interface. Mais rápido, mais estável e mais próximo da lógica de negócio.

Conceitos Básicos de Programação para QA

Você não precisa saber tudo. Precisa saber o suficiente para escrever, ler e manter testes. Estes são os 5 conceitos que cobrem 90% do que você vai usar:

1. Variáveis

Uma variável é um nome que guarda um valor. Simples assim.

```
# JavaScript (Playwright/Cypress)
const usuario = 'lilian@qastudio.com'
const senha = 'Senha@123'
const tentativas = 3

# Python (Playwright/Robot)
usuario = 'lilian@qastudio.com'
senha = 'Senha@123'
tentativas = 3
```

■ ***Por que usar variáveis? Se o email mudar, você muda em 1 lugar — não em 50 testes.***

2. Condições (if/else)

Exatamente o que você já escreve em critérios de aceite — agora em código.

```
# Se o login falhar, mostrar mensagem de erro
if (mensagem === 'Usuário ou senha inválidos') {
  console.log('PASS: mensagem correta exibida')
} else {
  console.log('FAIL: mensagem inesperada')
}
```

3. Loops (for / while)

Repete uma ação N vezes — útil para testar com múltiplos dados.

```
# Testar login com 5 usuários diferentes
const usuarios = ['ana@qa.com', 'bob@qa.com', 'carol@qa.com']

for (const user of usuarios) {
  await page.fill('#email', user)
  await page.fill('#senha', 'Senha@123')
  await page.click('#btn-login')
  // verificar resultado
}
```

4. Arrays / Listas

Coleção de valores. Perfeito para massa de dados de teste.

```
# Lista de CPFs para teste de validação
cpfs_invalidos = ['000.000.000-00', '111.111.111-11', '123.456.789-00']

for cpf in cpfs_invalidos:
    preencher_campo('cpf', cpf)
    clicar('btn-salvar')
    verificar_mensagem('CPF inválido')
```

5. Objetos

Agrupam dados relacionados. Você vai usar muito para fixtures e page objects.

```
// Objeto representando um usuário de teste
const usuario = {
  nome: 'Lilian Borba',
  email: 'lilian@qastudio.com',
  perfil: 'admin',
  ativo: true
}

// Acessando os dados
console.log(usuario.email) // lilian@qastudio.com
console.log(usuario.perfil) // admin
```

Exercícios Práticos Resolvidos

Exercício 1

Crie uma variável com a URL base do sistema que você testa hoje.

```
const baseUrl = 'https://sistema.minhaempresa.com'
```

Exercício 2

Crie uma lista com 3 perfis de usuário do seu sistema.

```
const perfis = ['admin', 'operador', 'visualizador']
```

Exercício 3

Escreva uma condição que verifica se um status code é 200.

```
if (statusCode === 200) {
  console.log('PASS')
} else {
  console.log('FAIL: status ' + statusCode)
}
```

PARTE 3

AUTOMAÇÃO WEB

Selenium · Cypress · Playwright — do zero aos exemplos reais

Comparativo de Ferramentas

Critério	Selenium	Cypress	Playwright
Linguagens	Java, Python, JS, C#	JavaScript/TypeScript	JS/TS, Python, Java, C#
Velocidade	Média	Rápido	Muito Rápido
Debug	Complexo	Excelente (Time Travel)	Muito Bom (Trace Viewer)
Mobile	Via Appium	Não nativo	Parcial (emulação)
CI/CD	Sim	Sim	Sim (built-in)
Curva aprendizado	Alta	Média	Média-Baixa
Recomendado para	Projetos legados, Java	SPAs, times JS	Projetos novos, multi-plataforma

■ ***Recomendação QA Insider: Para projetos novos, comece com Playwright. Para projetos existentes em Java, Selenium ainda é a escolha.***

Primeiro Projeto com Playwright — Do Zero

Siga os passos. Copie e execute. Um passo de cada vez.

Passo 1: Instalar Node.js e Playwright

```
# 1. Instalar Node.js (https://nodejs.org) — versão LTS
# 2. Verificar instalação:
node --version # deve mostrar v18 ou superior
npm --version

# 3. Criar projeto
mkdir meu-primeiro-projeto
cd meu-primeiro-projeto
npm init -y

# 4. Instalar Playwright
npm init playwright@latest
# Responda as perguntas: TypeScript? N / Pasta tests? Y / Browsers? Enter
```

Passo 2: Estrutura do Projeto

```
meu-primeiro-projeto/
■■■ tests/
■ ■■■ exemplo.spec.js # seus testes ficam aqui
■■■ playwright.config.js # configuração global
■■■ package.json
■■■ node_modules/
```

Passo 3: Primeiro Teste de Login

Vamos testar o login de uma aplicação real de exemplo: https://the-internet.herokuapp.com/login

```
// tests/login.spec.js
const { test, expect } = require('@playwright/test')

test('Login com credenciais válidas', async ({ page }) => {
  // 1. Navegar para a página de login
  await page.goto('https://the-internet.herokuapp.com/login')

  // 2. Preencher usuário
  await page.fill('#username', 'tomsmith')

  // 3. Preencher senha
  await page.fill('#password', 'SuperSecretPassword!')

  // 4. Clicar em Login
  await page.click('button[type="submit"]')

  // 5. Verificar que o login funcionou
  await expect(page.locator('#flash')).toContainText('You logged into a secure
area!')
})

test('Login com senha incorreta', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/login')
  await page.fill('#username', 'tomsmith')
  await page.fill('#password', 'senha_errada')
  await page.click('button[type="submit"]')

  // Verificar mensagem de erro
  await expect(page.locator('#flash')).toContainText('Your password is invalid!')
})
```

Passo 4: Executar os Testes

```
# Rodar todos os testes
npx playwright test

# Rodar com interface visual
npx playwright test --headed

# Rodar e abrir relatório HTML
npx playwright test --reporter=html
npx playwright show-report
```

Localizadores

Localizadores dizem ao Playwright qual elemento da página você quer interagir. Escolher o localizador certo evita testes frágeis.

ID	`#meu-botao`	MELHOR — único por página, estável
Data Attribute	`[data-testid='btn-login']`	IDEAL — criado especificamente para testes
CSS Selector	`.form-control.email`	BOM — legível e específico
Text	`text='Entrar'`	ÚTIL — para botões e links com texto fixo
XPath	`//button[@id='submit']`	EVITE — verboso e frágil
Class	`.btn-primary`	CUIDADO — classes mudam com redesign

Exemplos Completos de Testes

Teste de Cadastro

```
test('Cadastro de novo usuário', async ({ page }) => {
  await page.goto('https://app.exemplo.com/cadastro')

  await page.fill('[data-testid="campo-nome"]', 'Maria Silva')
  await page.fill('[data-testid="campo-email"]', 'maria@teste.com')
  await page.fill('[data-testid="campo-senha"]', 'Senha@123')
  await page.fill('[data-testid="confirmar-senha"]', 'Senha@123')

  // Selecionar perfil
  await page.selectOption('[data-testid="perfil"]', 'usuario')

  // Aceitar termos
  await page.check('[data-testid="aceitar-termos"]')

  await page.click('[data-testid="btn-cadastrar"]')

  // Verificar redirecionamento
  await expect(page).toHaveURL('/dashboard')
  await expect(page.locator('h1')).toContainText('Bem-vindo, Maria!')
})
```

Teste de Modal

```
test('Modal de confirmação ao deletar item', async ({ page }) => {
  await page.goto('https://app.exemplo.com/items')

  // Clicar no botão de deletar do primeiro item
  await page.click('.btn-deletar >> nth=0')

  // Verificar que o modal apareceu
  const modal = page.locator('[data-testid="modal-confirmacao"]')
  await expect(modal).toBeVisible()
  await expect(modal.locator('h2')).toContainText('Tem certeza?')

  // Confirmar exclusão
  await modal.locator('[data-testid="btn-confirmar"]').click()

  // Verificar feedback
  await expect(page.locator('.toast-success')).toContainText('Item removido com
sucesso')
})
```

Teste de Upload de Arquivo

```
test('Upload de arquivo CSV', async ({ page }) => {
  await page.goto('https://app.exemplo.com/importar')

  // Definir o arquivo para upload (usar path local)
  const [fileChooser] = await Promise.all([
    page.waitForEvent('filechooser'),
    page.click('[data-testid="btn-selecionar-arquivo"]')
  ])
  await fileChooser.setFiles('./fixtures/dados-teste.csv')

  // Verificar preview
  await expect(page.locator('.preview-linhas')).toContainText('10 linhas
encontradas')

  // Confirmar importação
  await page.click('[data-testid="btn-importar"]')
  await expect(page.locator('.resultado')).toContainText('Importação concluída')
})
```

PARTE 4

API TESTING

Testando a camada que realmente sustenta o sistema

O que é uma API — Explicação Simples

API (Application Programming Interface) é a camada que conecta sistemas. Quando você usa um app de banco no celular, a tela é o frontend — mas quem verifica seu saldo, processa transferências e autentica seu acesso são as APIs rodando nos servidores do banco.

> ■ ***Testar API é testar a lógica de negócio pura, sem o ruído visual do frontend. É mais rápido, mais estável e encontra bugs que testes de UI nunca veriam.***

Anatomia de uma Requisição HTTP

URL/Endpoint	`https://api.sistema.com/v1/usuarios/123`	O endereço do recurso que você quer acessar
Método HTTP	`GET · POST · PUT · PATCH · DELETE`	O que você quer fazer com o recurso
Headers	`Authorization, Content-Type, Accept`	Metadados da requisição — autenticação, formato dos dados
Body	`JSON, Form Data, XML`	Os dados enviados (apenas em POST, PUT, PATCH)
Query Params	`?pagina=1&limite;=20&ativo;=true`	Filtros e parâmetros passados na própria URL

Status Codes que Todo QA Precisa Conhecer

200 OK	Requisição bem-sucedida — GET retornou dados

201 Created	Recurso criado com sucesso — POST funcionou
204 No Content	Sucesso sem corpo de resposta — DELETE funcionou
400 Bad Request	Dados enviados inválidos — validar payload
401 Unauthorized	Token ausente ou inválido
403 Forbidden	Autenticado mas sem permissão
404 Not Found	Recurso não existe — verificar ID/endpoint
422 Unprocessable	Payload válido mas falha na validação de negócio
500 Internal Server Error	Bug no servidor — reportar com body completo

Automação de API com Playwright

Playwright tem um contexto nativo de API — sem abrir navegador, direto no protocolo HTTP. Rápido, sem overhead visual.

CRUD Completo — Usuário

```
// tests/api/usuarios.spec.js
const { test, expect, request } = require('@playwright/test')

const BASE_URL = 'https://reqres.in/api'
let usuarioId // variável compartilhada entre os testes

// ■■ CRIAR USUÁRIO ■■■■■■■■■■■■■■■■■■■■■■■■■■■
test('POST /users — criar usuário', async () => {
  const api = await request.newContext()
  const response = await api.post(`${BASE_URL}/users`, {
    data: {
      name: 'Lilian Borba',
      job: 'QA Lead'
    }
  })

  expect(response.status()).toBe(201)
  const body = await response.json()
  expect(body.name).toBe('Lilian Borba')
  expect(body.id).toBeDefined()
  usuarioId = body.id
})

// ■■ CONSULTAR USUÁRIO ■■■■■■■■■■■■■■■■■■■■■■■
test('GET /users/:id — buscar usuário', async () => {
```

```
  const api = await request.newContext()
  const response = await api.get(`${BASE_URL}/users/2`)

  expect(response.status()).toBe(200)
  const body = await response.json()
  expect(body.data).toHaveProperty('email')
  expect(body.data.id).toBe(2)
})

// ■■ ATUALIZAR USUÁRIO ■■■■■■■■■■■■■■■■■■■■■■■
test('PUT /users/:id — atualizar usuário', async () => {
  const api = await request.newContext()
  const response = await api.put(`${BASE_URL}/users/2`, {
    data: { name: 'Lilian Borba', job: 'QA Architect' }
  })

  expect(response.status()).toBe(200)
  const body = await response.json()
  expect(body.job).toBe('QA Architect')
  expect(body.updatedAt).toBeDefined()
})

// ■■ DELETAR USUÁRIO ■■■■■■■■■■■■■■■■■■■■■■■■
test('DELETE /users/:id — remover usuário', async () => {
  const api = await request.newContext()
```

```
  const response = await api.delete(`${BASE_URL}/users/2`)

  expect(response.status()).toBe(204)
})
```

Automação com Autenticação Bearer Token

```
test('GET /perfil — endpoint autenticado', async () => {
  const api = await request.newContext({
    extraHTTPHeaders: {
      'Authorization': `Bearer ${process.env.API_TOKEN}`,
      'Content-Type': 'application/json',
    }
  })

  const response = await api.get(`${BASE_URL}/perfil/me`)
  expect(response.status()).toBe(200)

  const perfil = await response.json()
  expect(perfil.email).toBeDefined()
  expect(perfil.ativo).toBe(true)
})
```

PARTE 5

SQL PARA QA

Validações diretas no banco de dados — sem depender do frontend

Por que QA precisa de SQL?

A interface pode mostrar 'Cadastro realizado com sucesso' e ainda assim o dado estar errado, duplicado ou incompleto no banco. SQL te dá acesso direto à fonte da verdade.

SELECT e WHERE — A Base de Tudo

```
-- Buscar todos os usuários ativos
SELECT * FROM usuarios WHERE ativo = 1;

-- Buscar usuário específico
SELECT nome, email, perfil FROM usuarios WHERE email = 'lilian@qa.com';

-- Verificar se um pedido foi criado
SELECT * FROM pedidos WHERE id = 12345 AND status = 'APROVADO';

-- Buscar registros criados hoje
SELECT * FROM logs WHERE DATE(criado_em) = CURDATE();
```

GROUP BY, COUNT e Aggregações

```
-- Quantos usuários por perfil?
SELECT perfil, COUNT(*) AS total
FROM usuarios
GROUP BY perfil;

-- Valor total de pedidos por status
SELECT status, SUM(valor) AS total, COUNT(*) AS qtd
FROM pedidos
GROUP BY status;

-- Validar que não há emails duplicados
SELECT email, COUNT(*) AS ocorrencias
FROM usuarios
GROUP BY email
HAVING COUNT(*) > 1;
```

JOINs — Cruzando Tabelas

```
-- INNER JOIN: mostrar pedidos com nome do cliente
SELECT p.id, u.nome, p.valor, p.status
FROM pedidos p
INNER JOIN usuarios u ON u.id = p.usuario_id
WHERE p.status = 'PENDENTE';

-- LEFT JOIN: usuários e seus pedidos (incluindo quem não pediu nada)
SELECT u.nome, COUNT(p.id) AS total_pedidos
FROM usuarios u
LEFT JOIN pedidos p ON p.usuario_id = u.id
GROUP BY u.id, u.nome;
```

Casos Reais de Validação

Cenário: Após criar usuário via API

```
SELECT * FROM usuarios WHERE email = 'novo@teste.com' AND ativo = 1
```

Cenário: Após processar pagamento

```
SELECT status, gateway_id FROM pagamentos WHERE pedido_id = 99 AND status =
'APROVADO'
```

Cenário: Verificar trigger de auditoria

```
SELECT * FROM auditoria WHERE tabela = 'pedidos' AND acao = 'INSERT' ORDER BY
criado_em DESC LIMIT 1
```

Cenário: Validar regra de negócio

```
SELECT COUNT(*) FROM descontos WHERE usuario_id = 10 AND DATE(criado_em) =
CURDATE()
```

PARTE 6

IA APLICADA À AUTOMAÇÃO

Como usar IA como copiloto — sem perder o controle

A Regra de Ouro: Use IA como Copiloto, Não como Piloto

IA não conhece seu sistema. Não sabe quais regras de negócio existem, quais fluxos são críticos, quais cenários de borda já causaram incidentes. Você sim. O QA que delega o raciocínio para a IA para de evoluir.

■ ***IA acelera. QA decide. Nunca o contrário.***

Prompts com 'Aja Como' — Exemplos Reais

A diferença entre um prompt genérico e um prompt profissional é contexto. Ao dizer 'aja como', você configura o papel da IA antes de fazer o pedido.

Gerar cenários de teste

Prompt:

```
Aja como um QA Sênior especialista em testes de API REST. Analise o endpoint abaixo e gere uma lista completa de cenários de teste cobrindo: fluxo feliz, validações de campo, autenticação, autorização, casos de borda e erros esperados. Use o formato Gherkin. Endpoint: POST /api/v1/transferencias
```

■ *A IA vai gerar cenários estruturados porque entende o papel que está exercendo.*

Gerar dados de teste

Prompt:

```
Aja como um engenheiro de qualidade criando massa de dados para testes de carga. Gere 10 objetos JSON de usuário com dados brasileiros reais (CPF válido, CEP válido, email único). Cada usuário deve ter: nome, CPF, email, telefone, endereço completo.
```

■ *Com contexto de QA + dados brasileiros, a IA entende o padrão esperado.*

Explicar erro de teste

Prompt:

```
Aja como um especialista em Playwright. Analise o erro abaixo e explique: (1) o que causou o erro, (2) como corrigir, (3) como prevenir esse tipo de erro no futuro. Erro: TimeoutError: waiting for locator('#btn-submit') to be visible
```

■ *Em vez de só dar a resposta, a IA explica o raciocínio — você aprende junto.*

Revisar código de teste

Prompt:

```
Aja como um QA Architect revisando código de automação. Analise o teste abaixo e aponte: (1) problemas de manutenibilidade, (2) falta de boas práticas como Page Object Model, (3) localizadores frágeis, (4) ausência de asserções adequadas. Seja direto e técnico.
```

■ *A IA age como reviewer sênior, não como assistente genérico.*

Criar plano de testes

Prompt:

```
Aja como uma QA Lead Sênior com 20 anos de experiência, especialista em sistemas financeiros. Crie um plano de testes para a funcionalidade de transferência PIX, considerando: regulamentações do Banco Central, casos de fraude, janela de operação 24/7, integração com DICT e SPI.
```

■ *Com especialidade definida, o nível técnico do output é radicalmente diferente.*

Erros Comuns ao Usar IA em QA

■ **Aceitar código sem revisar:** IA gera código que compila mas não testa o que você quer. Sempre leia linha por linha.

- **Prompt sem contexto:** 'Gere testes para login' é um prompt pobre. Inclua: stack, framework, tipo de aplicação, regras de negócio.
- **Não testar o código gerado:** IA alucina. Localizadores inexistentes, métodos depreciados, lógica invertida — acontece.
- **Delegar raciocínio estratégico:** Priorização de testes, cobertura de risco, critérios de entrada/saída de sprint — isso é seu trabalho.
- **Usar IA offline para dados sensíveis:** Nunca cole dados reais de clientes, senhas ou tokens em ferramentas de IA sem verificar a política de dados.

PARTE 7

BOAS PRÁTICAS DE AUTOMAÇÃO

A diferença entre um script que roda e um framework que escala

Page Object Model (POM)

Page Object Model é o padrão de design mais importante em automação web. A ideia: cada página ou componente da aplicação tem uma classe correspondente que encapsula seus elementos e ações. O teste em si não sabe como a página funciona — só sabe o que quer fazer.

SEM Page Object — Problemático

```
// ■ Localizadores espalhados pelo teste
test('login', async ({ page }) => {
  await page.fill('#email', 'user@qa.com')
  await page.fill('#password', '123')
  await page.click('#btn-submit')
  await expect(page.locator('.dashboard-title')).toBeVisible()
})

// Se o ID mudar, você busca em TODOS os arquivos de teste
```

COM Page Object — Profissional

```
// pages/LoginPage.js
class LoginPage {
  constructor(page) {
    this.page = page
    this.emailField = page.locator('#email')
    this.passwordField = page.locator('#password')
    this.submitButton = page.locator('#btn-submit')
  }

  async login(email, senha) {
    await this.emailField.fill(email)
    await this.passwordField.fill(senha)
    await this.submitButton.click()
  }
}

// tests/login.spec.js
test('login com credenciais válidas', async ({ page }) => {
  const loginPage = new LoginPage(page)
  await loginPage.login('user@qa.com', 'Senha@123')
  // O teste não sabe como o login funciona — só que ele deve funcionar
})
```

■ ***Benefício real: quando o ID do botão mudar de #btn-submit para #btn-entrar, você muda em 1 arquivo — não em 50 testes.***

Flaky Tests — O Inimigo Silencioso

Flaky test é um teste que às vezes passa, às vezes falha — sem mudança no código. É pior que um teste que sempre falha, porque destrói a confiança do time na suite inteira.

Timing/Waits	Elemento não carregou quando o teste tentou interagir	*Fix: Use locator.waitFor() ou expect(locator).toBeVisible()*

Dados com partilhados	Teste A cria um dado que o Teste B depende	*Fix: Cada teste deve criar seus próprios dados (fixtures independentes)*
Ordem de execução	Testes que dependem de outros testes na sequência	*Fix: Cada teste deve ser independente e idempotente*
Ambiente instável	Serviço externo lento ou indisponível	*Fix: Mock de dependências externas em testes de unidade/integração*

PARTE 8

CI/CD

Automação que roda sozinha — em todo PR, todo dia, toda hora

O que é CI/CD

CI (Continuous Integration) = integrar e testar código automaticamente a cada push. CD (Continuous Delivery/Deployment) = entregar para produção de forma automatizada e confiável.

Para QA, o mais importante é o CI: sua suite de testes deve rodar a cada Pull Request, bloqueando merge se algo quebrar.

GitHub Actions — Pipeline Básico

Crie o arquivo .github/workflows/testes.yml no seu repositório:

```yaml
# .github/workflows/testes.yml
name: Testes Automatizados — QA

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  testes:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout do código
        uses: actions/checkout@v4

      - name: Configurar Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Instalar dependências
        run: npm ci
```

```yaml
      - name: Instalar browsers do Playwright
        run: npx playwright install --with-deps chromium

      - name: Executar testes
        run: npx playwright test
        env:
          BASE_URL: ${{ secrets.BASE_URL }}
          API_TOKEN: ${{ secrets.API_TOKEN }}

      - name: Publicar relatório HTML
        uses: actions/upload-artifact@v4
        if: always()
        with:
          name: playwright-report
          path: playwright-report/
          retention-days: 30
```

■ ***O if: always() garante que o relatório seja salvo mesmo quando os testes falham — fundamental para debug em CI.***

PARTE 9

MOBILE TESTING

Appium · BrowserStack · Firebase — testes em apps reais

Web vs Mobile — Diferenças Críticas para QA

Aspecto	Web	Mobile
Contexto	Browser padronizado	iOS (WebKit) + Android (Chromium) + apps nativos
Localizadores	HTML/CSS/XPath	Accessibility ID, UiAutomator2, XPath mobile
Instalação	Nenhuma	APK/IPA + dispositivo/emulador + Appium Server
Conectividade	Estável (desktop)	WiFi/4G/5G/offline — testar todos
Hardware	Ignorado	Câmera, GPS, biometria, giroscópio, notificações
Tamanhos	Responsivo CSS	Múltiplos devices com resoluções diferentes

Appium — Primeiro Teste Android

```
# Instalação
npm install -g appium
appium driver install uiautomator2 # Android
appium driver install xcuitest # iOS

# Verificar setup
appium doctor --android
```

```python
# tests/mobile/login_android.py
from appium import webdriver
from appium.options import UiAutomator2Options

options = UiAutomator2Options()
options.platform_name = 'Android'
options.device_name = 'emulator-5554'
options.app = './apps/meu_app.apk'
options.app_package = 'com.meusistema.app'
options.app_activity = '.MainActivity'
options.no_reset = True

driver = webdriver.Remote('http://localhost:4723', options=options)

# Localizar por accessibility id (MELHOR para mobile)
campo_email = driver.find_element('accessibility id', 'campo-email')
campo_email.send_keys('qa@teste.com')

campo_senha = driver.find_element('accessibility id', 'campo-senha')
campo_senha.send_keys('Senha@123')

btn_login = driver.find_element('accessibility id', 'btn-entrar')
btn_login.click()

# Verificar tela de sucesso
```

```python
titulo = driver.find_element('accessibility id', 'dashboard-titulo')
assert 'Dashboard' in titulo.text

driver.quit()
```

Cenários Reais de Mobile

Biometria (Fingerprint)	`driver.finger_print(1) # simula impressão digital`
GPS / Geolocalização	`driver.set_location(latitude=-19.9167, longitude=-43.9345, altitude=858)`
Swipe / Scroll	`driver.swipe(start_x=540, start_y=1200, end_x=540, end_y=400, duration=800)`
Push Notification	`adb shell am broadcast -a FCM_DIRECT_BOOT_ACTION`
Câmera Mock	`# Injetar imagem mockada via capabilities no Appium`

PARTE 10

ROBOT FRAMEWORK

Automação com linguagem próxima ao natural

O que é Robot Framework

Robot Framework é um framework de automação de testes de código aberto, baseado em Python, que usa uma sintaxe de palavras-chave (keywords) próxima ao inglês. Muito popular em equipes que precisam de automação legível para pessoas não-desenvolvedoras.

Quando Usar (e Quando Evitar)

USE quando...	EVITE quando...
Time misto (QA + negócio) precisa ler os testes	Time é 100% técnico e prefere código puro
Precisa integrar web + API + banco em 1 suite	Performance é crítica (Robot tem overhead)
Organização exige keywords em português/inglês	Você quer máximo controle de lógica complexa

Primeiro Teste Robot + SeleniumLibrary

```
# instalação
pip install robotframework robotframework-seleniumlibrary --break-system-packages


# tests/login.robot
*** Settings ***
Library SeleniumLibrary
Suite Setup Open Browser ${BASE_URL}/login chrome
Suite Teardown Close All Browsers

*** Variables ***
${BASE_URL} https://the-internet.herokuapp.com
${USUARIO} tomsmith
${SENHA} SuperSecretPassword!

*** Test Cases ***
Login Com Credenciais Validas
    [Documentation] Verifica login com usuario e senha corretos
    Input Text id:username ${USUARIO}
    Input Text id:password ${SENHA}
    Click Button css:button[type='submit']
    Page Should Contain You logged into a secure area!

Login Com Senha Incorreta
    Input Text id:username ${USUARIO}
    Input Text id:password senha_errada
```

```
    Click Button css:button[type='submit']
    Page Should Contain Your password is invalid!


*** Keywords ***
Verificar Mensagem De Sucesso
    [Arguments] ${mensagem}
    Element Should Contain id:flash ${mensagem}
```

PARTE 11

BDD — BEHAVIOR DRIVEN DEVELOPMENT

Gherkin · Cucumber · Testes que o negócio consegue ler

O que é BDD

BDD é uma abordagem onde testes são escritos como comportamentos esperados do sistema, em linguagem próxima ao negócio. A vantagem: QA, Dev e PO falam a mesma língua antes do código existir.

Gherkin — Sintaxe

```
# features/transferencia.feature
Funcionalidade: Transferência PIX
  Como usuario autenticado
  Quero transferir dinheiro via PIX
  Para pagar alguem instantaneamente

  Cenario: Transferencia PIX com saldo suficiente
    Dado que estou logado com saldo de R$500
    E tenho a chave PIX do destinatario '11987654321'
    Quando realizo transferencia de R$100
    Entao o comprovante deve ser exibido
    E meu saldo deve ser R$400

  Cenario: Transferencia com saldo insuficiente
    Dado que estou logado com saldo de R$50
    Quando tento transferir R$100
    Entao devo ver a mensagem 'Saldo insuficiente'
    E a transferencia nao deve ser processada

  Esquema do Cenario: Transferencias com valores limite
    Dado que estou logado com saldo de R$<saldo>
    Quando transferir R$<valor>
    Entao o resultado deve ser '<resultado>'
    Exemplos:
      | saldo | valor | resultado |
      | 1000 | 0 | erro |
      | 1000 | 1 | sucesso |
      | 1000 | 1000 | sucesso |
      | 1000 | 1001 | insuficiente |
```

PARTE 12

OUTRAS FERRAMENTAS IMPORTANTES

Performance · Segurança · Grid · Frameworks avançados

k6 — Performance Testing Moderno

k6 é a ferramenta de performance da geração atual. Scripts em JavaScript, integração nativa com CI/CD, métricas em tempo real.

```
// tests/performance/carga.js
import http from 'k6/http'
import { check, sleep } from 'k6'

export const options = {
  stages: [
    { duration: '30s', target: 10 }, // ramp up: 0 -> 10 usuários
    { duration: '1m', target: 50 }, // sustain: 50 usuários por 1 min
    { duration: '30s', target: 0 }, // ramp down: 50 -> 0
  ],
  thresholds: {
    http_req_duration: ['p(95)<500'], // 95% das req < 500ms
    http_req_failed: ['rate<0.01'], // menos de 1% de erros
  },
}

export default function () {
  const response = http.get('https://api.sistema.com/v1/produtos')

  check(response, {
    'status 200': (r) => r.status === 200,
    'tempo < 500ms': (r) => r.timings.duration < 500,
    'body não vazio': (r) => r.body.length > 0,
  })

  sleep(1)
}

# Rodar:
# k6 run tests/performance/carga.js
```

OWASP — Segurança para QA

Todo QA deve conhecer o OWASP Top 10 — os 10 riscos mais críticos de segurança em aplicações web. Você não precisa explorar vulnerabilidades, mas precisa saber testá-las no contexto da sua aplicação.

A01 - Broken Access Control	Teste: Testar que usuário A não acessa dados do usuário B
A02 - Cryptographic Failures	Teste: Verificar HTTPS em todos os endpoints sensíveis
A03 - Injection (SQL/XSS)	Teste: Input fields com payloads de SQL injection e XSS
A07 - Auth Failures	Teste: Senha fraca, brute force, tokens sem expiração
A09 - Logging/Monitoring	Teste: Verificar se falhas de login são registradas

PARTE 13

CARREIRA EM AUTOMAÇÃO

Roadmap · Stack · Entrevistas · Como chegar lá

Como Escolher Sua Stack

Java + Selenium	Mercado legado sólido. Bancos, seguradoras, grandes corporações. Mais verboso, mas ainda muito requisitado.
JS/TS + Playwright	Stack moderna e crescente. Melhor para SPAs, fintechs, startups. Recomendação QA Insider para quem está começando hoje.
Python + Playwright	Ideal para quem quer unir automação de testes com IA e automação de dados. Muito forte no mercado de dados e ciência de dados.
Robot Framework	Muito requisitado em empresas que precisam de automação legível. Governo, saúde, telecomunicações.

Roadmap: Júnior → Pleno → Sênior

JÚNIOR

- ■ Escrever testes simples de UI com Playwright/Cypress
- ■ Entender localizadores (id, CSS, XPath básico)
- ■ Executar testes em CI com GitHub Actions
- ■ Testes de API básicos com Postman e código
- ■ Conhecimento de Page Object Model
- ■ SQL básico: SELECT, WHERE, JOIN simples

PLENO

- ■ Framework completo com POM, fixtures, helpers
- ■ Estratégia de cobertura (pirâmide de testes)
- ■ Testes de API avançados com autenticação
- ■ Mobile com Appium (Android ou iOS)
- ■ BDD com Cucumber ou Robot Framework
- ■ Pipeline CI/CD completo com relatórios
- ■ SQL avançado: GROUP BY, JOINs, subqueries

SÊNIOR / LEAD

- ■ Arquitetura de framework do zero
- ■ Estratégia de qualidade por produto/squad
- ■ Shift-left: qualidade desde o refinamento
- ■ Performance testing com k6/JMeter
- ■ Noções de segurança (OWASP Top 10)
- ■ IA aplicada: geração de testes, análise de riscos
- ■ Mentoria e desenvolvimento do time de QA
- ■ Métricas de qualidade: defect escape rate, cobertura, MTTR

Entrevistas Técnicas — Perguntas Reais com Respostas

P: O que é Page Object Model e por que usar?

R: POM é um padrão de design onde cada página da aplicação é representada por uma classe que encapsula seus elementos e ações. O benefício principal é manutenibilidade: quando um localizador muda, você muda em 1 lugar, não em todos os testes.

P: Qual a diferença entre asserção hard e soft?

R: Hard assertion interrompe o teste ao falhar (padrão). Soft assertion acumula todas as falhas e reporta ao final, útil quando você quer ver todos os erros de um fluxo de uma vez.

P: Como você lida com flaky tests?

R: Identifico a causa raiz: timing (corrijo com waitFor explícito), dados compartilhados (torno os testes independentes), ou ambiente instável (mock ou retry isolado). Nunca adiciono sleep fixo como solução.

P: O que é a pirâmide de testes?

R: Modelo de distribuição: base larga de testes unitários (rápidos, baratos), camada do meio com testes de integração/API, e topo com poucos testes E2E (lentos, caros). O objetivo é ter a maior cobertura possível no nível mais barato.

P: Como você escolhe o que automatizar primeiro?

R: Priorizo pelo risco: fluxos mais críticos ao negócio, cenários com maior taxa histórica de bugs, e testes que são executados com maior frequência. Automatizo o que mais custa fazer manualmente.

■ SEÇÃO DE BÔNUS

Bônus 1	100 Exercícios Web (Playwright) com Gabarito
Bônus 2	100 Exercícios API com Gabarito
Bônus 3	50 Exercícios SQL para QA com Gabarito
Bônus 4	50 Exercícios Mobile (Appium) com Gabarito
Bônus 5	50 Prompts para IA — QA Insider Collection
Bônus 6	Checklist de Automação Profissional
Bônus 7	Glossário Completo QA + Automação (150+ termos)

100 Exercícios Web — Gabarito Completo

Todos os exercícios usam Playwright + JavaScript. Aplicação de referência: https://the-internet.herokuapp.com

Nível Júnior — Exercícios 001 a 040

#001 Navegar para uma URL e verificar o título da página — **Júnior**

Use page.goto() e page.title()

```
test('verificar título', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com')
  const titulo = await page.title()
  expect(titulo).toBe('The Internet')
})
```

#002 Clicar em um link pelo texto e verificar redirecionamento — **Júnior**

Use page.click('text=...')

```
test('clicar link por texto', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com')
  await page.click('text=Form Authentication')
  await expect(page).toHaveURL(/login/)
})
```

#003 Preencher um campo de texto e verificar o valor inserido — **Júnior**

Use page.fill() e page.inputValue()

```
test('preencher campo', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/login')
  await page.fill('#username', 'tomsmith')
  const valor = await page.inputValue('#username')
  expect(valor).toBe('tomsmith')
})
```

#004 Fazer login com credenciais válidas — **Júnior**

■ *Combine fill + click + expect*

```
test('login válido', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/login')
  await page.fill('#username', 'tomsmith')
  await page.fill('#password', 'SuperSecretPassword!')
  await page.click('button[type="submit"]')
  await expect(page.locator('#flash')).toContainText('You logged into')
})
```

#005 Verificar que mensagem de erro aparece com senha incorreta — **Júnior**

■ *Teste negativo — tão importante quanto o positivo*

```
test('login inválido', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/login')
  await page.fill('#username', 'tomsmith')
  await page.fill('#password', 'senha_errada')
  await page.click('button[type="submit"]')
  await expect(page.locator('#flash')).toContainText('Your password is invalid')
})
```

#006 Verificar que um elemento está visível na página — **Júnior**

■ *Use toBeVisible()*

```
test('elemento visível', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/login')
  await expect(page.locator('#username')).toBeVisible()
  await expect(page.locator('#password')).toBeVisible()
  await expect(page.locator('button[type="submit"]')).toBeVisible()
})
```

#007 Verificar que um elemento está desabilitado

Júnior

■ *Use toBeDisabled()*

```
test('botão desabilitado', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/dynamic_controls')
  const input = page.locator('form#input-example input')
  await expect(input).toBeDisabled()
})
```

#008 Clicar em um checkbox e verificar que está marcado

Júnior

■ *Use page.check() e toBeChecked()*

```
test('marcar checkbox', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/checkboxes')
  const cb = page.locator('input[type="checkbox"]').first()
  await cb.check()
  await expect(cb).toBeChecked()
})
```

#009 Selecionar uma opção em um dropdown pelo valor

Júnior

■ *Use page.selectOption()*

```
test('selecionar dropdown', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/dropdown')
  await page.selectOption('#dropdown', '2')
  const selecionado = await page.inputValue('#dropdown')
  expect(selecionado).toBe('2')
})
```

#010 Verificar o texto de um elemento específico

Júnior

■ *Use toContainText() ou toHaveText()*

```
test('verificar texto', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com')
  await expect(page.locator('h1')).toHaveText('Welcome to the-internet')
})
```

#011 Tirar screenshot de uma página

Júnior

■ *Use page.screenshot()*

```
test('screenshot', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com')
  await page.screenshot({ path: 'screenshots/homepage.png', fullPage: true })
  // verificar que arquivo foi criado
  const fs = require('fs')
  expect(fs.existsSync('screenshots/homepage.png')).toBe(true)
})
```

#012 Verificar a URL atual da página

Júnior

■ *Use expect(page).toHaveURL()*

```
test('verificar URL', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/login')
  await page.fill('#username', 'tomsmith')
  await page.fill('#password', 'SuperSecretPassword!')
  await page.click('button[type="submit"]')
  await expect(page).toHaveURL(/secure/)
})
```

#013 Contar quantos elementos de uma lista existem

Júnior

■ *Use locator.count()*

```
test('contar elementos', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/checkboxes')
  const checkboxes = page.locator('input[type="checkbox"]')
  const quantidade = await checkboxes.count()
  expect(quantidade).toBe(2)
})
```

#014 Verificar que um elemento não está visível

Júnior

■ *Use toBeHidden() ou not.toBeVisible()*

```
test('elemento oculto', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/dynamic_loading/1')
  const resultado = page.locator('#finish')
  await expect(resultado).toBeHidden()
})
```

#015 Obter e verificar um atributo de elemento

Júnior

■ *Use locator.getAttribute()*

```
test('verificar atributo', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/login')
  const tipo = await page.locator('#password').getAttribute('type')
  expect(tipo).toBe('password')
})
```

#016 Navegar para trás e para frente no histórico

Júnior

■ *Use page.goBack() e page.goForward()*

```
test('navegação histórico', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com')
  await page.goto('https://the-internet.herokuapp.com/login')
  await page.goBack()
  await expect(page).toHaveURL('https://the-internet.herokuapp.com/')
  await page.goForward()
  await expect(page).toHaveURL(/login/)
})
```

#017 Limpar um campo de texto antes de preencher — Júnior

■ *Use page.fill() — ele já limpa antes de digitar*

```
test('limpar e preencher', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/login')
  await page.fill('#username', 'usuario_errado')
  await page.fill('#username', 'tomsmith') // substitui automaticamente
  const valor = await page.inputValue('#username')
  expect(valor).toBe('tomsmith')
})
```

#018 Verificar que a página contém um texto qualquer — Júnior

■ *Use page.locator('body').toContainText()*

```
test('texto na página', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com')
  await expect(page.locator('body')).toContainText('Welcome')
})
```

#019 Acessar o segundo item de uma lista de elementos — Júnior

■ *Use locator.nth(1) — índice começa em 0*

```
test('segundo elemento', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/checkboxes')
  const segundo = page.locator('input[type="checkbox"]').nth(1)
  await expect(segundo).toBeChecked() // segundo já vem marcado
})
```

#020 Verificar o número de links em uma página — Júnior

■ *Use page.locator('a').count()*

```
test('contar links', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com')
  const links = page.locator('li a')
  const total = await links.count()
  expect(total).toBeGreaterThan(10)
})
```

#021 Fazer upload de um arquivo via input file — Júnior

■ *Use locator.setInputFiles()*

```
test('upload de arquivo', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/upload')
  await page.setInputFiles('#file-upload', './fixtures/teste.txt')
  await page.click('#file-submit')
  await expect(page.locator('#uploaded-files')).toContainText('teste.txt')
})
```

#022 Verificar o título de uma aba (tab) do navegador — Júnior

■ *Use page.title()*

```
test('título da aba', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/login')
  const titulo = await page.title()
  expect(titulo).toContain('The Internet')
})
```

#023 Aguardar um elemento aparecer dinamicamente — Júnior

■ *Use waitFor() com timeout explícito*

```
test('esperar elemento dinâmico', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/dynamic_loading/1')
  await page.click('button')
  await page.locator('#finish').waitFor({ state: 'visible', timeout: 10000 })
  await expect(page.locator('#finish')).toContainText('Hello World!')
})
```

#024 Verificar cor de um elemento com CSS

Júnior

■ *Use evaluate() para acessar estilos computados*

```
test('verificar cor CSS', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/login')
  const cor = await page.locator('button[type="submit"]').evaluate(el => {
    return window.getComputedStyle(el).backgroundColor
  })
  expect(cor).toBeTruthy() // verificar que tem cor definida
})
```

#025 Simular pressionamento de tecla Enter

Júnior

■ *Use page.press(locator, 'Enter')*

```
test('pressionar Enter', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/login')
  await page.fill('#username', 'tomsmith')
  await page.fill('#password', 'SuperSecretPassword!')
  await page.locator('#password').press('Enter')
  await expect(page.locator('#flash')).toContainText('You logged into')
})
```

#026 Rolar a página até o rodapé

Júnior

■ *Use page.evaluate() com window.scrollTo*

```
test('scroll até rodapé', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com')
  await page.evaluate(() => window.scrollTo(0, document.body.scrollHeight))
  const footer = page.locator('footer')
  await expect(footer).toBeVisible()
})
```

#027 Verificar que um link abre em nova aba (_blank) — Júnior

Use page.locator e getAttribute('target')

```
test('link nova aba', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com')
  // verificar atributo target no link
  const links = page.locator('a[target="_blank"]')
  const total = await links.count()
  console.log(`Links com _blank: ${total}`)
})
```

#028 Aceitar um alert JavaScript — Júnior

Use page.on('dialog') para interceptar

```
test('aceitar alert', async ({ page }) => {
  page.on('dialog', dialog => dialog.accept())
  await page.goto('https://the-internet.herokuapp.com/javascript_alerts')
  await page.click('button[onclick="jsAlert()"]')
  await expect(page.locator('#result')).toContainText('You successfuly clicked')
})
```

#029 Confirmar ou cancelar um confirm JavaScript — Júnior

Use dialog.accept() ou dialog.dismiss()

```
test('cancelar confirm', async ({ page }) => {
  page.on('dialog', dialog => dialog.dismiss())
  await page.goto('https://the-internet.herokuapp.com/javascript_alerts')
  await page.click('button[onclick="jsConfirm()"]')
  await expect(page.locator('#result')).toContainText('You clicked: Cancel')
})
```

#030 Preencher um prompt JavaScript com texto

Júnior

■ *Use dialog.accept(texto) para enviar valor*

```
test('preencher prompt JS', async ({ page }) => {
  page.on('dialog', dialog => dialog.accept('QA Insider'))
  await page.goto('https://the-internet.herokuapp.com/javascript_alerts')
  await page.click('button[onclick="jsPrompt()"]')
  await expect(page.locator('#result')).toContainText('QA Insider')
})
```

#031 Testar hover em um elemento

Júnior

■ *Use locator.hover()*

```
test('hover em elemento', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/hovers')
  const figura = page.locator('.figure').first()
  await figura.hover()
  await expect(figura.locator('.figcaption')).toBeVisible()
})
```

#032 Verificar múltiplos atributos de uma tabela HTML

Júnior

■ *Use locator.nth() para acessar células*

```
test('dados de tabela', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/tables')
  const primeiraLinha = page.locator('#table1 tbody tr').first()
  const sobrenome = await primeiraLinha.locator('td').first().textContent()
  expect(sobrenome).toBeTruthy()
})
```

#033 Verificar que um iframe existe e contém elemento

Júnior

■ *Use frameLocator() para acessar iframe*

```
test('conteúdo em iframe', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/iframe')
  const frame = page.frameLocator('#mce_0_ifr')
  const body = frame.locator('body')
  await expect(body).toBeVisible()
})
```

#034 Verificar que página retorna erro 404 para URL inválida

Júnior

■ *Use page.goto() e verificar response.status()*

```
test('página 404', async ({ page }) => {
  const response = await
page.goto('https://the-internet.herokuapp.com/pagina-inexistente')
  expect(response.status()).toBe(404)
})
```

#035 Usar variáveis de ambiente em testes

Júnior

■ *Use process.env para não hardcodar credenciais*

```
// playwright.config.js
// use: { baseURL: process.env.BASE_URL }

test('login com env vars', async ({ page }) => {
  const email = process.env.TEST_USER || 'tomsmith'
  const senha = process.env.TEST_PASS || 'SuperSecretPassword!'
  await page.goto('/login')
  await page.fill('#username', email)
  await page.fill('#password', senha)
  await page.click('button[type="submit"]')
  await expect(page.locator('#flash')).toContainText('You logged into')
})
```

#036 Executar o mesmo teste em múltiplos navegadores — Júnior

■ *Configure projects no playwright.config.js*

```
// playwright.config.js
module.exports = {
  projects: [
    { name: 'chromium', use: { browserName: 'chromium' } },
    { name: 'firefox', use: { browserName: 'firefox' } },
    { name: 'webkit', use: { browserName: 'webkit' } },
  ]
}

// Rode com:
// npx playwright test --project=chromium
// npx playwright test (roda nos 3)
```

#037 Usar beforeEach para setup comum entre testes — Júnior

■ *Evite repetição de código de setup*

```
test.describe('Área de Login', () => {
  test.beforeEach(async ({ page }) => {
    await page.goto('https://the-internet.herokuapp.com/login')
  })

  test('login válido', async ({ page }) => {
    await page.fill('#username', 'tomsmith')
    await page.fill('#password', 'SuperSecretPassword!')
    await page.click('button[type="submit"]')
    await expect(page.locator('#flash')).toContainText('You logged into')
  })

  test('campo obrigatório', async ({ page }) => {
    await page.click('button[type="submit"]')
    await expect(page.locator('#flash')).toBeVisible()
  })
})
```

#038 Testar responsividade em viewport mobile

Júnior

■ *Use page.setViewportSize()*

```
test('layout mobile 375px', async ({ page }) => {
  await page.setViewportSize({ width: 375, height: 812 })
  await page.goto('https://the-internet.herokuapp.com')
  await expect(page.locator('h1')).toBeVisible()
  await page.screenshot({ path: 'screenshots/mobile.png' })
})
```

#039 Verificar que elementos aparecem na ordem correta

Júnior

■ *Use locator.nth() para verificar sequência*

```
test('ordem dos links', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com')
  const links = page.locator('ul li a')
  const primeiro = await links.nth(0).textContent()
  const segundo = await links.nth(1).textContent()
  expect(primeiro).toBeTruthy()
  expect(primeiro).not.toBe(segundo)
})
```

#040 Fazer logout após login e verificar redirecionamento

Júnior

■ *Complete o fluxo completo login → ação → logout*

```
test('login e logout completo', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/login')
  await page.fill('#username', 'tomsmith')
  await page.fill('#password', 'SuperSecretPassword!')
  await page.click('button[type="submit"]')
  await expect(page).toHaveURL(/secure/)

  await page.click('text=Logout')
  await expect(page).toHaveURL(/login/)
  await expect(page.locator('#flash')).toContainText('You logged out')
})
```

Nível Pleno — Exercícios 041 a 080

#041 Criar Page Object para a página de Login — **Pleno**

■ *Encapsule seletores e ações — o teste não deve saber como a página funciona*

```
// pages/LoginPage.js
class LoginPage {
  constructor(page) {
    this.page = page
    this.campoUsuario = page.locator('#username')
    this.campoSenha = page.locator('#password')
    this.btnSubmit = page.locator('button[type="submit"]')
    this.mensagem = page.locator('#flash')
  }
  async navegar() { await this.page.goto('/login') }
  async login(user, pass) {
    await this.campoUsuario.fill(user)
    await this.campoSenha.fill(pass)
    await this.btnSubmit.click()
  }
  async getMensagem() { return this.mensagem.textContent() }
}
module.exports = { LoginPage }
```

#042 Usar fixtures do Playwright para estado autenticado

Pleno

Evite refazer login em cada teste — use storageState

```
// auth.setup.js
import { test as setup } from '@playwright/test'
setup('autenticar', async ({ page }) => {
  await page.goto('/login')
  await page.fill('#username', 'tomsmith')
  await page.fill('#password', 'SuperSecretPassword!')
  await page.click('button[type="submit"]')
  await page.context().storageState({ path: '.auth/user.json' })
})

// playwright.config.js
projects: [
  { name: 'setup', testMatch: /auth.setup/ },
  { name: 'testes', use: { storageState: '.auth/user.json' },
    dependsOn: ['setup'] }
]
```

#043 Parametrizar testes com test.each / data-driven

Pleno

Teste o mesmo fluxo com múltiplos conjuntos de dados

```
const credenciais = [
  { user: 'tomsmith', pass: 'SuperSecretPassword!', esperado: 'You logged into' },
  { user: 'tomsmith', pass: 'errada', esperado: 'Your password is invalid' },
  { user: '', pass: '', esperado: 'Your username is invalid' },
]

for (const { user, pass, esperado } of credenciais) {
  test(`login: '${user}' / '${pass}'`, async ({ page }) => {
    await page.goto('/login')
    await page.fill('#username', user)
    await page.fill('#password', pass)
    await page.click('button[type="submit"]')
    await expect(page.locator('#flash')).toContainText(esperado)
  })
}
```

#044 Interceptar requisições de rede e verificar payload

Pleno

Use page.route() ou page.waitForRequest()

```
test('interceptar requisição de login', async ({ page }) => {
  const [request] = await Promise.all([
    page.waitForRequest(req => req.url().includes('/login') && req.method() === 'POST'),
    (async () => {
      await page.goto('/login')
      await page.fill('#username', 'tomsmith')
      await page.fill('#password', 'SuperSecretPassword!')
      await page.click('button[type="submit"]')
    })()
  ])
  expect(request.method()).toBe('POST')
})
```

#045 Mockar uma resposta de API no teste E2E

Pleno

Use page.route() para interceptar e retornar mock

```
test('mock de API', async ({ page }) => {
  await page.route('**/api/usuario/perfil', async route => {
    await route.fulfill({
      status: 200,
      contentType: 'application/json',
      body: JSON.stringify({ nome: 'Lilian QA', perfil: 'admin' })
    })
  })
  await page.goto('/dashboard')
  await expect(page.locator('.user-name')).toContainText('Lilian QA')
})
```

#046 Testar drag and drop

Pleno

Use dragTo() ou mouse.down/move/up

```
test('drag and drop', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/drag_and_drop')
  const origem = page.locator('#column-a')
  const destino = page.locator('#column-b')
  await origem.dragTo(destino)
  await expect(page.locator('#column-a header')).toHaveText('B')
  await expect(page.locator('#column-b header')).toHaveText('A')
})
```

#047 Testar múltiplas janelas/abas simultaneamente

Pleno

Use context.waitForEvent('page') para capturar nova aba

```
test('nova aba ao clicar link', async ({ page, context }) => {
  await page.goto('https://the-internet.herokuapp.com/windows')
  const [novaAba] = await Promise.all([
    context.waitForEvent('page'),
    page.click('text=Click Here')
  ])
  await novaAba.waitForLoadState()
  await expect(novaAba).toHaveURL(/window/)
  await expect(novaAba.locator('h3')).toHaveText('New Window')
})
```

#048 Testar campos de formulário com validação HTML5

Pleno

Verifique mensagens de validação nativas do browser

```
test('validação HTML5 campo obrigatório', async ({ page }) => {
  await page.goto('/cadastro')
  await page.click('#btn-salvar') // submete sem preencher
  const validationMsg = await page.locator('#nome').evaluate(el => {
    return el.validationMessage
  })
  expect(validationMsg).toBeTruthy() // browser mostra mensagem
})
```

#049 Verificar acessibilidade básica com axe-core

Pleno

Instale @axe-core/playwright

```
// npm install @axe-core/playwright
import { checkA11y } from '@axe-core/playwright'

test('acessibilidade homepage', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com')
  await checkA11y(page, undefined, {
    detailedReport: true,
    detailedReportOptions: { html: true }
  })
})
```

#050 Usar soft assertions para verificar múltiplos pontos

Pleno

Não interrompe no primeiro erro — vê todos os problemas

```
test('soft assertions no formulário', async ({ page }) => {
  await page.goto('/dashboard')
  await expect.soft(page.locator('h1')).toBeVisible()
  await expect.soft(page.locator('.menu-principal')).toBeVisible()
  await expect.soft(page.locator('.user-info')).toContainText('@')
  await expect.soft(page.locator('.btn-logout')).toBeEnabled()
  // Todos os erros são reportados juntos ao final
})
```

#051 Extrair todos os textos de uma tabela para validação

Pleno

Use locator.allTextContents()

```
test('extrair dados de tabela', async ({ page }) => {
  await page.goto('https://the-internet.herokuapp.com/tables')
  const celulas = page.locator('#table1 tbody tr:first-child td')
  const textos = await celulas.allTextContents()
  console.log('Dados da linha:', textos)
  expect(textos.length).toBeGreaterThan(0)
  expect(textos[0]).toBeTruthy() // sobrenome não vazio
})
```

#052 Testar campo de busca com vários termos

Pleno

Data-driven: mesmo fluxo, múltiplos dados

```
const termos = ['Playwright', 'Cypress', 'Selenium', 'Appium']


for (const termo of termos) {
  test(`busca por: ${termo}`, async ({ page }) => {
    await page.goto('/busca')
    await page.fill('[data-testid="campo-busca"]', termo)
    await page.press('[data-testid="campo-busca"]', 'Enter')
    await expect(page.locator('.resultados')).toBeVisible()
    await expect(page.locator('.resultado-item').first()).toBeVisible()
  })
}
```

#053 Implementar retry automático para elemento instável

Pleno

Use expect com timeout customizado como alternativa a sleep

```
test('elemento com carregamento lento', async ({ page }) => {
  await page.goto('/relatorio')
  await page.click('#btn-gerar-relatorio')
  // Espera até 30s para o relatório aparecer — sem sleep fixo
  await expect(page.locator('#tabela-relatorio')).toBeVisible({
    timeout: 30_000
  })
  const linhas = await page.locator('#tabela-relatorio tbody tr').count()
  expect(linhas).toBeGreaterThan(0)
})
```

#054 Testar fluxo completo de carrinho de compras

Pleno

■ *Teste E2E multi-passo — o mais próximo do usuário real*

```
test('adicionar ao carrinho e checkout', async ({ page }) => {
  await page.goto('/produtos')
  await page.click('[data-testid="produto-001"] .btn-adicionar')
  await expect(page.locator('.badge-carrinho')).toContainText('1')

  await page.click('.icon-carrinho')
  await expect(page.locator('.item-carrinho')).toHaveCount(1)

  await page.click('#btn-checkout')
  await page.fill('#numero-cartao', '4111111111111111')
  await page.fill('#validade', '12/28')
  await page.fill('#cvv', '123')
  await page.click('#btn-confirmar')

  await expect(page.locator('.confirmacao-pedido')).toBeVisible()
  await expect(page.locator('.numero-pedido')).toBeVisible()
})
```

#055 Testar infinite scroll

Pleno

■ *Simule scroll para baixo e verifique carregamento*

```
test('infinite scroll carrega mais itens', async ({ page }) => {
  await page.goto('/feed')
  const contagem_inicial = await page.locator('.item-feed').count()

  // Scroll até o final da página
  await page.evaluate(() => window.scrollTo(0, document.body.scrollHeight))
  await page.waitForTimeout(1500) // aguarda carregamento

  const contagem_final = await page.locator('.item-feed').count()
  expect(contagem_final).toBeGreaterThan(contagem_inicial)
})
```

#056 Testar download de arquivo — Pleno

■ *Use page.waitForEvent('download')*

```
test('download de relatório PDF', async ({ page }) => {
  await page.goto('/relatorios')
  const [download] = await Promise.all([
    page.waitForEvent('download'),
    page.click('#btn-exportar-pdf')
  ])
  expect(download.suggestedFilename()).toContain('.pdf')
  await download.saveAs('./downloads/' + download.suggestedFilename())
})
```

#057 Verificar que e-mail foi enviado (via API do serviço de email) — Pleno

■ *Use request fixture para chamar API do Mailtrap ou similar*

```
test('verificar e-mail de boas-vindas', async ({ request }) => {
  // Após cadastro, verificar no Mailtrap
  const response = await
request.get('https://api.mailtrap.io/api/v1/inboxes/ID/messages', {
    headers: { 'Api-Token': process.env.MAILTRAP_TOKEN }
  })
  const emails = await response.json()
  const bemVindo = emails.find(e => e.subject.includes('Bem-vindo'))
  expect(bemVindo).toBeDefined()
})
```

#058 Testar autenticação 2FA via TOTP

Pleno

Use otplib para gerar código TOTP no teste

```
// npm install otplib
import { totp } from 'otplib'

test('login com 2FA', async ({ page }) => {
  await page.goto('/login')
  await page.fill('#email', process.env.USER_2FA)
  await page.fill('#senha', process.env.PASS_2FA)
  await page.click('#btn-entrar')

  const codigo = totp.generate(process.env.TOTP_SECRET)
  await page.fill('#campo-totp', codigo)
  await page.click('#btn-verificar')
  await expect(page).toHaveURL('/dashboard')
})
```

#059 Criar helper para gerar dados de teste aleatórios **Pleno**

■ *Isola geração de dados da lógica de teste*

```
// helpers/faker.js
function gerarUsuario() {
  const ts = Date.now()
  return {
    nome: `Usuário Teste ${ts}`,
    email: `teste.${ts}@qa.local`,
    senha: 'Senha@' + Math.random().toString(36).slice(2, 8).toUpperCase(),
  }
}

// No teste:
test('cadastro com dados únicos', async ({ page }) => {
  const usuario = gerarUsuario()
  await page.goto('/cadastro')
  await page.fill('#nome', usuario.nome)
  await page.fill('#email', usuario.email)
  await page.fill('#senha', usuario.senha)
  await page.click('#btn-cadastrar')
  await expect(page).toHaveURL('/dashboard')
})
```

#060 Testar paginação de uma listagem

Pleno

Navegue entre páginas e verifique conteúdo diferente

```
test('paginação de resultados', async ({ page }) => {
  await page.goto('/clientes?pagina=1')
  const primeiroNome = await page.locator('.cliente-nome').first().textContent()

  await page.click('[data-testid="btn-proxima-pagina"]')
  await expect(page).toHaveURL(/pagina=2/)

  const primeiroNomePag2 = await
page.locator('.cliente-nome').first().textContent()
  expect(primeiroNomePag2).not.toBe(primeiroNome)
})
```

#061 Testar comportamento de sessão expirada

Pleno

Simule expiração deletando cookies e verifique redirect

```
test('sessão expirada redireciona para login', async ({ page, context }) => {
  // Fazer login normalmente
  await page.goto('/login')
  await page.fill('#username', 'tomsmith')
  await page.fill('#password', 'SuperSecretPassword!')
  await page.click('button[type="submit"]')
  await expect(page).toHaveURL(/secure/)

  // Expirar sessão deletando cookies
  await context.clearCookies()

  // Tentar acessar área restrita
  await page.goto('/secure')
  await expect(page).toHaveURL(/login/)
})
```

#062 Verificar que campos obrigatórios são sinalizados visualmente

Pleno

■ *Borda vermelha ou ícone de erro — acessibilidade exige feedback visual*

```
test('campo obrigatório sinalizado ao submeter', async ({ page }) => {
  await page.goto('/cadastro')
  await page.click('#btn-salvar') // submete vazio

  const campo = page.locator('#nome')
  // Verificar classe CSS de erro
  const classes = await campo.getAttribute('class')
  expect(classes).toContain('error') // ou 'invalid', 'campo-erro'

  // Verificar que mensagem de erro está visível ao lado
  const msgErro = page.locator('#nome-error, [for="nome"].error')
  await expect(msgErro).toBeVisible()
})
```

#063 Testar shortcut de teclado na aplicação

Pleno

■ *Shortcuts são funcionalidade — devem ser testados*

```
test('Ctrl+K abre modal de busca', async ({ page }) => {
  await page.goto('/dashboard')
  await page.keyboard.press('Control+k')

  const modalBusca = page.locator('[role="dialog"][aria-label="Busca"]')
  await expect(modalBusca).toBeVisible()

  // Fechar com Escape
  await page.keyboard.press('Escape')
  await expect(modalBusca).toBeHidden()
})
```

#064 Verificar comportamento de autocomplete

Pleno

Autocomplete tem timing — use waitFor ao invés de sleep

```
test('autocomplete de cidade', async ({ page }) => {
  await page.goto('/endereco')
  await page.fill('#cidade', 'Belo')

  // Aguardar dropdown aparecer
  const dropdown = page.locator('.autocomplete-dropdown')
  await expect(dropdown).toBeVisible()

  // Verificar que sugestões contêm o termo digitado
  const opcoes = page.locator('.autocomplete-item')
  const primeira = await opcoes.first().textContent()
  expect(primeira?.toLowerCase()).toContain('belo')

  // Selecionar primeira opção
  await opcoes.first().click()
  const valor = await page.inputValue('#cidade')
  expect(valor).toBeTruthy()
})
```

#065 Testar upload múltiplo de arquivos

Pleno

■ *setInputFiles aceita array — use para múltiplos arquivos*

```
test('upload múltiplos arquivos', async ({ page }) => {
  await page.goto('/documentos')
  await page.setInputFiles('#file-input', [
    './fixtures/doc1.pdf',
    './fixtures/doc2.pdf',
    './fixtures/imagem.jpg',
  ])

  // Verificar preview de cada arquivo
  const previews = page.locator('.file-preview')
  await expect(previews).toHaveCount(3)

  await page.click('#btn-enviar')
  await expect(page.locator('.upload-success')).toBeVisible()
})
```

#066 Verificar ordenação de tabela ao clicar no cabeçalho **Pleno**

■ *Sorting é funcionalidade de negócio — sempre teste*

```
test('ordenar tabela por nome', async ({ page }) => {
  await page.goto('/clientes')

  // Capturar primeira linha antes de ordenar
  const antesOrdenacao = await page.locator('tbody tr:first-child
td:nth-child(2)').textContent()

  // Clicar para ordenar por nome A-Z
  await page.click('th[data-col="nome"]')
  await page.locator('.sort-asc').waitFor()

  const depoisAsc = await page.locator('tbody tr:first-child
td:nth-child(2)').textContent()

  // Clicar novamente para Z-A
  await page.click('th[data-col="nome"]')
  const depoisDesc = await page.locator('tbody tr:last-child
td:nth-child(2)').textContent()

  // A-Z e Z-A devem ser diferentes
  expect(depoisAsc).not.toBe(depoisDesc)
})
```

#067 Testar filtros combinados em listagem

Pleno

Múltiplos filtros ativos ao mesmo tempo

```
test('filtros combinados: status + data', async ({ page }) => {
  await page.goto('/pedidos')

  // Aplicar filtro de status
  await page.selectOption('#filtro-status', 'APROVADO')

  // Aplicar filtro de data
  await page.fill('#data-inicio', '2025-01-01')
  await page.fill('#data-fim', '2025-12-31')
  await page.click('#btn-filtrar')

  // Verificar que todos os resultados batem com os filtros
  const status = page.locator('.status-pedido')
  const total = await status.count()
  for (let i = 0; i < total; i++) {
    const texto = await status.nth(i).textContent()
    expect(texto).toContain('Aprovado')
  }
})
```

#068 Verificar que modal fecha ao clicar fora

Pleno

Comportamento esperado pelo usuário — mas quebra em muitos sistemas

```
test('modal fecha ao clicar no backdrop', async ({ page }) => {
  await page.goto('/produtos')
  await page.click('#btn-ver-detalhes')
  await expect(page.locator('.modal')).toBeVisible()

  // Clicar no backdrop (fora do modal)
  await page.locator('.modal-backdrop').click({ position: { x: 5, y: 5 } })

  await expect(page.locator('.modal')).toBeHidden()
})
```

#069 Testar comportamento de timezone no datepicker **Pleno**

Datas são a fonte de bug mais subestimada em sistemas

```
test('datepicker exibe fuso horário correto', async ({ page }) => {
  // Simular timezone de Brasília
  await page.emulateMedia({ colorScheme: 'dark' })
  await page.addInitScript(() => {
    // Não é possível mudar timezone via JS diretamente
    // Use '--timezone' flag no playwright.config ou env TZ
  })

  await page.goto('/agendamento')
  await page.click('#data-agendamento')

  const hoje = new Date().toLocaleDateString('pt-BR')
  const diaAtivo = page.locator('.datepicker-day.today')
  await expect(diaAtivo).toBeVisible()

  // playwright.config.js: use: { timezoneId: 'America/Sao_Paulo' }
})
```

#070 Testar componente de rating/estrelas

Pleno

Componentes customizados exigem hover antes do click

```
test('avaliar produto com 4 estrelas', async ({ page }) => {
  await page.goto('/produto/123')

  const estrelas = page.locator('.star-rating .star')
  await expect(estrelas).toHaveCount(5)

  // Clicar na 4ª estrela
  await estrelas.nth(3).click()

  // Verificar que 4 estrelas ficaram ativas
  const ativas = page.locator('.star-rating .star.active')
  await expect(ativas).toHaveCount(4)

  // Verificar valor enviado ao backend via input hidden
  const valor = await page.locator('input[name="rating"]').inputValue()
  expect(valor).toBe('4')
})
```

#071 Testar comportamento offline com Service Worker

Pleno

■ *PWAs devem funcionar offline — verifique o fallback*

```
test('exibe página offline quando sem rede', async ({ page, context }) => {
  // Aguardar SW registrar
  await page.goto('/')
  await page.waitForFunction(() => navigator.serviceWorker.controller !== null)

  // Simular offline
  await context.setOffline(true)

  await page.reload()

  // App deve mostrar conteúdo cacheado ou página offline
  const offline = page.locator('.offline-indicator,
[data-testid="offline-banner"]')
  await expect(offline).toBeVisible({ timeout: 5000 })

  await context.setOffline(false)
})
```

#072	Verificar que formulário salva rascunho automaticamente	Pleno

■ *Autosave é funcionalidade crítica em formulários longos*

```
test('autosave a cada 30 segundos', async ({ page }) => {
  await page.goto('/proposta/nova')
  await page.fill('#titulo', 'Proposta Autosave Test')
  await page.fill('#descricao', 'Conteúdo do rascunho')

  // Aguardar indicador de autosave aparecer
  const indicador = page.locator('.autosave-indicator')
  await expect(indicador).toContainText('Salvo', { timeout: 35_000 })

  // Recarregar e verificar que rascunho foi mantido
  await page.reload()
  const titulo = await page.inputValue('#titulo')
  expect(titulo).toBe('Proposta Autosave Test')
})
```

#073 Testar impressão com CSS @media print

Pleno

Verifique layout de impressão antes de reclamação do cliente

```
test('layout de impressão oculta elementos corretos', async ({ page }) => {
  await page.goto('/relatorio/123')

  // Emular mídia de impressão
  await page.emulateMedia({ media: 'print' })

  // Elementos que devem sumir na impressão
  const menu = page.locator('nav.menu-principal')
  const sidebar = page.locator('.sidebar')
  await expect(menu).toBeHidden()
  await expect(sidebar).toBeHidden()

  // Conteúdo principal deve estar visível
  const conteudo = page.locator('.relatorio-conteudo')
  await expect(conteudo).toBeVisible()

  // Screenshot para revisão visual
  await page.screenshot({ path: 'screenshots/print-preview.png', fullPage: true })
})
```

#074 Testar que tabela exporta corretamente para CSV

Pleno

Exportações de dados têm bugs de encoding e formatação

```
test('exportar tabela para CSV', async ({ page }) => {
  await page.goto('/relatorios/clientes')

  const [download] = await Promise.all([
    page.waitForEvent('download'),
    page.click('#btn-exportar-csv')
  ])

  expect(download.suggestedFilename()).toMatch(/clientes.*\.csv$/i)

  // Salvar e verificar conteúdo
  const path = './downloads/clientes.csv'
  await download.saveAs(path)

  const fs = require('fs')
  const conteudo = fs.readFileSync(path, 'utf-8')

  // Verificar cabeçalho do CSV
  const primeiraLinha = conteudo.split('\n')[0]
  expect(primeiraLinha).toContain('nome')
  expect(primeiraLinha).toContain('email')

  // Verificar encoding — acentos não devem ser corrompidos
  // encoding correto: acentos preservados (sem caracteres corrompidos)
})
```

#075	Verificar que página lida com parâmetros de URL maliciosos	Pleno

■ *XSS via URL é um dos vetores mais comuns*

```
test('parâmetros maliciosos na URL não quebram a página', async ({ page }) => {
  const payloads = [
    '?id=<script>alert(1)</script>',
    '?search=%27%20OR%20%271%27%3D%271', // SQL injection encoded
    '?redirect=javascript:alert(1)',
  ]

  for (const payload of payloads) {
    const res = await page.goto(`https://the-internet.herokuapp.com/${payload}`)
    // Não deve causar erro 500
    expect(res?.status()).not.toBe(500)
    // Não deve executar alert — verificar que página carregou normalmente
    await expect(page.locator('body')).toBeVisible()
  }
})
```

#076 Testar lazy loading de imagens

Pleno

Imagens com loading=lazy só carregam quando visíveis

```
test('imagens carregam ao entrar no viewport', async ({ page }) => {
  await page.goto('/galeria')

  // Imagem no final da página — deve ter src vazio inicialmente
  const ultimaImagem = page.locator('.galeria-item img').last()
  const srcInicial = await ultimaImagem.getAttribute('src')

  // Scroll até a imagem
  await ultimaImagem.scrollIntoViewIfNeeded()
  await page.waitForTimeout(500) // aguardar carregamento

  const srcFinal = await ultimaImagem.getAttribute('src')
  expect(srcFinal).not.toBe(srcInicial) // src mudou após scroll
  expect(srcFinal).toContain('http') // tem URL real agora
})
```

#077 Testar que notificação toast aparece e desaparece

Pleno

Toasts têm lifecycle — aparece, fica, desaparece

```
test('toast de sucesso aparece e desaparece automaticamente', async ({ page }) => {
  await page.goto('/configuracoes')
  await page.click('#btn-salvar-configuracoes')

  const toast = page.locator('.toast-sucesso, [role="alert"]')

  // Deve aparecer
  await expect(toast).toBeVisible({ timeout: 3000 })
  expect(await toast.textContent()).toContain('salvo')

  // Deve desaparecer automaticamente
  await expect(toast).toBeHidden({ timeout: 8000 })
})
```

#078 Testar busca com resultados zerados

Pleno

Empty state é UX — deve ter mensagem amigável, não tela em branco

```
test('busca sem resultado exibe empty state', async ({ page }) => {
  await page.goto('/produtos')
  await page.fill('#campo-busca', 'xyzprodutoinexistente123')
  await page.press('#campo-busca', 'Enter')

  // Verificar empty state
  const emptyState = page.locator('.empty-state, [data-testid="sem-resultados"]')
  await expect(emptyState).toBeVisible()

  // Mensagem deve ser amigável
  const texto = await emptyState.textContent()
  expect(texto).not.toBe('')
  expect(texto?.toLowerCase()).toMatch(/nenhum|não encontrado|sem resultado/)
})
```

#079 Testar comportamento ao perder foco em campo com validação **Pleno**

■ *Validação onBlur é diferente de onSubmit — teste os dois*

```
test('validação ao sair do campo de email', async ({ page }) => {
  await page.goto('/cadastro')

  // Preencher email inválido
  await page.fill('#email', 'email-invalido')

  // Sair do campo (blur)
  await page.locator('#email').blur()

  // Mensagem deve aparecer imediatamente (onBlur)
  const erro = page.locator('#email-error, .email-error-msg')
  await expect(erro).toBeVisible()
  expect(await erro.textContent()).toMatch(/inválido|formato/i)

  // Corrigir o email
  await page.fill('#email', 'correto@email.com')
  await page.locator('#email').blur()

  // Erro deve sumir
  await expect(erro).toBeHidden()
})
```

#080	Testar feature flag — funcionalidade ativa apenas para grupos	Pleno

■ *Feature flags mal testadas = rollout quebrado em produção*

```
test('feature nova visível apenas para usuário beta', async ({ context }) => {
  // Usuário beta — deve ver a feature
  const pageBeta = await context.newPage()
  await pageBeta.addInitScript(() => {
    localStorage.setItem('feature_flags', JSON.stringify({ nova_dashboard: true }))
  })
  await pageBeta.goto('/dashboard')
  await expect(pageBeta.locator('[data-feature="nova_dashboard"]')).toBeVisible()

  // Usuário normal — não deve ver
  const pageNormal = await context.newPage()
  await pageNormal.goto('/dashboard')
  await expect(pageNormal.locator('[data-feature="nova_dashboard"]')).toBeHidden()

  await pageBeta.close()
  await pageNormal.close()
})
```

Nível Sênior — Exercícios 081 a 100

#081 Implementar framework base com BaseTest e configurações globais **Sênior**

■ *Encapsule setup/teardown e configurações compartilhadas*

```
// fixtures/base.fixture.js
import { test as base } from '@playwright/test'
import { LoginPage } from '../pages/LoginPage'
import { DashboardPage } from '../pages/DashboardPage'

export const test = base.extend({
  loginPage: async ({ page }, use) => {
    await use(new LoginPage(page))
  },
  dashboardPage: async ({ page }, use) => {
    await use(new DashboardPage(page))
  },
  autenticado: async ({ page }, use) => {
    const login = new LoginPage(page)
    await login.navegar()
    await login.login(process.env.USER, process.env.PASS)
    await use(page)
  }
})
```

#082 Configurar relatório customizado com allure-playwright **Sênior**

■ *Relatórios são evidência — configure do jeito certo*

```
// npm install allure-playwright
// playwright.config.js
reporter: [
  ['html'],
  ['allure-playwright', {
    detail: true,
    outputFolder: 'allure-results',
    suiteTitle: false
  }]
]

// No teste, adicione metadados:
import { allure } from 'allure-playwright'
test('compra com cartão', async ({ page }) => {
  allure.story('Checkout')
  allure.severity('critical')
  allure.owner('lilian.borba')
  // ... lógica do teste
})
```

#083 Implementar visual regression testing com screenshots **Sênior**

Detecte quebras visuais automaticamente

```
test('visual regression — homepage', async ({ page }) => {
  await page.goto('/')
  // Na primeira execução: cria baseline
  // Nas seguintes: compara pixel a pixel
  await expect(page).toHaveScreenshot('homepage.png', {
    maxDiffPixelRatio: 0.02, // tolerância de 2%
    threshold: 0.2,
    animations: 'disabled',
  })
})

// Atualizar baseline quando mudança é intencional:
// npx playwright test --update-snapshots
```

#084 Criar teste de contrato de API com schema JSON

Sênior

Valide estrutura da resposta, não só o conteúdo

```
// npm install ajv
import Ajv from 'ajv'

const schemaUsuario = {
  type: 'object',
  required: ['id', 'nome', 'email', 'ativo'],
  properties: {
    id: { type: 'integer' },
    nome: { type: 'string', minLength: 2 },
    email: { type: 'string', format: 'email' },
    ativo: { type: 'boolean' }
  }
}

test('contrato GET /usuario/:id', async ({ request }) => {
  const res = await request.get('/api/usuarios/1')
  const body = await res.json()
  const ajv = new Ajv()
  const valid = ajv.validate(schemaUsuario, body)
  expect(valid, ajv.errorsText()).toBe(true)
})
```

#085 Implementar retry estratégico apenas para falhas de infraestrutura

Sênior

Retry cego mascara bugs — retry inteligente evita flakiness

```
// playwright.config.js
retries: process.env.CI ? 2 : 0, // retry apenas em CI

// Para retry granular por teste:
test('operação crítica', async ({ page }) => {
  test.slow() // multiplica timeout por 3
  test.info().annotations.push({
    type: 'workaround',
    description: 'Retry por latência do ambiente de staging'
  })
  // ... lógica
})

// Identificar flaky tests com:
// npx playwright test --repeat-each=5 tests/critico.spec.js
```

#086 Configurar Playwright em monorepo com múltiplos projetos

Sênior

Use workspaces do npm/yarn + playwright.config por projeto

```
// raiz/playwright.config.js
projects: [
  { name: 'web-app', testDir: './apps/web/tests' },
  { name: 'admin-app', testDir: './apps/admin/tests' },
  { name: 'api', testDir: './services/api/tests'},
]
// Rodar apenas um projeto:
// npx playwright test --project=web-app
```

#087 Implementar Page Object com herança para componentes compartilhados **Sênior**

■ *Components comuns (header, modal, toast) em classe base*

```
// pages/BasePage.js
class BasePage {
  constructor(page) {
    this.page = page
    this.header = new HeaderComponent(page)
    this.toast = new ToastComponent(page)
  }
  async verificarToastSucesso(msg) {
    await expect(this.toast.sucesso).toContainText(msg)
  }
}
// pages/ClientesPage.js
class ClientesPage extends BasePage {
  constructor(page) {
    super(page)
    this.btnNovo = page.locator('#btn-novo-cliente')
  }
}
```

#088 Medir e validar performance de carregamento de página

Sênior

Use window.performance.timing via evaluate()

```
test('performance de carregamento', async ({ page }) => {
  await page.goto('/dashboard')
  const timing = await page.evaluate(() => {
    const t = window.performance.timing
    return {
      ttfb: t.responseStart - t.requestStart,
      domLoaded: t.domContentLoadedEventEnd - t.navigationStart,
      totalLoad: t.loadEventEnd - t.navigationStart
    }
  })
  console.log('TTFB:', timing.ttfb, 'ms')
  expect(timing.domLoaded).toBeLessThan(3000) // < 3s
  expect(timing.totalLoad).toBeLessThan(5000) // < 5s
})
```

#089 Criar smoke test suite executada em < 2 minutos **Sênior**

■ *Smoke rápido = feedback rápido no pipeline*

```
// tests/smoke/critical.spec.js
// @tags: smoke
test.describe('@smoke', () => {
  test('homepage carrega', async ({ page }) => {
    await page.goto('/')
    await expect(page.locator('h1')).toBeVisible({ timeout: 5000 })
  })
  test('login funciona', async ({ page }) => {
    // ... login rápido
  })
  test('API de saúde retorna 200', async ({ request }) => {
    const res = await request.get('/health')
    expect(res.status()).toBe(200)
  })
})
// Rodar: npx playwright test --grep @smoke
```

#090 Integrar testes com Slack notification no pipeline

Sênior

Notifique o time automaticamente com resultado e link

```
# .github/workflows/testes.yml
- name: Notificar Slack
  if: always()
  uses: slackapi/slack-github-action@v1.24.0
  with:
    channel-id: 'C0QA123456'
    slack-message: |
      *Testes E2E:* ${{ job.status }}
      *Branch:* ${{ github.ref_name }}
      *Commit:* ${{ github.sha }}
      *Relatório:* ${{ github.server_url }}/${{ github.repository
}}/actions/runs/${{ github.run_id }}
  env:
    SLACK_BOT_TOKEN: ${{ secrets.SLACK_BOT_TOKEN }}
```

#091 Implementar helper de limpeza de dados pós-teste **Sênior**

Testes que criam dados devem limpar depois (teardown)

```
// fixtures/cleanup.fixture.js
export const test = base.extend({
  cleanupIds: async ({}, use) => {
    const ids = []
    await use(ids)
    // Teardown: deletar todos os recursos criados
    for (const { tipo, id } of ids) {
      await apiClient.delete(`/${tipo}/${id}`)
    }
  }
})

test('criar e deletar cliente', async ({ request, cleanupIds }) => {
  const res = await request.post('/api/clientes', { data: dadosCliente })
  const { id } = await res.json()
  cleanupIds.push({ tipo: 'clientes', id })
  // teste...
})
```

#092 Validar conteúdo de PDF gerado pela aplicação

Sênior

Use pdf-parse para extrair e verificar texto do PDF

```
// npm install pdf-parse
import pdfParse from 'pdf-parse'
import * as fs from 'fs'

test('PDF de contrato contém dados corretos', async ({ page }) => {
  await page.goto('/contratos/123')
  const [download] = await Promise.all([
    page.waitForEvent('download'),
    page.click('#btn-baixar-contrato')
  ])
  await download.saveAs('./temp/contrato.pdf')
  const dataBuffer = fs.readFileSync('./temp/contrato.pdf')
  const pdfData = await pdfParse(dataBuffer)
  expect(pdfData.text).toContain('CONTRATO DE PRESTAÇÃO')
  expect(pdfData.text).toContain('CPF: 123.456.789-00')
})
```

#093 Criar relatório customizado com métricas de QA

Sênior

■ *Publique métricas além do pass/fail: tempo, cobertura, flakyness*

```
// reporters/metricas-reporter.js
class MetricasReporter {
  onBegin(config, suite) {
    this.inicio = Date.now()
    this.total = suite.allTests().length
  }
  onTestEnd(test, result) {
    if (result.status === 'failed') this.falhas++
    if (result.status === 'flaky') this.flaky++
  }
  onEnd(result) {
    const duracao = (Date.now() - this.inicio) / 1000
    console.log(`Total: ${this.total} | Falhas: ${this.falhas} | Flaky:
${this.flaky} | Tempo: ${duracao}s`)
  }
}
```

#094 Testar fluxo de renovação de token JWT expirado

Sênior

■ *Simule expiração e verifique refresh automático*

```
test('renovação automática de token', async ({ page, context }) => {
  // Injetar token expirado via localStorage
  await context.addInitScript(() => {
    localStorage.setItem('token', 'token.expirado.jwt')
  })
  await page.goto('/dashboard')
  // Sistema deve tentar refresh automaticamente
  // Se implementado corretamente, usuário não é redirecionado para login
  await expect(page).not.toHaveURL(/login/)
  // OU verificar redirecionamento correto se refresh falhar
  await expect(page).toHaveURL(/login/)
})
```

#095 Implementar teste de resiliência — simular falha de serviço externo

Sênior

Use page.route para simular timeout ou erro de API

```
test('sistema trata falha do serviço de CEP', async ({ page }) => {
  // Simular timeout no serviço de CEP
  await page.route('**/api.cep.la/**', async route => {
    await route.abort('timedout')
  })
  await page.goto('/endereco')
  await page.fill('#cep', '30112-000')
  await page.locator('#cep').blur()
  // Verificar que sistema exibe mensagem amigável, não erro técnico
  await expect(page.locator('.aviso-cep')).toContainText('não foi possível
consultar')
  // Verificar que formulário ainda pode ser preenchido manualmente
  await expect(page.locator('#rua')).toBeEditable()
})
```

#096 Criar teste de fluxo crítico com gravação de trace

Sênior

Trace é evidência + debug tool — habilite para testes críticos

```
// playwright.config.js
use: {
  trace: 'retain-on-failure', // salva trace se o teste falhar
  video: 'retain-on-failure', // salva vídeo se falhar
  screenshot: 'only-on-failure'
}

// Para trace sempre (fluxos críticos):
test('checkout completo', async ({ page }) => {
  await page.context().tracing.start({ screenshots: true, snapshots: true })
  // ... fluxo completo
  await page.context().tracing.stop({ path: './traces/checkout.zip' })
})
// Visualizar: npx playwright show-trace traces/checkout.zip
```

#097 Medir taxa de flakiness e reportar no pipeline

Sênior

Detecte testes instáveis antes que destruam a confiança da suite

```
# .github/workflows/flakiness-check.yml
- name: Detectar flaky tests
  run: npx playwright test --repeat-each=3 --reporter=json > results.json

- name: Analisar flakiness
  run: |
    node -e "
      const r = require('./results.json')
      const flaky = r.suites.flatMap(s => s.specs)
        .filter(s => s.tests.some(t => t.results.some(r => r.status === 'flaky')))
      if (flaky.length > 0) {
        console.error('Flaky tests encontrados:', flaky.map(f => f.title))
        process.exit(1)
      }
    "
```

#098 Documentar arquitetura do framework com diagrama em código **Sênior**

Mermaid no README — documentação que não envelhece

````
# README.md do framework

## Arquitetura

```mermaid
graph TD
  A[playwright.config.js] --> B[fixtures/base.fixture.js]
  B --> C[pages/BasePage.js]
  C --> D[pages/LoginPage.js]
  C --> E[pages/DashboardPage.js]
  B --> F[helpers/]
  F --> G[faker.js]
  F --> H[cleanup.js]
  A --> I[tests/]
  I --> J[smoke/]
  I --> K[e2e/]
  I --> L[api/]
```
````

#099 Implementar teste de acessibilidade com relatório detalhado **Sênior**

■ *WCAG 2.1 AA — obrigatório em sistemas governamentais e financeiros*

```
// npm install @axe-core/playwright
import AxeBuilder from '@axe-core/playwright'

test('acessibilidade WCAG 2.1 AA — checkout', async ({ page }) => {
  await page.goto('/checkout')
  const resultado = await new AxeBuilder({ page })
    .withTags(['wcag2a', 'wcag2aa', 'wcag21aa'])
    .exclude('.cookie-banner') // excluir componente terceiro
    .analyze()

  const criticos = resultado.violations.filter(v => v.impact === 'critical')
  if (criticos.length > 0) {
    console.log('Violações críticas:', JSON.stringify(criticos, null, 2))
  }
  expect(criticos.length).toBe(0) // zero tolerância para críticos
})
```

#100 Criar health check automatizado executado a cada 5 minutos **Sênior**

■ *Monitoring + testing juntos — detecte problemas antes do usuário*

```
// tests/monitoring/health.spec.js
// Executado via GitHub Actions schedule

test('health check — serviços críticos', async ({ request }) => {
  const endpoints = [
    { url: '/health', esperado: 200 },
    { url: '/api/auth/status', esperado: 200 },
    { url: '/api/produtos', esperado: 200 },
  ]
  for (const ep of endpoints) {
    const res = await request.get(process.env.BASE_URL + ep.url)
    expect(res.status(), `${ep.url} retornou ${res.status()}`).toBe(ep.esperado)
  }
})

# .github/workflows/health.yml
# on:
# schedule:
# - cron: '*/5 * * * *' # a cada 5 minutos
```

100 Exercícios de API — Gabarito Completo

Todos os exercícios usam Playwright API (request fixture) ou Supertest. Base URL de prática: https://reqres.in

Nível Júnior — Exercícios 001 a 040

#001 Fazer requisição GET e verificar status code 200 — **Júnior**

■ *O básico — verificar que o endpoint existe e responde*

```
test('GET /users — status 200', async ({ request }) => {
  const response = await request.get('https://reqres.in/api/users?page=1')
  expect(response.status()).toBe(200)
})
```

#002 Verificar Content-Type do header de resposta — **Júnior**

■ *APIs JSON devem retornar application/json*

```
test('Content-Type JSON', async ({ request }) => {
  const response = await request.get('https://reqres.in/api/users/1')
  const contentType = response.headers()['content-type']
  expect(contentType).toContain('application/json')
})
```

#003 Verificar campos obrigatórios no body da resposta — **Júnior**

■ *Use toHaveProperty() para verificar estrutura*

```
test('GET /users/:id — campos obrigatórios', async ({ request }) => {
  const res = await request.get('https://reqres.in/api/users/2')
  const body = await res.json()
  expect(body).toHaveProperty('data')
  expect(body.data).toHaveProperty('id')
  expect(body.data).toHaveProperty('email')
  expect(body.data).toHaveProperty('first_name')
})
```

#004 Fazer POST e verificar status 201 — **Júnior**

■ *Criação de recurso deve retornar 201 Created*

```
test('POST /users — criar usuário', async ({ request }) => {
  const response = await request.post('https://reqres.in/api/users', {
    data: { name: 'QA Insider', job: 'QA Lead' }
  })
  expect(response.status()).toBe(201)
  const body = await response.json()
  expect(body.name).toBe('QA Insider')
  expect(body.id).toBeDefined()
})
```

#005 Verificar que ID retornado no POST é string ou número — **Júnior**

■ *Valide o tipo do dado, não só a existência*

```
test('POST — id no body de resposta', async ({ request }) => {
  const res = await request.post('https://reqres.in/api/users', {
    data: { name: 'teste', job: 'qa' }
  })
  const body = await res.json()
  expect(typeof body.id === 'string' || typeof body.id === 'number').toBe(true)
})
```

#006 Fazer PUT e verificar status 200 + campo atualizado — **Júnior**

■ *Atualização completa do recurso*

```
test('PUT /users/:id — atualizar', async ({ request }) => {
  const res = await request.put('https://reqres.in/api/users/2', {
    data: { name: 'Novo Nome', job: 'QA Architect' }
  })
  expect(res.status()).toBe(200)
  const body = await res.json()
  expect(body.job).toBe('QA Architect')
  expect(body.updatedAt).toBeDefined()
})
```

#007 Fazer PATCH e verificar apenas o campo alterado — **Júnior**

■ *PATCH = atualização parcial, não substitui o recurso inteiro*

```
test('PATCH /users/:id — atualização parcial', async ({ request }) => {
  const res = await request.patch('https://reqres.in/api/users/2', {
    data: { job: 'QA Senior' }
  })
  expect(res.status()).toBe(200)
  const body = await res.json()
  expect(body.job).toBe('QA Senior')
})
```

#008 Fazer DELETE e verificar status 204 — **Júnior**

■ *Delete bem-sucedido não tem body — só status*

```
test('DELETE /users/:id', async ({ request }) => {
  const res = await request.delete('https://reqres.in/api/users/2')
  expect(res.status()).toBe(204)
  const body = await res.text()
  expect(body).toBe('') // body deve estar vazio
})
```

#009 Verificar que GET com ID inexistente retorna 404 — **Júnior**

■ *Teste negativo — recursos inexistentes devem retornar 404*

```
test('GET /users/999 — 404 Not Found', async ({ request }) => {
  const res = await request.get('https://reqres.in/api/users/999')
  expect(res.status()).toBe(404)
})
```

#010 Verificar que POST sem body retorna erro de validação — **Júnior**

■ *400 ou 422 — depende da implementação da API*

```
test('POST sem body — erro de validação', async ({ request }) => {
  const res = await request.post('https://reqres.in/api/register', {
    data: {} // body vazio
  })
  expect([400, 422]).toContain(res.status())
  const body = await res.json()
  expect(body.error).toBeDefined()
})
```

#011 Verificar paginação — total_pages e per_page — **Júnior**

■ *Dados de paginação são contratos importantes*

```
test('paginação — metadados', async ({ request }) => {
  const res = await request.get('https://reqres.in/api/users?page=1&per_page=3')
  const body = await res.json()
  expect(body).toHaveProperty('total')
  expect(body).toHaveProperty('total_pages')
  expect(body.per_page).toBe(3)
  expect(body.data.length).toBeLessThanOrEqual(3)
})
```

#012 Verificar que lista retornada não está vazia — **Júnior**

■ *Dados mínimos esperados na listagem*

```
test('GET /users — lista não vazia', async ({ request }) => {
  const res = await request.get('https://reqres.in/api/users')
  const body = await res.json()
  expect(body.data).toBeDefined()
  expect(body.data.length).toBeGreaterThan(0)
})
```

#013 Verificar formato do campo email na resposta — **Júnior**

■ *Valide padrão de dados com regex*

```
test('email em formato válido', async ({ request }) => {
  const res = await request.get('https://reqres.in/api/users/1')
  const body = await res.json()
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/
  expect(emailRegex.test(body.data.email)).toBe(true)
})
```

#014 Verificar que response time está dentro do SLA — **Júnior**

■ *Performance básica de API — todos devem fazer isso*

```
test('response time < 2s', async ({ request }) => {
  const inicio = Date.now()
  const res = await request.get('https://reqres.in/api/users')
  const tempo = Date.now() - inicio
  expect(res.status()).toBe(200)
  expect(tempo).toBeLessThan(2000) // SLA de 2 segundos
  console.log(`Response time: ${tempo}ms`)
})
```

#015 Verificar autenticação — endpoint sem token retorna 401 — **Júnior**

■ *Teste de segurança básico — nunca pule*

```
test('sem token — 401 Unauthorized', async ({ request }) => {
  const res = await request.get('https://reqres.in/api/users/me')
  // Endpoint protegido deve recusar sem token
  expect([401, 403]).toContain(res.status())
})
```

#016 Testar login e capturar token JWT — **Júnior**

■ *Fluxo de autenticação completo*

```
test('POST /login — capturar token', async ({ request }) => {
  const res = await request.post('https://reqres.in/api/login', {
    data: { email: 'eve.holt@reqres.in', password: 'cityslicka' }
  })
  expect(res.status()).toBe(200)
  const body = await res.json()
  expect(body.token).toBeDefined()
  expect(body.token.length).toBeGreaterThan(10)
})
```

#017 Testar login com credenciais inválidas — 400 — **Júnior**

■ *Sempre teste o caminho de erro*

```
test('login inválido — 400', async ({ request }) => {
  const res = await request.post('https://reqres.in/api/login', {
    data: { email: 'nao@existe.com', password: 'errada' }
  })
  expect(res.status()).toBe(400)
  const body = await res.json()
  expect(body.error).toBeDefined()
})
```

#018 Verificar header de segurança X-Content-Type-Options

Júnior

■ *Headers de segurança são parte do contrato da API*

```
test('header de segurança', async ({ request }) => {
  const res = await request.get('https://reqres.in/api/users')
  const headers = res.headers()
  // Verificar headers de segurança presentes
  console.log('Headers:', JSON.stringify(headers, null, 2))
  // Em APIs bem configuradas:
  // expect(headers['x-content-type-options']).toBe('nosniff')
  // expect(headers['x-frame-options']).toBeDefined()
})
```

#019 Verificar que IDs são únicos na listagem

Júnior

■ *Integridade de dados — duplicatas indicam bug sério*

```
test('IDs únicos na listagem', async ({ request }) => {
  const res = await request.get('https://reqres.in/api/users')
  const body = await res.json()
  const ids = body.data.map(u => u.id)
  const idsUnicos = new Set(ids)
  expect(idsUnicos.size).toBe(ids.length) // sem duplicatas
})
```

#020 Verificar que dados retornados batem com o filtro enviado

Júnior

■ *Se enviei page=2, os dados devem ser da página 2*

```
test('filtro de página funciona', async ({ request }) => {
  const pag1 = await (await
request.get('https://reqres.in/api/users?page=1')).json()
  const pag2 = await (await
request.get('https://reqres.in/api/users?page=2')).json()
  const ids1 = pag1.data.map(u => u.id)
  const ids2 = pag2.data.map(u => u.id)
  const overlap = ids1.filter(id => ids2.includes(id))
  expect(overlap.length).toBe(0) // páginas não se sobrepõem
})
```

Nível Pleno — Exercícios 021 a 060

#021	Implementar fluxo CRUD completo em um único contexto de teste	Pleno

■ *Create → Read → Update → Delete — valide cada etapa*

```
test('CRUD completo de usuário', async ({ request }) => {
  // CREATE
  const criar = await request.post('https://reqres.in/api/users', {
    data: { name: 'CRUD Test', job: 'QA' }
  })
  expect(criar.status()).toBe(201)
  const { id } = await criar.json()

  // READ
  const ler = await request.get(`https://reqres.in/api/users/${id}`)
  // reqres.in não persiste, mas em API real:
  // expect(ler.status()).toBe(200)

  // UPDATE
  const atualizar = await request.put(`https://reqres.in/api/users/${id}`, {
    data: { name: 'CRUD Test Updated', job: 'QA Senior' }
  })
  expect(atualizar.status()).toBe(200)

  // DELETE
  const deletar = await request.delete(`https://reqres.in/api/users/${id}`)
  expect(deletar.status()).toBe(204)
})
```

#022 Testar rate limiting — muitas requisições retornam 429

Pleno

Verificar que a API tem proteção contra abuso

```
test('rate limiting', async ({ request }) => {
  const requisicoes = Array.from({ length: 50 }, () =>
    request.get('https://reqres.in/api/users')
  )
  const respostas = await Promise.all(requisicoes)
  const status = respostas.map(r => r.status())
  const comRateLimit = status.filter(s => s === 429)
  console.log(`429s recebidos: ${comRateLimit.length} de 50`)
  // Se a API tem rate limiting, pelo menos alguns devem ser 429
})
```

#023 Validar contrato com JSON Schema (Ajv)

Pleno

Schema validation é teste de contrato — não apenas de conteúdo

```
import Ajv from 'ajv'
import addFormats from 'ajv-formats'

const schema = {
  type: 'object',
  required: ['data'],
  properties: {
    data: {
      type: 'object',
      required: ['id', 'email', 'first_name', 'last_name'],
      properties: {
        id: { type: 'integer', minimum: 1 },
        email: { type: 'string', format: 'email' },
        first_name: { type: 'string', minLength: 1 },
        last_name: { type: 'string', minLength: 1 },
      }
    }
  }
}

test('schema validation GET /users/:id', async ({ request }) => {
  const res = await request.get('https://reqres.in/api/users/2')
  const body = await res.json()
  const ajv = new Ajv()
  addFormats(ajv)
  const valid = ajv.validate(schema, body)
  expect(valid, ajv.errorsText()).toBe(true)
})
```

#024 Testar endpoint de busca com query parameters **Pleno**

■ *Valide filtros, ordenação e busca textual*

```
test('busca com query params', async ({ request }) => {
  const casos = [
    { params: '?q=admin&ativo=true', statusEsperado: 200 },
    { params: '?q=', statusEsperado: 200 },
    { params: '?ordem=nome&dir=asc', statusEsperado: 200 },
    { params: '?pagina=0', statusEsperado: 400 }, // página inválida
    { params: '?limite=10000', statusEsperado: 400 }, // limite absurdo
  ]
  for (const { params, statusEsperado } of casos) {
    const res = await request.get(`/api/usuarios${params}`)
    expect(res.status(), `Params: ${params}`).toBe(statusEsperado)
  }
})
```

#025	Testar upload de arquivo via API multipart/form-data	Pleno

■ *APIs de upload têm validações específicas de tipo e tamanho*

```
import * as fs from 'fs'

test('upload de avatar via API', async ({ request }) => {
  const arquivo = fs.readFileSync('./fixtures/avatar.jpg')
  const res = await request.post('/api/usuarios/1/avatar', {
    multipart: {
      arquivo: {
        name: 'avatar.jpg',
        mimeType: 'image/jpeg',
        buffer: arquivo
      }
    },
    headers: { Authorization: `Bearer ${process.env.TOKEN}` }
  })
  expect(res.status()).toBe(200)
  const body = await res.json()
  expect(body.avatar_url).toContain('https://')
})
```

#026 Verificar que API rejeita tipos de arquivo não permitidos **Pleno**

■ *Segurança: bloquear extensões perigosas*

```
test('upload de .exe bloqueado', async ({ request }) => {
  const res = await request.post('/api/upload', {
    multipart: {
      arquivo: {
        name: 'malware.exe',
        mimeType: 'application/octet-stream',
        buffer: Buffer.from('fake exe content')
      }
    },
    headers: { Authorization: `Bearer ${process.env.TOKEN}` }
  })
  expect([400, 415, 422]).toContain(res.status())
})
```

#027 Testar campos com injeção SQL nos parâmetros

Pleno

■ *Segurança básica — API não deve vazar dados nem crashar*

```
const payloads_sql = [
  "' OR '1'='1",
  "'; DROP TABLE usuarios; --",
  "1 UNION SELECT * FROM usuarios--",
  "admin'--",
]

for (const payload of payloads_sql) {
  test(`SQL injection: ${payload.substring(0, 20)}`, async ({ request }) => {
    const res = await request.get(`/api/usuarios?q=${encodeURIComponent(payload)}`)
    // Não deve retornar 500 (erro interno)
    expect(res.status()).not.toBe(500)
    // Não deve retornar mais dados do que o esperado
    if (res.status() === 200) {
      const body = await res.json()
      expect(body.data.length).toBeLessThanOrEqual(100)
    }
  })
}
```

#028 Testar campos com XSS nos parâmetros de entrada **Pleno**

■ *API não deve refletir scripts sem sanitização*

```
const payloads_xss = [
  '<script>alert(1)</script>',
  '<img src=x one rror=alert(1)>',
  'javascript:alert(1)',
]

for (const payload of payloads_xss) {
  test(`XSS: ${payload.substring(0, 30)}`, async ({ request }) => {
    const res = await request.post('/api/comentarios', {
      data: { texto: payload },
      headers: { Authorization: `Bearer ${process.env.TOKEN}` }
    })
    if (res.status() === 201) {
      const body = await res.json()
      // Payload deve estar escapado, não refletido raw
      expect(body.texto).not.toContain('<script>')
    }
  })
}
```

#029 Verificar idempotência do método PUT

Pleno

■ *PUT idempotente: chamadas repetidas devem ter mesmo efeito*

```
test('PUT é idempotente', async ({ request }) => {
  const dados = { name: 'Idempotente', job: 'QA' }
  const req1 = await request.put('https://reqres.in/api/users/2', { data: dados })
  const req2 = await request.put('https://reqres.in/api/users/2', { data: dados })
  const req3 = await request.put('https://reqres.in/api/users/2', { data: dados })
  // Todas devem retornar 200
  expect(req1.status()).toBe(200)
  expect(req2.status()).toBe(200)
  expect(req3.status()).toBe(200)
  // Estado final deve ser o mesmo
  const b1 = await req1.json()
  const b3 = await req3.json()
  expect(b1.name).toBe(b3.name)
})
```

#030 Testar concorrência — 2 requisições simultâneas para o mesmo recurso

Pleno

■ *Race conditions são bugs difíceis de reproduzir*

```
test('concorrência — sem race condition em cadastro', async ({ request }) => {
  const dados = { email: `concurrent_${Date.now()}@qa.test`, senha: 'Senha@123' }
  // Enviar 2 requisições simultâneas com mesmo email
  const [res1, res2] = await Promise.all([
    request.post('/api/usuarios', { data: dados }),
    request.post('/api/usuarios', { data: dados }),
  ])
  const statuses = [res1.status(), res2.status()].sort()
  // Uma deve criar (201) e outra deve falhar por duplicata (409)
  expect(statuses).toEqual([201, 409])
})
```

#031 Testar versionamento de API — v1 e v2 coexistem

Pleno

Breaking changes exigem nova versão — valide que ambas funcionam

```
test('v1 e v2 da API coexistem', async ({ request }) => {
  const v1 = await request.get('/api/v1/usuarios/1')
  const v2 = await request.get('/api/v2/usuarios/1')

  expect(v1.status()).toBe(200)
  expect(v2.status()).toBe(200)

  const bodyV1 = await v1.json()
  const bodyV2 = await v2.json()

  // v2 tem campo novo 'avatar_url' que v1 não tem
  expect(bodyV1).not.toHaveProperty('avatar_url')
  expect(bodyV2).toHaveProperty('avatar_url')

  // Campo comum deve ter mesmo valor
  expect(bodyV1.email).toBe(bodyV2.email)
})
```

#032 Testar cabeçalho de depreciação em versão antiga

Pleno

APIs boas avisam antes de quebrar — verifique o header Deprecation

```
test('API v1 retorna header de depreciação', async ({ request }) => {
  const res = await request.get('/api/v1/usuarios')
  const headers = res.headers()

  // Header padrão de depreciação (RFC 8594)
  expect(headers['deprecation']).toBeDefined()
  // Ou Sunset com a data de desativação
  const sunset = headers['sunset']
  if (sunset) {
    const dataDesativacao = new Date(sunset)
    expect(dataDesativacao > new Date()).toBe(true) // ainda no futuro
    console.log(`API v1 será desativada em:
${dataDesativacao.toLocaleDateString('pt-BR')}`)
  }
})
```

#033 Testar endpoint de health check da aplicação

Pleno

Health check é o primeiro teste a rodar no monitoramento

```
test('GET /health — sistema saudável', async ({ request }) => {
  const res = await request.get('/api/health')
  expect(res.status()).toBe(200)

  const body = await res.json()
  expect(body.status).toBe('ok')

  // Verificar dependências críticas
  if (body.dependencies) {
    expect(body.dependencies.database).toBe('ok')
    expect(body.dependencies.cache).toBe('ok')
  }

  // Verificar tempo de resposta
  expect(body.uptime).toBeGreaterThan(0)
})
```

#034 Testar header de cache — ETag e Cache-Control

Pleno

Cache mal configurado = dados desatualizados para o usuário

```
test('recursos estáticos têm cache adequado', async ({ request }) => {
  const res1 = await request.get('/api/produtos/categorias')
  const etag = res1.headers()['etag']
  const cacheControl = res1.headers()['cache-control']

  // Deve ter estratégia de cache definida
  expect(cacheControl).toBeDefined()
  console.log('Cache-Control:', cacheControl)

  if (etag) {
    // Segunda requisição com ETag deve retornar 304
    const res2 = await request.get('/api/produtos/categorias', {
      headers: { 'If-None-Match': etag }
    })
    expect(res2.status()).toBe(304) // Not Modified
  }
})
```

#035 Testar CORS — origens permitidas e bloqueadas

Pleno

■ *CORS mal configurado = vulnerabilidade de segurança*

```
test('CORS permite origens autorizadas', async ({ request }) => {
  // Origem permitida
  const resPermitida = await request.get('/api/usuarios', {
    headers: { 'Origin': 'https://app.meusistema.com' }
  })
  const corspermitido = resPermitida.headers()['access-control-allow-origin']
  expect(['https://app.meusistema.com', '*']).toContain(corspermitido)

  // Origem bloqueada (se CORS for restrito)
  const resBloqueada = await request.get('/api/usuarios', {
    headers: { 'Origin': 'https://site-malicioso.com' }
  })
  const corsBloqueado = resBloqueada.headers()['access-control-allow-origin']
  expect(corsBloqueado).not.toBe('https://site-malicioso.com')
})
```

#036 Testar Content Negotiation — JSON vs XML

Pleno

API que suporta múltiplos formatos deve respeitar Accept header

```
test('Content Negotiation via Accept header', async ({ request }) => {
  // Solicitar JSON
  const resJson = await request.get('/api/usuarios/1', {
    headers: { 'Accept': 'application/json' }
  })
  expect(resJson.headers()['content-type']).toContain('application/json')

  // Solicitar XML (se suportado)
  const resXml = await request.get('/api/usuarios/1', {
    headers: { 'Accept': 'application/xml' }
  })
  // Deve responder 200 com XML ou 406 Not Acceptable
  expect([200, 406]).toContain(resXml.status())
  if (resXml.status() === 200) {
    expect(resXml.headers()['content-type']).toContain('xml')
  }
})
```

#037 Testar bulk operations — criar múltiplos recursos de uma vez

Pleno

■ *Bulk endpoints têm regras de validação específicas*

```
test('POST /usuarios/bulk — criar múltiplos', async ({ request }) => {
  const usuarios = Array.from({ length: 5 }, (_, i) => ({
    nome: `Bulk User ${i + 1}`,
    email: `bulk.${Date.now()}.${i}@qa.test`,
    perfil: 'usuario'
  }))

  const res = await request.post('/api/usuarios/bulk', {
    data: { usuarios },
    headers: { Authorization: `Bearer ${process.env.TOKEN}` }
  })

  expect(res.status()).toBe(207) // 207 Multi-Status é o padrão para bulk
  const body = await res.json()
  expect(body.criados).toBe(5)
  expect(body.erros).toBe(0)
})
```

#038	Verificar que soft delete não remove do banco mas esconde da API	Pleno

■ *Soft delete é contrato de API — recurso deletado não aparece em GET*

```
test('recurso soft-deletado não aparece na listagem', async ({ request }) => {
  const headers = { Authorization: `Bearer ${process.env.TOKEN}` }

  // Criar e deletar
  const criar = await request.post('/api/tags', {
    data: { nome: `tag-delete-test-${Date.now()}` }, headers
  })
  const { id } = await criar.json()

  await request.delete(`/api/tags/${id}`, { headers })

  // Não deve aparecer na listagem
  const lista = await request.get('/api/tags', { headers })
  const body = await lista.json()
  const ids = body.data.map((t: any) => t.id)
  expect(ids).not.toContain(id)

  // GET direto deve retornar 404 ou 410
  const direto = await request.get(`/api/tags/${id}`, { headers })
  expect([404, 410]).toContain(direto.status())
})
```

#039 Testar endpoint de busca com operadores avançados **Pleno**

■ *Busca textual com operadores deve retornar resultados precisos*

```
test('busca com operadores: AND, OR, NOT', async ({ request }) => {
  const headers = { Authorization: `Bearer ${process.env.TOKEN}` }

  // Busca AND
  const resAnd = await request.get('/api/produtos?q=notebook+AND+gamer', { headers
})
  expect(resAnd.status()).toBe(200)

  // Busca com campo específico
  const resCampo = await
request.get('/api/produtos?categoria=eletronicos&preco_max=2000', { headers })
  const body = await resCampo.json()
  if (body.data.length > 0) {
    // Todos os resultados devem ter preço <= 2000
    body.data.forEach((p: any) => {
      expect(p.preco).toBeLessThanOrEqual(2000)
    })
  }
})
```

#040 Testar webhook — endpoint que recebe eventos externos **Pleno**

■ *Webhooks têm assinatura HMAC — valide a segurança*

```
import crypto from 'crypto'

test('webhook com assinatura HMAC válida é aceito', async ({ request }) => {
  const payload = JSON.stringify({
    evento: 'pagamento.aprovado',
    pedido_id: 12345,
    valor: 199.90
  })

  // Gerar assinatura HMAC-SHA256
  const assinatura = crypto
    .createHmac('sha256', process.env.WEBHOOK_SECRET!)
    .update(payload)
    .digest('hex')

  const res = await request.post('/api/webhooks/pagamento', {
    data: payload,
    headers: {
      'Content-Type': 'application/json',
      'X-Signature': `sha256=${assinatura}`
    }
  })
  expect(res.status()).toBe(200)
})
```

#041 Verificar que webhook rejeita assinatura inválida **Pleno**

■ *Segurança do webhook — payload sem assinatura deve ser rejeitado*

```
test('webhook sem assinatura válida retorna 401', async ({ request }) => {
  const res = await request.post('/api/webhooks/pagamento', {
    data: JSON.stringify({ evento: 'pagamento.aprovado' }),
    headers: {
      'Content-Type': 'application/json',
      'X-Signature': 'sha256=assinatura_invalida_aqui'
    }
  })
  expect([401, 403]).toContain(res.status())
})
```

#042 Testar endpoint de exportação assíncrona com polling

Pleno

■ *Exports grandes são assíncronos — poll até concluir*

```
test('exportação assíncrona — poll até concluir', async ({ request }) => {
  const headers = { Authorization: `Bearer ${process.env.TOKEN}` }

  // Iniciar exportação
  const iniciar = await request.post('/api/exportacoes', {
    data: { tipo: 'clientes', formato: 'xlsx' }, headers
  })
  expect(iniciar.status()).toBe(202) // Accepted
  const { job_id } = await iniciar.json()

  // Poll até concluir (máx 30s)
  let status = 'PROCESSANDO'
  const inicio = Date.now()
  while (status === 'PROCESSANDO' && Date.now() - inicio < 30_000) {
    await new Promise(r => setTimeout(r, 1000))
    const poll = await request.get(`/api/exportacoes/${job_id}`, { headers })
    status = (await poll.json()).status
  }

  expect(status).toBe('CONCLUIDO')

  // Baixar o arquivo
  const download = await request.get(`/api/exportacoes/${job_id}/arquivo`, {
headers })
  expect(download.status()).toBe(200)
  expect(download.headers()['content-type']).toContain('spreadsheet')
})
```

#043 Testar payload com campos extras (should be ignored)

Pleno

API robusta ignora campos desconhecidos sem erro

```
test('campos extras no payload são ignorados', async ({ request }) => {
  const res = await request.post('https://reqres.in/api/users', {
    data: {
      name: 'Teste Extra Fields',
      job: 'QA',
      campo_inexistente: 'valor qualquer',
      __proto__: { admin: true }, // prototype pollution attempt
      constructor: { name: 'hacked' }
    }
  })
  expect(res.status()).toBe(201)
  const body = await res.json()
  expect(body.name).toBe('Teste Extra Fields')
  // Campos extras não devem aparecer na resposta
  expect(body.campo_inexistente).toBeUndefined()
})
```

#044 Testar limites de tamanho de payload

Pleno

Payloads muito grandes devem ser rejeitados — 413 Payload Too Large

```
test('payload muito grande retorna 413', async ({ request }) => {
  // Gerar string grande (> limite da API)
  const stringGrande = 'A'.repeat(10 * 1024 * 1024) // 10MB

  const res = await request.post('/api/documentos', {
    data: { conteudo: stringGrande },
    headers: { Authorization: `Bearer ${process.env.TOKEN}` }
  })
  expect(res.status()).toBe(413) // Payload Too Large
})
```

#045 Testar comportamento com Accept-Language header

Pleno

APIs internacionalizadas devem responder no idioma correto

```
test('mensagens de erro em PT-BR com Accept-Language', async ({ request }) => {
  const res = await request.post('/api/login', {
    data: { email: 'invalido', senha: '' },
    headers: { 'Accept-Language': 'pt-BR,pt;q=0.9' }
  })

  const body = await res.json()
  // Mensagem deve estar em português
  if (body.mensagem || body.error) {
    const msg = body.mensagem || body.error
    // Verificar que não está em inglês
    expect(msg.toLowerCase()).not.toContain('invalid credentials')
    console.log('Mensagem localizada:', msg)
  }
})
```

#046 Testar endpoint com filtros de data em ISO 8601

Pleno

Formato de data é contrato — aceitar só ISO 8601

```
test('filtro de data em formato ISO 8601', async ({ request }) => {
  const headers = { Authorization: `Bearer ${process.env.TOKEN}` }

  // Formato correto
  const correto = await request.get(
    '/api/pedidos?inicio=2025-01-01T00:00:00Z&fim=2025-12-31T23:59:59Z',
    { headers }
  )
  expect(correto.status()).toBe(200)

  // Formato incorreto — deve rejeitar
  const incorreto = await request.get(
    '/api/pedidos?inicio=01/01/2025&fim=31/12/2025',
    { headers }
  )
  expect(incorreto.status()).toBe(400)
  const body = await incorreto.json()
  expect(body.error).toMatch(/formato|date|ISO/i)
})
```

#047 Testar rate limiting com header Retry-After

Pleno

Quando 429, API deve informar quando tentar de novo

```
test('429 inclui header Retry-After', async ({ request }) => {
  let ultimaResposta: any

  // Fazer requisições até atingir o limite
  for (let i = 0; i < 100; i++) {
    ultimaResposta = await request.get('/api/usuarios')
    if (ultimaResposta.status() === 429) break
  }

  if (ultimaResposta?.status() === 429) {
    const retryAfter = ultimaResposta.headers()['retry-after']
    expect(retryAfter).toBeDefined()
    const segundos = parseInt(retryAfter)
    expect(segundos).toBeGreaterThan(0)
    console.log(`Rate limited. Retry em ${segundos}s`)
  }
})
```

#048 Testar endpoint de métricas/telemetria

Pleno

Endpoint /metrics é padrão Prometheus — verifique formato

```
test('GET /metrics — formato Prometheus', async ({ request }) => {
  const res = await request.get('/metrics', {
    headers: { Authorization: `Bearer ${process.env.METRICS_TOKEN}` }
  })

  // Pode ser 200 com texto ou 403 se protegido
  if (res.status() === 200) {
    const texto = await res.text()
    // Formato Prometheus: linhas com 'nome_metrica{labels} valor timestamp'
    expect(texto).toMatch(/^# HELP /m) // deve ter comentários HELP
    expect(texto).toMatch(/^# TYPE /m) // deve ter declaração TYPE
    console.log('Métricas disponíveis:', texto.match(/^# HELP .+/gm)?.length)
  }
})
```

#049	Verificar que API implementa HATEOAS (links de navegação)	Pleno

■ *HATEOAS é o nível 3 do modelo de maturidade REST*

```
test('response inclui links HATEOAS', async ({ request }) => {
  const res = await request.get('/api/pedidos/1', {
    headers: { Authorization: `Bearer ${process.env.TOKEN}` }
  })
  const body = await res.json()

  // Links de navegação devem estar presentes
  if (body._links || body.links) {
    const links = body._links || body.links
    expect(links.self).toBeDefined()
    console.log('Links disponíveis:', Object.keys(links))
    // Links relacionados ao recurso
    // ex: links.usuario, links.itens, links.cancelar
  } else {
    console.log('API não implementa HATEOAS — documentar como melhoria')
  }
})
```

#050	Verificar proteção contra SSRF em campos de URL	Pleno

■ *Server-Side Request Forgery — API não deve fazer requests para IPs internos*

```
test('SSRF — campo de URL não acessa rede interna', async ({ request }) => {
  const payloads_ssrf = [
    'http://localhost/admin',
    'http://127.0.0.1:6379', // Redis
    'http://169.254.169.254/', // AWS metadata
    'http://10.0.0.1/', // rede interna
  ]

  for (const url of payloads_ssrf) {
    const res = await request.post('/api/importar-url', {
      data: { url },
      headers: { Authorization: `Bearer ${process.env.TOKEN}` }
    })
    // Deve bloquear — nunca retornar 200 com conteúdo interno
    expect([400, 403, 422]).toContain(res.status())
    const body = await res.json()
    expect(body.error).toBeDefined()
  }
})
```

#051 Testar autenticação por API Key no header

Pleno

API Key é alternativa ao JWT — verifique todos os cenários

```
test('autenticação por API Key', async ({ request }) => {
  // Key válida
  const valida = await request.get('/api/dados', {
    headers: { 'X-API-Key': process.env.API_KEY }
  })
  expect(valida.status()).toBe(200)

  // Key inválida
  const invalida = await request.get('/api/dados', {
    headers: { 'X-API-Key': 'chave-invalida-123' }
  })
  expect([401, 403]).toContain(invalida.status())

  // Sem key
  const semKey = await request.get('/api/dados')
  expect([401, 403]).toContain(semKey.status())
})
```

#052 Testar endpoint de refresh token

Pleno

■ *JWT expira — o refresh não pode falhar*

```
test('refresh token gera novo access token', async ({ request }) => {
  // Fazer login para obter tokens
  const login = await request.post('/api/auth/login', {
    data: { email: process.env.TEST_USER, senha: process.env.TEST_PASS }
  })
  const { access_token, refresh_token } = await login.json()

  // Usar refresh token
  const refresh = await request.post('/api/auth/refresh', {
    data: { refresh_token }
  })
  expect(refresh.status()).toBe(200)
  const { access_token: novo_token } = await refresh.json()

  // Novo token deve ser diferente do original
  expect(novo_token).not.toBe(access_token)

  // Novo token deve ser válido
  const teste = await request.get('/api/me', {
    headers: { Authorization: `Bearer ${novo_token}` }
  })
  expect(teste.status()).toBe(200)
})
```

#053 Verificar que refresh token não pode ser reutilizado **Pleno**

■ *Refresh token rotation — segurança contra roubo de sessão*

```
test('refresh token usado uma vez é invalidado', async ({ request }) => {
  const login = await request.post('/api/auth/login', {
    data: { email: process.env.TEST_USER, senha: process.env.TEST_PASS }
  })
  const { refresh_token } = await login.json()

  // Primeiro uso — deve funcionar
  const primeiro = await request.post('/api/auth/refresh', {
    data: { refresh_token }
  })
  expect(primeiro.status()).toBe(200)

  // Segundo uso do mesmo token — deve falhar
  const segundo = await request.post('/api/auth/refresh', {
    data: { refresh_token }
  })
  expect([401, 400]).toContain(segundo.status())
})
```

#054 Testar GraphQL — query básica

Pleno

GraphQL substitui múltiplos endpoints REST por um único /graphql

```
test('GraphQL — query de usuário', async ({ request }) => {
  const res = await request.post('/graphql', {
    data: {
      query: `query {
        usuario(id: 1) {
          id
          nome
          email
          perfil {
            cargo
          }
        }
      }`
    },
    headers: { Authorization: `Bearer ${process.env.TOKEN}` }
  })
  expect(res.status()).toBe(200) // GraphQL sempre retorna 200
  const body = await res.json()
  expect(body.errors).toBeUndefined() // sem erros
  expect(body.data.usuario.email).toBeDefined()
})
```

#055 Testar GraphQL — mutation de criação

Pleno

■ *Mutations GraphQL modificam dados — mesmo cuidado que POST REST*

```
test('GraphQL — mutation criar usuário', async ({ request }) => {
  const res = await request.post('/graphql', {
    data: {
      query: `mutation CriarUsuario($input: UsuarioInput!) {
        criarUsuario(input: $input) {
          id
          nome
          email
        }
      }`,
      variables: {
        input: {
          nome: 'GraphQL Test',
          email: `gql.${Date.now()}@qa.test`,
          senha: 'Senha@123'
        }
      }
    },
    headers: { Authorization: `Bearer ${process.env.TOKEN}` }
  })

  const body = await res.json()
  expect(body.errors).toBeUndefined()
  expect(body.data.criarUsuario.id).toBeDefined()
})
```

#056 Testar GraphQL — introspection desabilitada em produção

Pleno

Introspection expõe toda a estrutura da API — deve estar off em prod

```
test('GraphQL introspection bloqueada em produção', async ({ request }) => {
  const res = await request.post('/graphql', {
    data: { query: '{ __schema { types { name } } }' },
    headers: { Authorization: `Bearer ${process.env.TOKEN}` }
  })

  const body = await res.json()
  // Em produção deve estar bloqueado
  if (process.env.AMBIENTE === 'producao') {
    expect(body.errors).toBeDefined()
    expect(body.errors[0].message).toMatch(/introspection|disabled/i)
  } else {
    console.log('Ambiente não-produção — introspection pode estar habilitada')
  }
})
```

#057 Testar WebSocket — conectar e receber mensagem

Pleno

■ *WebSockets para tempo real: chat, notificações, dashboard ao vivo*

```
// npm install ws @types/ws
import WebSocket from 'ws'

test('WebSocket — receber evento em tempo real', async () => {
  await new Promise<void>((resolve, reject) => {
    const ws = new WebSocket(`${process.env.WS_URL}/eventos`, {
      headers: { Authorization: `Bearer ${process.env.TOKEN}` }
    })

    const timeout = setTimeout(() => {
      ws.close()
      reject(new Error('Timeout — nenhum evento recebido em 5s'))
    }, 5000)

    ws.on('open', () => {
      ws.send(JSON.stringify({ tipo: 'subscribe', canal: 'pedidos' }))
    })

    ws.on('message', (data) => {
      const msg = JSON.parse(data.toString())
      expect(msg.tipo).toBeDefined()
      clearTimeout(timeout)
      ws.close()
      resolve()
    })

    ws.on('error', reject)
  })
})
```

#058 Testar idempotency key — evitar duplicação em pagamentos

Pleno

Pagamentos com mesma Idempotency-Key não podem ser duplicados

```
test('Idempotency-Key evita pagamento duplicado', async ({ request }) => {
  const headers = {
    Authorization: `Bearer ${process.env.TOKEN}`,
    'Idempotency-Key': `key-${Date.now()}`
  }
  const dados = { valor: 100.00, descricao: 'Teste idempotência' }

  // Primeira chamada — cria o pagamento
  const res1 = await request.post('/api/pagamentos', { data: dados, headers })
  expect(res1.status()).toBe(201)
  const body1 = await res1.json()

  // Segunda chamada com MESMA key — deve retornar o mesmo resultado
  const res2 = await request.post('/api/pagamentos', { data: dados, headers })
  expect(res2.status()).toBe(200) // 200 = retornando resultado existente
  const body2 = await res2.json()

  // IDs devem ser iguais — não criou duplicata
  expect(body2.id).toBe(body1.id)
})
```

#059 Testar endpoint de auditoria — todas as ações são logadas

Pleno

■ *Compliance: toda ação sensível deve gerar log imutável*

```
test('ação sensível gera log de auditoria', async ({ request }) => {
  const headers = { Authorization: `Bearer ${process.env.TOKEN_ADMIN}` }

  // Realizar ação auditável
  const antes = new Date().toISOString()
  await request.delete('/api/usuarios/99', { headers })

  // Verificar no log de auditoria
  const auditoria = await request.get(
    `/api/auditoria?acao=DELETE&recurso=usuario&depois=${antes}`,
    { headers }
  )
  expect(auditoria.status()).toBe(200)
  const body = await auditoria.json()
  expect(body.data.length).toBeGreaterThan(0)

  const log = body.data[0]
  expect(log.acao).toBe('DELETE')
  expect(log.recurso_id).toBe(99)
  expect(log.usuario_id).toBeDefined() // quem fez a ação
  expect(log.ip_origem).toBeDefined() // de onde
})
```

#060	Testar que dados são mascarados em respostas de acordo com perfil	Pleno

■ *RBAC nos dados: usuário comum não vê CPF completo de outro*

```
test('CPF mascarado para perfil sem permissão', async ({ request }) => {
  // Token de usuário comum
  const headers = { Authorization: `Bearer ${process.env.TOKEN_USUARIO}` }

  const res = await request.get('/api/clientes/1', { headers })
  expect(res.status()).toBe(200)
  const body = await res.json()

  // CPF deve estar mascarado: ***.***.789-00
  if (body.cpf) {
    expect(body.cpf).toMatch(/\*/) // tem asteriscos
    expect(body.cpf).not.toMatch(/^\d{3}\.\d{3}/) // não começa com 3 dígitos
  }

  // Admin vê CPF completo
  const headersAdmin = { Authorization: `Bearer ${process.env.TOKEN_ADMIN}` }
  const resAdmin = await request.get('/api/clientes/1', { headers: headersAdmin })
  const bodyAdmin = await resAdmin.json()
  if (bodyAdmin.cpf) {
    expect(bodyAdmin.cpf).toMatch(/^\d{3}\.\d{3}\.\d{3}-\d{2}$/)
  }
})
```

Nível Sênior — Exercícios 031 a 100

Os exercícios 031-100 cobrem: autenticação OAuth2, testes de contrato com Pact, GraphQL, WebSockets, LGPD/privacidade, testes financeiros (PIX, conciliação), tolerância a falhas, versionamento de API e validação de SLA com métricas.

#031 Testar fluxo OAuth2 Authorization Code completo

Sênior

OAuth2 tem múltiplos passos — cada um deve ser validado

```
test('OAuth2 Authorization Code Flow', async ({ request }) => {
  // Passo 1: obter authorization code
  const authRes = await request.get('/oauth/authorize', {
    params: {
      client_id: process.env.CLIENT_ID,
      redirect_uri: 'https://app.qa.local/callback',
      response_type: 'code',
      scope: 'read:profile'
    }
  })
  const code = new URL(authRes.headers()['location']).searchParams.get('code')

  // Passo 2: trocar code por access_token
  const tokenRes = await request.post('/oauth/token', {
    form: {
      grant_type: 'authorization_code',
      code,
      client_id: process.env.CLIENT_ID,
      client_secret: process.env.CLIENT_SECRET,
      redirect_uri: 'https://app.qa.local/callback'
    }
  })
  expect(tokenRes.status()).toBe(200)
  const { access_token, token_type, expires_in } = await tokenRes.json()
  expect(access_token).toBeDefined()
  expect(token_type).toBe('Bearer')
  expect(expires_in).toBeGreaterThan(0)
})
```

#050 Testar transferência PIX e verificar comprovante **Sênior**

■ *Fluxo financeiro crítico — tolerância zero para erros*

```
test('PIX — transferência e comprovante', async ({ request }) => {
  const headers = { Authorization: `Bearer ${process.env.TOKEN_FINANCEIRO}` }

  // Iniciar transferência
  const pix = await request.post('/api/pix/transferencia', {
    headers,
    data: {
      chave_destino: '11987654321',
      valor: 50.00,
      descricao: 'Pagamento teste QA'
    }
  })
  expect(pix.status()).toBe(201)
  const { id_transacao, status } = await pix.json()
  expect(['PENDENTE', 'PROCESSANDO']).toContain(status)

  // Aguardar processamento e verificar comprovante
  await new Promise(r => setTimeout(r, 2000))
  const comp = await request.get(`/api/pix/comprovante/${id_transacao}`, { headers
})
  expect(comp.status()).toBe(200)
  const compBody = await comp.json()
  expect(compBody.status).toBe('APROVADA')
  expect(compBody.valor).toBe(50.00)
})
```

#075 Testar endpoint de exclusão de dados LGPD

Sênior

■ *LGPD Art. 18 — direito ao esquecimento deve ser testado*

```
test('LGPD — exclusão de dados pessoais', async ({ request }) => {
  const headers = { Authorization: `Bearer ${process.env.TOKEN_ADMIN}` }
  const usuarioId = 42

  // Solicitar exclusão
  const excluir = await request.delete(`/api/lgpd/titular/${usuarioId}/dados`, { headers })
  expect(excluir.status()).toBe(202) // 202 Accepted — processamento assíncrono
  const { protocolo } = await excluir.json()
  expect(protocolo).toBeDefined()

  // Aguardar processamento
  await new Promise(r => setTimeout(r, 3000))

  // Verificar que dados foram removidos
  const check = await request.get(`/api/usuarios/${usuarioId}`, { headers })
  expect([404, 410]).toContain(check.status()) // 410 Gone = excluído permanentemente
})
```

#100 Medir e validar SLA de todos os endpoints críticos **Sênior**

■ *Qualidade não é só funcional — performance é requisito*

```
const endpointsCriticos = [
  { path: '/api/auth/token', sla: 300 },
  { path: '/api/usuarios/perfil', sla: 500 },
  { path: '/api/produtos?limite=10', sla: 800 },
  { path: '/api/pedidos/recentes', sla: 1000 },
  { path: '/api/relatorios/resumo', sla: 3000 },
]

for (const { path, sla } of endpointsCriticos) {
  test(`SLA ${path} < ${sla}ms`, async ({ request }) => {
    const amostras = []
    for (let i = 0; i < 5; i++) {
      const t0 = Date.now()
      await request.get(process.env.BASE_URL + path, {
        headers: { Authorization: `Bearer ${process.env.TOKEN}` }
      })
      amostras.push(Date.now() - t0)
    }
    const p95 = amostras.sort((a,b) => a-b)[Math.floor(amostras.length * 0.95)]
    console.log(`P95 de ${path}: ${p95}ms (SLA: ${sla}ms)`)
    expect(p95).toBeLessThanOrEqual(sla)
  })
}
```

#032 Implementar teste de contrato com Pact — Consumer

Sênior

Consumer-Driven Contract Testing: consumer define o contrato

```
// npm install @pact-foundation/pact
import { Pact } from '@pact-foundation/pact'
import { like, eachLike } from '@pact-foundation/pact/src/dsl/matchers'

const provider = new Pact({
  consumer: 'frontend-app',
  provider: 'usuarios-api',
  port: 1234,
  log: './logs/pact.log',
  dir: './pacts',
})

describe('Contrato: GET /usuarios/:id', () => {
  beforeAll(() => provider.setup())
  afterAll(() => provider.finalize())
  afterEach(() => provider.verify())

  it('retorna usuário com campos obrigatórios', async () => {
    await provider.addInteraction({
      state: 'usuário 1 existe',
      uponReceiving: 'request de GET /usuarios/1',
      withRequest: { method: 'GET', path: '/usuarios/1' },
      willRespondWith: {
        status: 200,
        body: {
```

```
          id: like(1),
          nome: like('João Silva'),
          email: like('joao@email.com'),
        }
      }
    })
    const res = await fetch('http://localhost:1234/usuarios/1')
    const body = await res.json()
    expect(body.id).toBeDefined()
  })
})
```

#033 Implementar teste de provider com Pact — **Sênior**

■ *Provider verifica que implementa o contrato do consumer*

```
import { Verifier } from '@pact-foundation/pact'

test('Provider verifica contrato Pact', async () => {
  const opts = {
    provider: 'usuarios-api',
    providerBaseUrl: process.env.API_URL!,
    pactUrls: ['./pacts/frontend-app-usuarios-api.json'],
    stateHandlers: {
      'usuário 1 existe': async () => {
        // Setup: garantir que usuário 1 existe no banco de teste
        await fetch(`${process.env.API_URL}/test/setup/usuario/1`, { method: 'POST'
})
      }
    },
    publishVerificationResult: true,
    providerVersion: process.env.GIT_SHA || '1.0.0',
  }
  await new Verifier(opts).verifyProvider()
})
```

#034 Testar tolerância a falhas — circuit breaker

Sênior

Circuit breaker: após N falhas, para de tentar por um tempo

```
test('circuit breaker abre após falhas consecutivas', async ({ request }) => {
  const headers = { Authorization: `Bearer ${process.env.TOKEN}` }
  const statuses: number[] = []

  // Fazer requisições para serviço instável
  for (let i = 0; i < 20; i++) {
    const res = await request.get('/api/servico-externo/status', { headers })
    statuses.push(res.status())
    await new Promise(r => setTimeout(r, 200))
  }

  // Verificar que circuit breaker abriu (503 Service Unavailable)
  // Quando aberto, retorna 503 imediatamente sem tentar o downstream
  const circuitAberto = statuses.filter(s => s === 503).length
  console.log(`503s recebidos (circuit open): ${circuitAberto}/20`)
  console.log('Sequência de status:', statuses.join(', '))
})
```

#035 Testar timeout e retry com backoff exponencial

Sênior

Retry sem backoff causa thundering herd — sempre use backoff

```
import axiosRetry from 'axios-retry'
import axios from 'axios'

test('retry com backoff exponencial', async () => {
  const client = axios.create({ baseURL: process.env.API_URL })
  axiosRetry(client, {
    retries: 3,
    retryDelay: axiosRetry.exponentialDelay, // 1s, 2s, 4s
    retryCondition: (err) => {
      return axiosRetry.isNetworkOrIdempotentRequestError(err) ||
             err.response?.status === 503
    }
  })

  const inicio = Date.now()
  try {
    const res = await client.get('/api/servico-instavel',
      { headers: { Authorization: `Bearer ${process.env.TOKEN}` } }
    )
    console.log('Sucesso após retry. Tempo total:', Date.now() - inicio, 'ms')
  } catch (e) {
    const tempo = Date.now() - inicio
    // Com backoff 1+2+4 = 7s, tempo total deve refletir isso
    expect(tempo).toBeGreaterThan(6000)
    console.log('Falhou após retries. Tempo:', tempo, 'ms')
  }
})
```

#036 Testar bulk delete com lista de IDs

Sênior

Bulk delete tem regras específicas: IDs inválidos, parcial vs total

```
test('DELETE /bulk — deletar múltiplos IDs', async ({ request }) => {
  const headers = { Authorization: `Bearer ${process.env.TOKEN_ADMIN}` }

  // Criar registros para deletar
  const ids: number[] = []
  for (let i = 0; i < 3; i++) {
    const r = await request.post('/api/tags', {
      data: { nome: `bulk-del-${Date.now()}-${i}` }, headers
    })
    ids.push((await r.json()).id)
  }

  // Deletar em bulk
  const del = await request.delete('/api/tags/bulk', {
    data: { ids }, headers
  })
  expect(del.status()).toBe(200)
  const body = await del.json()
  expect(body.deletados).toBe(3)

  // Verificar que foram removidos
  for (const id of ids) {
    const check = await request.get(`/api/tags/${id}`, { headers })
    expect([404, 410]).toContain(check.status())
  }
})
```

#037 Testar migração zero-downtime — API aceita ambas as versões de payload

Sênior

■ *Durante migração, API deve aceitar formato antigo E novo*

```
test('API aceita payload v1 e v2 durante migração', async ({ request }) => {
  const headers = { Authorization: `Bearer ${process.env.TOKEN}` }

  // Payload formato antigo (v1)
  const payloadV1 = { usuario: 'joao', senha_plana: 'Senha@123' }
  const resV1 = await request.post('/api/auth/login', { data: payloadV1, headers })
  // Durante migração: aceitar ou dar mensagem de migração clara
  expect([200, 400]).toContain(resV1.status())
  if (resV1.status() === 400) {
    const body = await resV1.json()
    // Mensagem deve guiar para o novo formato
    expect(body.error).toMatch(/email|formato|migr/i)
  }

  // Payload formato novo (v2)
  const payloadV2 = { email: process.env.TEST_USER, senha: process.env.TEST_PASS }
  const resV2 = await request.post('/api/auth/login', { data: payloadV2, headers })
  expect(resV2.status()).toBe(200)
})
```

#038 Testar que API registra X-Request-ID para rastreabilidade

Sênior

Request ID é fundamental para correlacionar logs em produção

```
test('X-Request-ID enviado é refletido na resposta', async ({ request }) => {
  const requestId = `req-${Date.now()}-${Math.random().toString(36).slice(2)}`

  const res = await request.get('/api/usuarios', {
    headers: {
      Authorization: `Bearer ${process.env.TOKEN}`,
      'X-Request-ID': requestId
    }
  })

  const responseId = res.headers()['x-request-id']
  // API deve refletir o mesmo ID ou gerar um novo
  if (responseId) {
    expect(responseId).toBe(requestId) // ou qualquer UUID válido
    console.log('Request ID rastreável:', responseId)
  } else {
    console.warn('API não implementa X-Request-ID — risco para observabilidade')
  }
})
```

#039 Testar endpoint de renegociação de dívida/parcelamento **Sênior**

Regras de negócio financeiro complexas exigem cobertura ampla

```
test('renegociação — cálculo de parcelas correto', async ({ request }) => {
  const headers = { Authorization: `Bearer ${process.env.TOKEN}` }

  const res = await request.post('/api/dividas/123/renegociar', {
    data: {
      entrada: 500.00,
      parcelas: 12,
      data_primeira: '2025-02-01'
    },
    headers
  })
  expect(res.status()).toBe(200)
  const { plano } = await res.json()

  // Verificar consistência financeira
  const totalParcelas = plano.parcelas.reduce((s: number, p: any) => s + p.valor,
0)
  const totalEsperado = plano.valor_total - plano.entrada
  expect(Math.abs(totalParcelas - totalEsperado)).toBeLessThan(0.01) // tolerância
de 1 centavo
  expect(plano.parcelas.length).toBe(12)
})
```

#040 Testar comportamento da API quando banco de dados está lento

Sênior

■ *Simule latência no banco e verifique comportamento da API*

```
test('API responde adequadamente com banco lento', async ({ request }) => {
  const headers = {
    Authorization: `Bearer ${process.env.TOKEN}`,
    'X-Test-Delay': '3000' // header para simular lentidão no ambiente de teste
  }

  const inicio = Date.now()
  const res = await request.get('/api/relatorios/complexo', {
    headers,
    timeout: 10_000
  })
  const tempo = Date.now() - inicio

  // Deve responder dentro do timeout, mesmo com banco lento
  expect([200, 503, 504]).toContain(res.status())

  if (res.status() === 200) {
    console.log(`Respondeu em ${tempo}ms com banco lento`)
  } else {
    // 503/504 são respostas válidas para timeout de banco
    const body = await res.json()
    expect(body.error).toBeDefined()
  }
})
```

#041 Testar que API suporta compressão gzip **Sênior**

■ *Compressão reduz tráfego em 60-80% — deveria ser padrão*

```
test('resposta comprimida com gzip quando Accept-Encoding enviado', async ({
request }) => {
  const res = await request.get('/api/usuarios', {
    headers: {
      Authorization: `Bearer ${process.env.TOKEN}`,
      'Accept-Encoding': 'gzip, deflate, br'
    }
  })
  expect(res.status()).toBe(200)

  const encoding = res.headers()['content-encoding']
  if (encoding) {
    expect(['gzip', 'br', 'deflate']).toContain(encoding)
    console.log(`Compressão ativa: ${encoding}`)
  } else {
    console.warn('API não comprime respostas — oportunidade de otimização')
  }
})
```

#042 Testar ordenação estável em listagens paginadas

Sênior

Paginação com ordenação inconsistente duplica e perde registros

```
test('ordenação estável entre páginas', async ({ request }) => {
  const headers = { Authorization: `Bearer ${process.env.TOKEN}` }
  const todosIds: number[] = []

  // Coletar todos os IDs paginando
  for (let pagina = 1; pagina <= 5; pagina++) {
    const res = await
request.get(`/api/produtos?pagina=${pagina}&limite=10&ordem=id_asc`, { headers })
    const body = await res.json()
    const ids = body.data.map((p: any) => p.id)
    todosIds.push(...ids)
    if (!body.has_next) break
  }

  // Sem duplicatas entre páginas
  const unicos = new Set(todosIds)
  expect(unicos.size).toBe(todosIds.length)

  // IDs devem estar em ordem crescente
  for (let i = 1; i < todosIds.length; i++) {
    expect(todosIds[i]).toBeGreaterThan(todosIds[i-1])
  }
})
```

#043 Testar que API retorna 405 para métodos não permitidos

Sênior

Method Not Allowed deve ter header Allow com métodos suportados

```
test('método não suportado retorna 405 + header Allow', async ({ request }) => {
  // DELETE em endpoint que só aceita GET
  const res = await request.delete('/api/configuracoes-globais', {
    headers: { Authorization: `Bearer ${process.env.TOKEN}` }
  })
  expect(res.status()).toBe(405) // Method Not Allowed

  const allow = res.headers()['allow']
  expect(allow).toBeDefined()
  expect(allow).toContain('GET') // deve listar métodos permitidos
  expect(allow).not.toContain('DELETE')
})
```

#044 Criar helper de autenticação reutilizável para toda a suite **Sênior**

■ *DRY no setup de autenticação — toda a suite usa o mesmo helper*

```
// helpers/auth.helper.ts
export class AuthHelper {
  private static tokenCache: Map<string, { token: string; expiry: number }> = new Map()

  static async getToken(request: any, perfil: 'admin' | 'usuario' | 'financeiro') {
    const cached = this.tokenCache.get(perfil)
    if (cached && cached.expiry > Date.now()) return cached.token

    const credenciais = {
      admin: { email: process.env.ADMIN_EMAIL, senha: process.env.ADMIN_PASS },
      usuario: { email: process.env.USER_EMAIL, senha: process.env.USER_PASS },
      financeiro: { email: process.env.FIN_EMAIL, senha: process.env.FIN_PASS },
    }

    const res = await request.post('/api/auth/login', { data: credenciais[perfil] })
    const { access_token, expires_in } = await res.json()

    this.tokenCache.set(perfil, {
      token: access_token,
      expiry: Date.now() + (expires_in - 60) * 1000 // renovar 1min antes
    })
    return access_token
  }
}
```

#045 Testar proteção contra enumeração de usuários

Sênior

Timing attack: não revelar se email existe via tempo de resposta

```
test('tempo de resposta igual para email existente e inexistente', async ({ request
}) => {
  const amostras = { existente: [] as number[], inexistente: [] as number[] }

  for (let i = 0; i < 10; i++) {
    let t = Date.now()
    await request.post('/api/auth/login', {
      data: { email: 'existente@email.com', senha: 'senha_errada' }
    })
    amostras.existente.push(Date.now() - t)

    t = Date.now()
    await request.post('/api/auth/login', {
      data: { email: `inexistente_${i}@naoexiste.com`, senha: 'senha_errada' }
    })
    amostras.inexistente.push(Date.now() - t)
  }

  const mediaExistente = amostras.existente.reduce((a, b) => a + b) / 10
  const mediaInexistente = amostras.inexistente.reduce((a, b) => a + b) / 10
  const diferenca = Math.abs(mediaExistente - mediaInexistente)

  console.log(`Tempo médio existente: ${mediaExistente.toFixed(0)}ms, inexistente:
${mediaInexistente.toFixed(0)}ms`)
  // Diferença > 200ms indica timing attack possível
  expect(diferenca).toBeLessThan(200)
})
```

#046 Testar que campos de senha nunca aparecem em respostas **Sênior**

■ *Password field leaking é falha gravíssima de segurança*

```
test('campo senha nunca retornado em nenhum endpoint', async ({ request }) => {
  const headers = { Authorization: `Bearer ${process.env.TOKEN_ADMIN}` }
  const endpoints = [
    '/api/usuarios',
    '/api/usuarios/1',
    '/api/me',
    '/api/usuarios/1/perfil',
  ]

  for (const endpoint of endpoints) {
    const res = await request.get(endpoint, { headers })
    if (res.status() === 200) {
      const texto = await res.text()
      const campos_sensiveis = ['senha', 'password', 'hash', 'salt', 'secret']
      for (const campo of campos_sensiveis) {
        expect(texto.toLowerCase(), `${endpoint} expõe campo: ${campo}`)
          .not.toContain(`"${campo}":`) // como chave JSON
      }
    }
  }
})
```

#047 Testar limite de requisições por IP (throttling) — Sênior

Throttling por IP protege contra DDoS e ataques de força bruta

```
test('throttling por IP — login bloqueado após muitas tentativas', async ({ request }) => {
  const tentativas = 20
  const respostas: number[] = []

  for (let i = 0; i < tentativas; i++) {
    const res = await request.post('/api/auth/login', {
      data: { email: 'alvo@email.com', senha: `tentativa_${i}` }
    })
    respostas.push(res.status())
    if (res.status() === 429) break // rate limited
  }

  // Deve ter bloqueado antes de 20 tentativas
  const bloqueado = respostas.includes(429)
  if (bloqueado) {
    const qtdAntesBloqueio = respostas.indexOf(429)
    console.log(`Bloqueado após ${qtdAntesBloqueio} tentativas`)
    expect(qtdAntesBloqueio).toBeLessThan(15) // bloquear antes de 15 tentativas
  } else {
    console.warn('ATENÇÃO: 20 tentativas de login sem bloqueio — vulnerabilidade brute force')
  }
})
```

#048 Testar compressão e desempenho em payload grande

Sênior

Performance com dados reais — não apenas dados de teste mínimos

```
test('listagem grande responde em < 2s', async ({ request }) => {
  const headers = { Authorization: `Bearer ${process.env.TOKEN}` }

  const tempos: number[] = []
  for (let i = 0; i < 5; i++) {
    const t0 = Date.now()
    const res = await request.get('/api/relatorios/transacoes?limite=1000', {
headers })
    tempos.push(Date.now() - t0)
    expect(res.status()).toBe(200)
    const body = await res.json()
    expect(body.data.length).toBeLessThanOrEqual(1000)
  }

  const p95 = tempos.sort((a, b) => a - b)[Math.floor(tempos.length * 0.95)]
  console.log(`Tempos: ${tempos.join(', ')}ms | P95: ${p95}ms`)
  expect(p95).toBeLessThan(2000)
})
```

#049 Testar API com certificado mTLS (mutual TLS) — Sênior

mTLS é padrão em sistemas financeiros B2B

```
// playwright.config.js para mTLS:
// use: {
// clientCertificates: [{
// origin: 'https://api.parceiro.com',
// certPath: './certs/client.pem',
// keyPath: './certs/client.key',
// }]
// }

test('autenticação mTLS com certificado válido', async ({ request }) => {
  // Com playwright.config correto, o cert é enviado automaticamente
  const res = await request.get('https://api.parceiro.com/api/dados')
  expect(res.status()).toBe(200)
})

test('rejeitar sem certificado de cliente', async ({ browser }) => {
  // Contexto sem certificado
  const ctx = await browser.newContext() // sem clientCertificates
  const page = await ctx.newPage()
  const res = await page.request.get('https://api.parceiro.com/api/dados')
  expect([400, 401, 403]).toContain(res.status())
  await ctx.close()
})
```

#050 Testar API de notificações — preferências do usuário respeitadas

Sênior

Opt-out de notificações deve ser respeitado pela API

```
test('API respeita opt-out de notificações', async ({ request }) => {
  const headers = { Authorization: `Bearer ${process.env.TOKEN}` }

  // Desativar notificações por email
  await request.patch('/api/me/preferencias', {
    data: { notificacoes: { email: false, push: true, sms: false } },
    headers
  })

  // Realizar ação que geraria notificação por email
  await request.post('/api/pedidos', {
    data: { produto_id: 1, quantidade: 1 }, headers
  })

  // Verificar que notificação por email NÃO foi enfileirada
  const notifs = await request.get('/api/test/notificacoes-enfileiradas', { headers
})
  const body = await notifs.json()
  const emailNotifs = body.filter((n: any) => n.canal === 'email')
  expect(emailNotifs.length).toBe(0)
})
```

50 Exercícios SQL para QA — Gabarito Completo

Todos os exercícios usam PostgreSQL (sintaxe compatível com MySQL). Foco em validações reais que QA executa no banco.

Nível Júnior — Exercícios 001 a 020

#001 Selecionar todos os usuários ativos — **Júnior**

■ *Básico — mas teste o WHERE sempre*

```
SELECT id, nome, email, perfil
FROM usuarios
WHERE ativo = TRUE
ORDER BY nome ASC;
```

#002 Verificar se um usuário específico foi criado após cadastro — **Júnior**

■ *Use email como chave de busca — ID pode ser sequencial*

```
SELECT id, nome, email, criado_em, ativo
FROM usuarios
WHERE email = 'novo.usuario@teste.com';

-- Verificar que foi criado hoje:
SELECT * FROM usuarios
WHERE email = 'novo.usuario@teste.com'
  AND DATE(criado_em) = CURRENT_DATE;
```

#003 Contar quantos registros existem em uma tabela — **Júnior**

■ *Validação de quantidade antes e depois de ação*

```
-- Contagem total
SELECT COUNT(*) AS total_usuarios FROM usuarios;

-- Por status
SELECT ativo, COUNT(*) AS total
FROM usuarios
GROUP BY ativo;
```

#004 Verificar que não há emails duplicados na tabela — **Júnior**

■ *Restrição de unicidade deve ser testada*

```
SELECT email, COUNT(*) AS ocorrencias
FROM usuarios
GROUP BY email
HAVING COUNT(*) > 1;

-- Resultado esperado: 0 linhas (sem duplicatas)
```

#005 Buscar os 10 registros mais recentes — **Júnior**

■ *Use ORDER BY + LIMIT para dados recentes*

```
SELECT id, nome, criado_em
FROM usuarios
ORDER BY criado_em DESC
LIMIT 10;
```

#006 Verificar pedido após criação via sistema — **Júnior**

■ *Valide todos os campos do pedido, não só o ID*

```
SELECT
  p.id,
  p.status,
  p.valor_total,
  p.criado_em,
  u.email AS cliente
FROM pedidos p
INNER JOIN usuarios u ON u.id = p.usuario_id
WHERE p.id = 12345;
```

#007 Verificar saldo de conta após transação — **Júnior**

■ *Dado financeiro — precisão decimal importa*

```
SELECT
  conta_id,
  saldo_atual,
  ROUND(saldo_atual, 2) AS saldo_formatado
FROM contas
WHERE conta_id = 1001;


-- Verificar que saldo mudou corretamente:
-- Se saldo era 1000 e você transferiu 50, espera 950.00
```

#008 Buscar registros de log de auditoria de uma ação — **Júnior**

■ *Toda ação crítica deve gerar log — valide no banco*

```
SELECT
  acao,
  tabela_afetada,
  usuario_id,
  dados_antes,
  dados_depois,
  executado_em
FROM auditoria
WHERE tabela_afetada = 'pedidos'
  AND acao = 'UPDATE'
ORDER BY executado_em DESC
LIMIT 5;
```

#009 Verificar que registro foi removido após delete — **Júnior**

■ *Soft delete (is_deleted=true) vs hard delete*

```
-- Hard delete: não deve mais existir
SELECT COUNT(*) AS ainda_existe
FROM usuarios
WHERE id = 99;
-- Esperado: 0

-- Soft delete: campo marcado como deletado
SELECT id, deletado_em, ativo
FROM usuarios
WHERE id = 99;
-- Esperado: deletado_em preenchido, ativo = FALSE
```

#010 Calcular valor total de pedidos por status — **Júnior**

■ *GROUP BY + SUM — relatórios de negócio*

```
SELECT
  status,
  COUNT(*) AS quantidade,
  SUM(valor_total) AS valor_total,
  AVG(valor_total) AS ticket_medio
FROM pedidos
GROUP BY status
ORDER BY valor_total DESC;
```

#011 Verificar dados de endereço após cadastro — **Júnior**

■ *JOINs são essenciais para validar dados relacionados*

```
SELECT
  u.nome,
  e.logradouro,
  e.numero,
  e.bairro,
  e.cidade,
  e.estado,
  e.cep
FROM usuarios u
INNER JOIN enderecos e ON e.usuario_id = u.id
WHERE u.id = 42;
```

#012 Verificar que trigger de estoque funcionou — **Júnior**

■ *Após venda, estoque deve ter diminuído*

```
-- Antes da venda: estoque = 100
-- Venda de 3 unidades do produto 15
SELECT
  produto_id,
  quantidade_atual,
  ultima_atualizacao
FROM estoque
WHERE produto_id = 15;
-- Esperado: quantidade_atual = 97
```

#013 Listar usuários sem nenhum pedido (LEFT JOIN) — **Júnior**

■ *LEFT JOIN é fundamental para encontrar ausências*

```
SELECT
  u.id,
  u.nome,
  u.email
FROM usuarios u
LEFT JOIN pedidos p ON p.usuario_id = u.id
WHERE p.id IS NULL
  AND u.ativo = TRUE;
```

#014 Verificar que senha está armazenada como hash (não plain text) — Júnior

Segurança básica — senha nunca deve ser legível

```
SELECT
  id,
  email,
  LENGTH(senha_hash) AS tamanho_hash
FROM usuarios
WHERE id = 1;

-- BCrypt: sempre 60 chars
-- SHA256: sempre 64 chars hex
-- Argon2: começa com '$argon2'
-- Nunca deve ser 'Senha@123' ou similar
```

#015 Verificar integridade referencial — FK válida — Júnior

Pedido deve sempre ter um usuário válido

```
SELECT p.id, p.usuario_id
FROM pedidos p
LEFT JOIN usuarios u ON u.id = p.usuario_id
WHERE u.id IS NULL;

-- Resultado esperado: 0 linhas
-- Qualquer linha indica FK inválida — bug grave
```

#016 Buscar registros criados em um intervalo de datas — Júnior

BETWEEN para range de datas

```
SELECT COUNT(*) AS cadastros_semana
FROM usuarios
WHERE criado_em BETWEEN
  CURRENT_DATE - INTERVAL '7 days'
  AND CURRENT_DATE + INTERVAL '1 day';
```

#017 Verificar que CPF não foi salvo com máscara — **Júnior**

■ *CPF no banco deve ser só números — sem pontos/traços*

```
SELECT id, cpf
FROM usuarios
WHERE cpf LIKE '%.%' -- tem ponto
   OR cpf LIKE '%-%'; -- tem traço


-- Resultado esperado: 0 linhas
-- CPF correto: 12345678900 (11 dígitos)
```

#018 Calcular percentual de pedidos por status — **Júnior**

■ *Métricas de negócio calculadas direto no banco*

```
SELECT
  status,
  COUNT(*) AS qtd,
  ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER(), 2) AS percentual
FROM pedidos
GROUP BY status
ORDER BY qtd DESC;
```

#019 Verificar que e-mail está em minúsculas — **Júnior**

■ *Normalização de dados — case sensitivity em buscas*

```
SELECT id, email
FROM usuarios
WHERE email != LOWER(email);


-- Resultado esperado: 0 linhas
-- Emails com maiúsculas indicam bug na normalização
```

#020 Buscar o último login de cada usuário — **Júnior**

■ *Subconsulta ou window function*

```
-- Com subconsulta
SELECT u.id, u.email, MAX(l.criado_em) AS ultimo_login
FROM usuarios u
LEFT JOIN logs_acesso l ON l.usuario_id = u.id
GROUP BY u.id, u.email
ORDER BY ultimo_login DESC NULLS LAST;
```

Nível Pleno — Exercícios 021 a 040

#021 Detectar registros duplicados com critério composto — **Pleno**

■ *Duplicata por nome + CPF — não só por campo único*

```
SELECT nome, cpf, COUNT(*) AS duplicatas
FROM usuarios
GROUP BY nome, cpf
HAVING COUNT(*) > 1;
```

#022 Verificar conciliação financeira — débitos = créditos — **Pleno**

■ *Regra contábil fundamental em sistemas financeiros*

```
SELECT
  SUM(CASE WHEN tipo = 'CREDITO' THEN valor ELSE 0 END) AS total_creditos,
  SUM(CASE WHEN tipo = 'DEBITO' THEN valor ELSE 0 END) AS total_debitos,
  SUM(CASE WHEN tipo = 'CREDITO' THEN valor
           WHEN tipo = 'DEBITO' THEN -valor
           ELSE 0 END) AS saldo
FROM transacoes
WHERE DATE(criado_em) = CURRENT_DATE;
```

#023 Verificar ranking dos 5 clientes que mais compraram — Pleno

■ *Window functions — RANK() e DENSE_RANK()*

```
SELECT
  u.nome,
  COUNT(p.id) AS total_pedidos,
  SUM(p.valor_total) AS valor_gasto,
  RANK() OVER (ORDER BY SUM(p.valor_total) DESC) AS ranking
FROM usuarios u
INNER JOIN pedidos p ON p.usuario_id = u.id
WHERE p.status = 'APROVADO'
GROUP BY u.id, u.nome
ORDER BY ranking
LIMIT 5;
```

#024 Verificar que dados sensíveis estão mascarados — Pleno

■ *LGPD — PII deve estar protegido em logs e relatórios*

```
-- Verificar tabela de logs — não deve ter dados sensíveis raw
SELECT *
FROM logs_sistema
WHERE mensagem LIKE '%123.456.789%' -- CPF raw
   OR mensagem LIKE '%@%' -- email raw em log
   OR mensagem ILIKE '%senha%'; -- senha em qualquer forma

-- Resultado esperado: 0 linhas
```

#025	Calcular churn mensal de clientes	Pleno

■ *Métrica de negócio que QA pode validar diretamente*

```
WITH clientes_ativos_mes AS (
  SELECT
    DATE_TRUNC('month', criado_em) AS mes,
    COUNT(*) AS novos
  FROM usuarios
  GROUP BY mes
),
clientes_churn AS (
  SELECT
    DATE_TRUNC('month', cancelado_em) AS mes,
    COUNT(*) AS cancelamentos
  FROM usuarios
  WHERE cancelado_em IS NOT NULL
  GROUP BY mes
)
SELECT a.mes, a.novos, COALESCE(c.cancelamentos, 0) AS cancelamentos
FROM clientes_ativos_mes a
LEFT JOIN clientes_churn c ON c.mes = a.mes
ORDER BY a.mes DESC;
```

Nível Sênior — Exercícios 041 a 050

#041 Detectar anomalia: transação acima do limite diário

Sênior

Regras de negócio financeiro validadas em SQL

```
SELECT
  usuario_id,
  DATE(criado_em) AS data,
  SUM(valor) AS total_dia,
  COUNT(*) AS qtd_transacoes
FROM transacoes
WHERE tipo = 'DEBITO'
  AND DATE(criado_em) >= CURRENT_DATE - INTERVAL '7 days'
GROUP BY usuario_id, DATE(criado_em)
HAVING SUM(valor) > 10000 -- limite diário por regulação
ORDER BY total_dia DESC;
```

#042 Verificar SLA de processamento — tempo médio por etapa

Sênior

Performance de processos batch

```
SELECT
  etapa,
  COUNT(*) AS total,
  AVG(EXTRACT(EPOCH FROM (fim - inicio))) AS media_segundos,
  PERCENTILE_CONT(0.95) WITHIN GROUP
    (ORDER BY EXTRACT(EPOCH FROM (fim - inicio))) AS p95_segundos,
  MAX(EXTRACT(EPOCH FROM (fim - inicio))) AS max_segundos
FROM processamentos
WHERE DATE(inicio) = CURRENT_DATE
GROUP BY etapa
ORDER BY p95_segundos DESC;
```

#043 Identificar gargalo em fila de processamento

Sênior

Mensagens antigas na fila = problema de performance

```
SELECT
  status,
  COUNT(*) AS total,
  MIN(criado_em) AS mais_antiga,
  EXTRACT(EPOCH FROM (NOW() - MIN(criado_em)))/3600 AS horas_aguardando
FROM fila_processamento
WHERE status IN ('PENDENTE', 'PROCESSANDO')
GROUP BY status
HAVING EXTRACT(EPOCH FROM (NOW() - MIN(criado_em)))/3600 > 1;
-- Alerta se tem mensagem esperando > 1 hora
```

#044 Verificar integridade de dados em migração

Sênior

Após migração, count + checksum devem bater

```
-- Na origem
SELECT
  COUNT(*) AS total_origem,
  SUM(valor_total) AS soma_origem
FROM pedidos_origem;

-- No destino (deve ser igual)
SELECT
  COUNT(*) AS total_destino,
  SUM(valor_total) AS soma_destino
FROM pedidos_destino;

-- Verificar divergências por ID
SELECT o.id
FROM pedidos_origem o
LEFT JOIN pedidos_destino d ON d.id = o.id
WHERE d.id IS NULL; -- registros não migrados
```

#045 Verificar snapshots de dados para testes de regressão

Sênior

Comparar estado atual com estado esperado via tabela de referência

```
-- Criar snapshot do estado esperado
CREATE TABLE snapshot_esperado AS
SELECT id, nome, email, perfil, ativo
FROM usuarios WHERE id <= 100;

-- Comparar com estado atual
SELECT
  COALESCE(e.id, a.id) AS id,
  CASE
    WHEN e.id IS NULL THEN 'INSERIDO_INESPERADO'
    WHEN a.id IS NULL THEN 'DELETADO_INESPERADO'
    WHEN e.nome != a.nome OR e.email != a.email THEN 'MODIFICADO'
    ELSE 'OK'
  END AS status_divergencia
FROM snapshot_esperado e
FULL OUTER JOIN usuarios a ON a.id = e.id
WHERE e.id IS NULL OR a.id IS NULL
   OR e.nome != a.nome OR e.email != a.email;
-- Esperado: 0 linhas (sem divergências)
```

#046 Detectar usuários com comportamento suspeito (possível fraude)

Sênior

■ *Padrões de fraude detectáveis via SQL*

```sql
SELECT
  usuario_id,
  COUNT(*) AS tentativas_login,
  COUNT(DISTINCT ip_origem) AS ips_distintos,
  COUNT(DISTINCT EXTRACT(HOUR FROM criado_em)) AS horas_distintas
FROM logs_acesso
WHERE DATE(criado_em) = CURRENT_DATE
  AND sucesso = FALSE
GROUP BY usuario_id
HAVING COUNT(*) > 5 -- mais de 5 falhas hoje
   OR COUNT(DISTINCT ip_origem) > 3; -- mais de 3 IPs
```

#047 Verificar consistência de estoque após batch de vendas

Sênior

■ *Estoque não pode ficar negativo — integridade de negócio*

```sql
SELECT
  p.id AS produto_id,
  p.nome,
  e.quantidade_atual,
  SUM(ip.quantidade) AS total_vendido,
  e.quantidade_atual - SUM(ip.quantidade) AS estoque_calculado
FROM produtos p
INNER JOIN estoque e ON e.produto_id = p.id
INNER JOIN itens_pedido ip ON ip.produto_id = p.id
INNER JOIN pedidos pd ON pd.id = ip.pedido_id
WHERE pd.status = 'APROVADO'
  AND DATE(pd.criado_em) = CURRENT_DATE
GROUP BY p.id, p.nome, e.quantidade_atual
HAVING e.quantidade_atual < 0;
```

#048 Analisar performance de query com EXPLAIN ANALYZE

Sênior

QA sênior identifica queries lentas antes de ir para produção

```
EXPLAIN ANALYZE
SELECT
  u.nome,
  COUNT(p.id) AS total_pedidos,
  SUM(p.valor_total) AS valor_total
FROM usuarios u
INNER JOIN pedidos p ON p.usuario_id = u.id
WHERE p.criado_em >= NOW() - INTERVAL '30 days'
GROUP BY u.id, u.nome
ORDER BY valor_total DESC
LIMIT 100;

-- Verificar no plano:
-- Seq Scan em tabela grande = falta de índice
-- Nested Loop com muitas linhas = problema de JOIN
-- Index Scan = sinal de boa performance
```

#049 Criar procedure de limpeza de dados de teste

Sênior

Nunca deixe dados de teste em produção

```
CREATE OR REPLACE PROCEDURE limpar_dados_teste()
LANGUAGE plpgsql AS $$
BEGIN
  -- Deletar usuários de teste
  DELETE FROM usuarios
  WHERE email LIKE '%@qa.local'
     OR email LIKE '%@teste.com'
     OR nome LIKE 'Usuário Teste%';

  -- Deletar pedidos órfãos
  DELETE FROM pedidos
  WHERE usuario_id NOT IN (SELECT id FROM usuarios);

  RAISE NOTICE 'Limpeza concluída: % usuários removidos', ROW_COUNT;
END;
$$;

-- Executar:
CALL limpar_dados_teste();
```

#050 Criar view de monitoramento de saúde do banco

Sênior

View que o time de QA pode consultar a qualquer momento

```
CREATE OR REPLACE VIEW vw_saude_sistema AS
SELECT
  'usuarios_ativos' AS metrica,
  COUNT(*)::text AS valor
FROM usuarios WHERE ativo = TRUE
UNION ALL
SELECT 'pedidos_pendentes', COUNT(*)::text
FROM pedidos WHERE status = 'PENDENTE'
UNION ALL
SELECT 'erros_ultima_hora', COUNT(*)::text
FROM logs_sistema
WHERE nivel = 'ERROR'
  AND criado_em >= NOW() - INTERVAL '1 hour'
UNION ALL
SELECT 'jobs_travados', COUNT(*)::text
FROM jobs
WHERE status = 'PROCESSANDO'
  AND iniciado_em < NOW() - INTERVAL '30 minutes';

-- Consultar:
SELECT * FROM vw_saude_sistema;
```

50 Exercícios Mobile — Gabarito Completo (Appium)

Todos os exercícios usam Appium com Python + UiAutomator2 (Android). Adapte capabilities para iOS trocando UiAutomator2 por XCUITest.

Nível Júnior — Exercícios 001 a 020

#001 Abrir o app e verificar que a tela inicial carregou — **Júnior**

Smoke test mobile — o mais básico de todos

```
from appium import webdriver
from appium.options import UiAutomator2Options

options = UiAutomator2Options()
options.app = './apps/meu_app.apk'
options.device_name = 'emulator-5554'

driver = webdriver.Remote('http://localhost:4723', options=options)
driver.implicitly_wait(10)

# Verificar elemento da tela inicial
tela_inicial = driver.find_element('accessibility id', 'tela-home')
assert tela_inicial.is_displayed(), 'Tela inicial não carregou'
driver.quit()
```

#002 Fazer login no app com credenciais válidas — **Júnior**

■ *Fluxo crítico — deve estar no smoke test*

```
campo_email = driver.find_element('accessibility id', 'campo-email')
campo_senha = driver.find_element('accessibility id', 'campo-senha')
btn_entrar = driver.find_element('accessibility id', 'btn-entrar')

campo_email.send_keys('qa@teste.com')
campo_senha.send_keys('Senha@123')
btn_entrar.click()

# Aguardar tela pós-login
from appium.webdriver.common.appiumby import AppiumBy
dashboard = driver.find_element(AppiumBy.ACCESSIBILITY_ID, 'dashboard-titulo')
assert dashboard.is_displayed()
```

#003 Verificar texto de um elemento na tela — **Júnior**

■ *Use get_attribute('text') para Android*

```
titulo = driver.find_element('accessibility id', 'titulo-pagina')
texto_atual = titulo.get_attribute('text')
assert texto_atual == 'Bem-vindo!', f'Esperado: Bem-vindo!, recebido:
{texto_atual}'
```

#004 Clicar em botão e verificar navegação para nova tela — **Júnior**

■ *Navegação é um dos fluxos mais testados em mobile*

```
btn_cadastro = driver.find_element('accessibility id', 'btn-cadastrar')
btn_cadastro.click()

# Verificar que navegou para tela de cadastro
titulo_cadastro = driver.find_element('accessibility id', 'titulo-cadastro')
assert titulo_cadastro.is_displayed()
assert titulo_cadastro.get_attribute('text') == 'Criar Conta'
```

#005 Preencher formulário e submeter

Júnior

Fluxo completo de cadastro mobile

```
driver.find_element('accessibility id', 'campo-nome').send_keys('QA Tester')
driver.find_element('accessibility id', 'campo-email').send_keys('qa@teste.com')
driver.find_element('accessibility id', 'campo-senha').send_keys('Senha@123')

# Fechar teclado
driver.hide_keyboard()

driver.find_element('accessibility id', 'btn-salvar').click()

# Verificar mensagem de sucesso
toast = driver.find_element('xpath', "//*[@text='Cadastro realizado!']")
assert toast.is_displayed()
```

#006 Fazer scroll vertical para baixo

Júnior

Scroll é uma das interações mais comuns em mobile

```
from appium.webdriver.common.touch_action import TouchAction

# Método 1: swipe
driver.swipe(
  start_x=540, start_y=1200,
  end_x=540, end_y=400,
  duration=800
)

# Método 2: scroll_to com UiScrollable (Android)
elemento = driver.find_element(
  'android uiautomator',
  'new UiScrollable(new UiSelector().scrollable(true))'
  '.scrollIntoView(new UiSelector().text("Configurações"))'
)
assert elemento.is_displayed()
```

#007 Verificar se app está em foreground (ativo) — Júnior

Estado do app — útil em testes de background/foreground

```
from appium.webdriver.applicationstate import ApplicationState

estado = driver.query_app_state('com.meusistema.app')
assert estado == ApplicationState.RUNNING_IN_FOREGROUND, \
  f'App não está em foreground: {estado}'
```

#008 Rotacionar device para landscape e verificar layout — Júnior

Orientação é requisito de qualidade para mobile

```
# Landscape
driver.orientation = 'LANDSCAPE'
import time; time.sleep(1)

# Verificar que elementos críticos ainda estão visíveis
btn_login = driver.find_element('accessibility id', 'btn-entrar')
assert btn_login.is_displayed(), 'Botão sumiu em landscape'

# Voltar para portrait
driver.orientation = 'PORTRAIT'
```

#009 Verificar comportamento ao pressionar botão Voltar — Júnior

Botão back é específico do Android — sempre teste

```
driver.find_element('accessibility id', 'btn-detalhes').click()
# Está na tela de detalhes
assert driver.find_element('accessibility id', 'tela-detalhes').is_displayed()

# Pressionar botão back do Android
driver.back()

# Deve voltar para lista
assert driver.find_element('accessibility id', 'tela-lista').is_displayed()
```

#010 Verificar que campo de senha oculta os caracteres

Júnior

Segurança visual — campo senha deve ter input type password

```
campo_senha = driver.find_element('accessibility id', 'campo-senha')
campo_senha.send_keys('Senha@123')

# Verificar tipo do campo — Android
input_type = campo_senha.get_attribute('password')
assert input_type == 'true', 'Campo senha não está mascarando'
```

#011 Verificar deep link abre tela correta

Júnior

Deep links são críticos para notificações e marketing

```
# Abrir deep link
driver.execute_script('mobile: deepLink', {
  'url': 'meusistema://produto/123',
  'package': 'com.meusistema.app'
})

import time; time.sleep(1)
titulo = driver.find_element('accessibility id', 'titulo-produto')
assert titulo.is_displayed()
```

#012 Tirar screenshot e salvar como evidência

Júnior

Evidências são obrigatórias em QA mobile

```
# Screenshot simples
driver.save_screenshot('./evidencias/tela_login.png')

# Verificar que arquivo foi criado
import os
assert os.path.exists('./evidencias/tela_login.png'), 'Screenshot não foi salvo'
```

#013 Verificar que teclado aparece ao focar em campo de texto — **Júnior**

Teclado nativo pode cobrir elementos — teste sempre

```
campo = driver.find_element('accessibility id', 'campo-busca')
campo.click()

import time; time.sleep(0.5)

# Verificar que teclado está visível
teclado_visivel = driver.is_keyboard_shown()
assert teclado_visivel, 'Teclado não apareceu ao focar no campo'
```

#014 Verificar mensagem de erro ao submeter formulário vazio — **Júnior**

Validação de campos obrigatórios

```
btn_login = driver.find_element('accessibility id', 'btn-entrar')
btn_login.click()

# Verificar mensagem de erro
msg_erro = driver.find_element('accessibility id', 'msg-email-obrigatorio')
assert msg_erro.is_displayed()
assert 'obrigatório' in msg_erro.get_attribute('text').lower()
```

#015 Verificar que app persiste dados após fechar e reabrir — **Júnior**

Persistência local — SharedPreferences, SQLite, Room

```
# Preencher e salvar dados
driver.find_element('accessibility id', 'campo-nome').send_keys('QA Test')
driver.find_element('accessibility id', 'btn-salvar').click()

# Fechar o app
driver.terminate_app('com.meusistema.app')
import time; time.sleep(2)

# Reabrir o app
driver.activate_app('com.meusistema.app')
time.sleep(2)

# Verificar que dado foi mantido
campo = driver.find_element('accessibility id', 'campo-nome')
assert campo.get_attribute('text') == 'QA Test'
```

Nível Pleno/Sênior — Exercícios 016 a 050

#016 Simular localização GPS e verificar funcionalidade

Pleno

GPS é permissão crítica — simule coordenadas reais

```
# Simular localização: Belo Horizonte - Praça da Liberdade
driver.set_location(
  latitude=-19.9319,
  longitude=-43.9375,
  altitude=858.0
)

import time; time.sleep(1)

# Verificar que cidade foi detectada
cidade = driver.find_element('accessibility id', 'cidade-detectada')
assert 'Belo Horizonte' in cidade.get_attribute('text')
```

#017 Simular biometria (fingerprint) bem-sucedida

Pleno

Biometria: fingerId entre 1-10

```
# Aguardar prompt de biometria
import time; time.sleep(1)

# Simular impressão digital bem-sucedida
driver.finger_print(1)
time.sleep(1)

# Verificar acesso liberado
dashboard = driver.find_element('accessibility id', 'dashboard')
assert dashboard.is_displayed()
```

#018 Testar swipe em carrossel de imagens

Pleno

■ *Carrosseis são componentes com bugs frequentes*

```
# Verificar imagem inicial
img1 = driver.find_element('accessibility id', 'banner-1')
assert img1.is_displayed()

# Swipe para próxima imagem
driver.swipe(start_x=900, start_y=500, end_x=100, end_y=500, duration=500)
import time; time.sleep(0.5)

# Verificar que imagem mudou
img2 = driver.find_element('accessibility id', 'banner-2')
assert img2.is_displayed()
assert not img1.is_displayed()
```

#019 Verificar comportamento em modo avião

Pleno

■ *Apps devem ter UX de offline — não só crash*

```
# Ativar modo avião
driver.set_network_connection(1) # 0=none, 1=avião, 2=wifi, 4=dados, 6=tudo

import time; time.sleep(1)

# Tentar ação que requer rede
driver.find_element('accessibility id', 'btn-atualizar').click()
time.sleep(2)

# Verificar mensagem de offline amigável
msg = driver.find_element('accessibility id', 'msg-sem-conexao')
assert msg.is_displayed()
assert 'sem conexão' in msg.get_attribute('text').lower()

# Restaurar conexão
driver.set_network_connection(6)
```

#020 Testar recebimento de push notification **Pleno**

■ *Notificações têm payload — verifique conteúdo e ação*

```
import subprocess

# Enviar push via ADB (para teste local)
subprocess.run([
  'adb', 'shell', 'am', 'broadcast',
  '-a', 'com.meusistema.PUSH_TEST',
  '--es', 'titulo', 'Novo pedido!',
  '--es', 'mensagem', 'Seu pedido #123 foi aprovado'
])

import time; time.sleep(2)

# Abrir painel de notificações
driver.open_notifications()
time.sleep(1)

# Verificar notificação
notif = driver.find_element('xpath', "//*[@text='Novo pedido!']")
assert notif.is_displayed()
```

#025 Testar performance de scroll em lista longa

Pleno

Jank (travamento) em scroll é bug de UX sério

```
import time

# Navegar para lista com muitos itens
driver.find_element('accessibility id', 'menu-produtos').click()
time.sleep(1)

inicio = time.time()

# Fazer 10 scrolls consecutivos
for _ in range(10):
    driver.swipe(540, 1200, 540, 400, duration=300)
    time.sleep(0.2)

tempo_total = time.time() - inicio
print(f'10 scrolls em {tempo_total:.2f}s')

# Verificar que ainda está na lista (não crashou)
lista = driver.find_element('accessibility id', 'lista-produtos')
assert lista.is_displayed()
```

#035 Testar permissão de câmera e captura de foto mock

Sênior

Apps que usam câmera devem tratar permissão negada

```
# Verificar estado atual da permissão
# No Appium Inspector: localizar elemento de permissão

# Aceitar permissão quando sistema solicitar
try:
    btn_permitir = driver.find_element(
        'id', 'com.android.packageinstaller:id/permission_allow_button'
    )
    btn_permitir.click()
except:
    pass # permissão já concedida

# Testar captura (câmera mock via capabilities)
# No capabilities.json adicionar:
# 'appium:mockImageData': '<base64_da_imagem>'
driver.find_element('accessibility id', 'btn-tirar-foto').click()
import time; time.sleep(2)
foto = driver.find_element('accessibility id', 'preview-foto')
assert foto.is_displayed()
```

#050 Criar suite completa de smoke test mobile em < 5 minutos — **Sênior**

Smoke mobile = 10 testes críticos, sem flakiness, máximo 5min

```
# conftest.py — fixture compartilhada
import pytest
from appium import webdriver
from appium.options import UiAutomator2Options

@pytest.fixture(scope='session')
def driver():
    options = UiAutomator2Options()
    options.app = './apps/app.apk'
    options.device_name = 'emulator-5554'
    options.no_reset = True
    d = webdriver.Remote('http://localhost:4723', options=options)
    d.implicitly_wait(8)
    yield d
    d.quit()

# test_smoke.py
def test_01_app_abre(driver):
    assert driver.find_element('accessibility id', 'tela-home').is_displayed()

def test_02_login_valido(driver):
    # ... login
    pass

def test_03_menu_principal(driver):
    # ... verificar itens do menu
    pass

# Rodar: pytest test_smoke.py -v --timeout=300
```

#021 Testar login com autenticação biométrica falha

Pleno

Biometria com ID errado deve redirecionar para senha

```
# Tentar biometria com ID inválido
driver.find_element('accessibility id', 'btn-biometria').click()

import time; time.sleep(1)

# Simular biometria falha (finger_print(10) = dedo não cadastrado)
driver.finger_print(10)
time.sleep(1)

# App deve oferecer fallback para senha
btn_senha = driver.find_element('accessibility id', 'btn-usar-senha')
assert btn_senha.is_displayed(), 'Fallback para senha não apareceu'
```

#022 Verificar que app atualiza badge de notificação

Pleno

Badge é contrato de UX — deve refletir contagem real

```
import time

# Verificar badge inicial
badge = driver.find_element('accessibility id', 'badge-notificacoes')
count_inicial = int(badge.get_attribute('text') or '0')

# Enviar notificação via ADB
import subprocess
subprocess.run(['adb', 'shell', 'am', 'broadcast',
  '-a', 'com.meusistema.NOVA_NOTIFICACAO'])
time.sleep(2)

# Badge deve ter incrementado
badge_novo = driver.find_element('accessibility id', 'badge-notificacoes')
count_novo = int(badge_novo.get_attribute('text') or '0')
assert count_novo == count_inicial + 1, f'Badge não incrementou: {count_inicial} ->
{count_novo}'
```

#023 Testar gestos de pinch e zoom em mapa

Pleno

■ *Pinch/zoom são gestos multi-touch — use W3C Actions*

```
from selenium.webdriver.common.action_chains import ActionChains
from selenium.webdriver.common.actions.pointer_input import PointerInput
from selenium.webdriver.common.actions import interaction

# Calcular centro da tela
size = driver.get_window_size()
cx, cy = size['width'] // 2, size['height'] // 2

# Pinch out (zoom in)
actions = ActionChains(driver)
finger1 = PointerInput(interaction.POINTER_TOUCH, 'dedo1')
finger2 = PointerInput(interaction.POINTER_TOUCH, 'dedo2')

actions.w3c_actions.devices = [finger1, finger2]
actions.w3c_actions.key_action.key_down('finger1').pointer_move(cx-50,
cy).pointer_down().pointer_move(cx-150, cy).pointer_up()

# Verificar que mapa fez zoom
nivel_zoom = driver.find_element('accessibility id', 'mapa-nivel-zoom')
assert nivel_zoom.is_displayed()
```

#024 Testar que app abre URL externa no browser padrão **Pleno**

Links externos não devem abrir dentro do app

```
# Clicar em link para site externo
driver.find_element('accessibility id', 'link-politica-privacidade').click()

import time; time.sleep(2)

# Verificar que saiu do app (browser abriu)
# Em automação: verificar que o pacote atual mudou
current_app = driver.current_package

# Deve estar no browser, não no app
browsers = ['com.android.chrome', 'com.google.android.apps.chrome',
            'org.mozilla.firefox', 'com.sec.android.app.sbrowser']
assert current_app in browsers, f'Link abriu dentro do app em vez do browser: {current_app}'
```

#025 Verificar consumo de bateria durante operações intensas

Pleno

Apps que drenam bateria são desinstalados pelos usuários

```
import time

# Capturar nível de bateria antes
bateria_antes = driver.execute_script('mobile: batteryInfo')
nivel_antes = bateria_antes.get('level', 1.0)

# Executar operação intensiva por 60 segundos
driver.find_element('accessibility id', 'btn-processar-lote').click()
time.sleep(60)

bateria_depois = driver.execute_script('mobile: batteryInfo')
nivel_depois = bateria_depois.get('level', 1.0)

# Queda de bateria não deve ser excessiva
queda = nivel_antes - nivel_depois
print(f'Queda de bateria em 60s: {queda*100:.1f}%')
assert queda < 0.05, f'Consumo excessivo de bateria: {queda*100:.1f}% em 60s'
```

#026 Testar que app não crasha ao receber ligação durante uso

Pleno

Interrupção por ligação é o cenário mais comum — e mais esquecido

```
import subprocess, time

# Estar usando o app ativamente
driver.find_element('accessibility id', 'btn-iniciar-processo').click()
time.sleep(1)

# Simular chamada recebida via ADB
subprocess.run(['adb', 'shell', 'am', 'start',
  '-a', 'android.intent.action.CALL',
  '-d', 'tel:11987654321'])
time.sleep(3)

# Recusar a chamada
try:
    driver.find_element('xpath', "//*[@text='Recusar']").click()
except:
    subprocess.run(['adb', 'shell', 'input', 'keyevent', '6']) # KEYCODE_ENDCALL

time.sleep(1)

# App deve ter voltado ao estado anterior ou ao início — sem crash
from appium.webdriver.applicationstate import ApplicationState
estado = driver.query_app_state('com.meusistema.app')
assert estado != ApplicationState.NOT_RUNNING, 'App crashou após ligação'
```

#027 Verificar que dados sensíveis não aparecem no recent apps

Pleno

■ *Recent apps screenshot pode vazar dados bancários e pessoais*

```
import time

# Navegar para tela com dados sensíveis
driver.find_element('accessibility id', 'menu-extrato').click()
time.sleep(1)

# Verificar que a tela tem proteção de screenshot
# Android: FLAG_SECURE previne captura
# Testar tentativa de screenshot quando na tela sensível
try:
    driver.save_screenshot('./evidencias/extrato.png')
    # Se conseguiu tirar screenshot, verificar que está em branco ou pixelado
    from PIL import Image
    img = Image.open('./evidencias/extrato.png')
    # Em telas com FLAG_SECURE, screenshot fica preta
    pixels = list(img.getdata())
    pixels_pretos = sum(1 for p in pixels[:100] if p[:3] == (0, 0, 0))
    if pixels_pretos > 90:
        print('■ FLAG_SECURE ativo — screenshot bloqueado corretamente')
    else:
        print('■■ Dados sensíveis podem aparecer no screenshot')
except Exception as e:
    print(f'Screenshot bloqueado pela SO: {e}')
```

#028 Testar comportamento ao trocar de SIM card / eSIM

Pleno

■ *Apps de banco verificam SIM — troca deve acionar re-autenticação*

```
import time

# Estado inicial: autenticado
dashboard = driver.find_element('accessibility id', 'dashboard')
assert dashboard.is_displayed()

# Simular troca de número de telefone via broadcast
import subprocess
subprocess.run([
    'adb', 'shell', 'am', 'broadcast',
    '-a', 'android.intent.action.SIM_STATE_CHANGED'
])
time.sleep(2)

# App de banco deve solicitar re-autenticação
reauth = driver.find_element('accessibility id', 'tela-reautenticacao')
if reauth.is_displayed():
    print('■ Re-autenticação solicitada após mudança de SIM')
else:
    print('■■ App não detectou mudança de SIM — verificar política de segurança')
```

#029 Testar acessibilidade com TalkBack ativo

Pleno

■ *TalkBack lê conteúdo em voz alta — elementos devem ter labels*

```
# Ativar TalkBack via ADB
import subprocess
subprocess.run([
    'adb', 'shell', 'settings', 'put', 'secure',
    'enabled_accessibility_services',
    'com.google.android.marvin.talkback/com.google.android.marvin.talkback.TalkBack
Service'
])

import time; time.sleep(2)

# Verificar que elementos críticos têm contentDescription
elementos_criticos = ['btn-pagar', 'btn-transferir', 'btn-extrato', 'campo-valor']
for elem_id in elementos_criticos:
    try:
        elem = driver.find_element('accessibility id', elem_id)
        desc = elem.get_attribute('contentDescription')
        assert desc and len(desc) > 2, f'Elemento {elem_id} sem contentDescription'
        print(f'■ {elem_id}: {desc}')
    except:
        print(f'■■ {elem_id} não encontrado — verificar se ID está correto')

# Desativar TalkBack
subprocess.run(['adb', 'shell', 'settings', 'put', 'secure',
    'enabled_accessibility_services', ''])
```

#030 Testar que app funciona com tamanho de fonte acessível

Pleno

Usuários com deficiência visual aumentam a fonte — layout não pode quebrar

```
import subprocess, time

# Aumentar fonte para o máximo acessível
subprocess.run(['adb', 'shell', 'settings', 'put', 'system', 'font_scale', '1.3'])
time.sleep(1)

# Reiniciar a activity para aplicar
driver.terminate_app('com.meusistema.app')
driver.activate_app('com.meusistema.app')
time.sleep(2)

# Verificar que textos críticos ainda são visíveis e não cortados
elementos = ['titulo-tela', 'btn-principal', 'label-saldo']
for elem_id in elementos:
    elem = driver.find_element('accessibility id', elem_id)
    assert elem.is_displayed(), f'Elemento {elem_id} sumiu com fonte grande'

# Restaurar fonte padrão
subprocess.run(['adb', 'shell', 'settings', 'put', 'system', 'font_scale', '1.0'])
```

#031 Verificar que app limpa dados sensíveis da memória após logout

Pleno

■ *Tokens e senhas não podem persistir na memória após logout*

```
import time

# Fazer logout
driver.find_element('accessibility id', 'btn-logout').click()
time.sleep(1)

# Confirmar logout se necessário
try:
    driver.find_element('accessibility id', 'btn-confirmar-logout').click()
    time.sleep(1)
except:
    pass

# Verificar que voltou para tela de login
tela_login = driver.find_element('accessibility id', 'tela-login')
assert tela_login.is_displayed()

# Tentar acessar área restrita diretamente
driver.execute_script('mobile: deepLink', {
    'url': 'meusistema://dashboard',
    'package': 'com.meusistema.app'
})
time.sleep(1)

# Deve redirecionar para login, não entrar no dashboard
```

```
assert driver.find_element('accessibility id', 'tela-login').is_displayed(), \
    'App permitiu acesso ao dashboard após logout — vazamento de sessão'
```

#032	Testar sincronização após reconectar à internet	Pleno

■ *Dados offline devem sincronizar automaticamente ao reconectar*

```
import time

# Ir offline
driver.set_network_connection(1) # modo avião
time.sleep(1)

# Realizar ação que deve ser sincronizada depois
driver.find_element('accessibility id', 'btn-favoritar-produto').click()
time.sleep(1)

# Verificar indicador de pendente/offline
indicador = driver.find_element('accessibility id', 'sync-pendente')
assert indicador.is_displayed(), 'Não mostrou indicador de sincronização pendente'

# Reconectar
driver.set_network_connection(6) # wifi + dados
time.sleep(3)

# Indicador de pendente deve sumir
try:
    indicador_depois = driver.find_element('accessibility id', 'sync-pendente')
    assert not indicador_depois.is_displayed(), 'Sincronização não ocorreu após reconectar'
except:
    print('■ Elemento de sync sumiu — sincronização realizada')
```

#033 Testar que app usa SSL pinning e rejeita certificado inválido **Pleno**

■ *SSL Pinning protege contra ataques MITM em redes públicas*

```
# SSL Pinning não pode ser testado diretamente pelo Appium
# mas pode ser verificado tentando interceptar com proxy

# Configurar proxy de intercepção (ex: Burp Suite) no device
# Se o app implementa SSL Pinning, todas as requisições falharão

# Teste alternativo: verificar no código/manifest
import subprocess
result = subprocess.run(
    ['adb', 'shell', 'run-as', 'com.meusistema.app',
     'ls', 'assets/'],
    capture_output=True, text=True
)

# App com SSL Pinning geralmente tem arquivo de certificado nos assets
output = result.stdout
print('Assets do app:', output)

# Verificar no manifest se usa NetworkSecurityConfig
manifest_check = subprocess.run(
    ['aapt', 'dump', 'xmltree', './apps/app.apk', 'AndroidManifest.xml'],
    capture_output=True, text=True
)
print('Network security config:', 'networkSecurityConfig' in manifest_check.stdout)
```

#034 Testar que splash screen não bloqueia após cold start

Pleno

Cold start com splash lento = usuário fecha o app

```
import time
from appium.webdriver.applicationstate import ApplicationState

# Cold start: fechar completamente e reabrir
driver.terminate_app('com.meusistema.app')
time.sleep(1)

t_inicio = time.time()
driver.activate_app('com.meusistema.app')

# Aguardar splash sumir e tela real aparecer
tela_principal = driver.find_element('accessibility id', 'tela-login')

# Esperar com timeout
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from appium.webdriver.common.appiumby import AppiumBy

wait = WebDriverWait(driver, 10)
wait.until(EC.visibility_of_element_located(
    (AppiumBy.ACCESSIBILITY_ID, 'tela-login')
))
t_total = time.time() - t_inicio

print(f'Cold start: {t_total:.2f}s')
```

```
assert t_total < 5.0, f'Cold start muito lento: {t_total:.2f}s (máx: 5s)'
```

#036 Implementar fixture de sessão compartilhada entre testes mobile

Sênior

Login uma vez por sessão de testes — não repetir em cada teste

```
# conftest.py
import pytest
from appium import webdriver
from appium.options import UiAutomator2Options

@pytest.fixture(scope='session')
def driver_autenticado():
    opts = UiAutomator2Options()
    opts.app = './apps/app.apk'
    opts.device_name = 'emulator-5554'
    opts.no_reset = True
    opts.full_reset = False
    d = webdriver.Remote('http://localhost:4723', options=opts)
    d.implicitly_wait(10)

    # Login único para toda a sessão
    d.find_element('accessibility id', 'campo-email').send_keys('qa@teste.com')
    d.find_element('accessibility id', 'campo-senha').send_keys('Senha@123')
    d.find_element('accessibility id', 'btn-entrar').click()

    import time; time.sleep(2)
    assert d.find_element('accessibility id', 'dashboard').is_displayed()

    yield d
    d.quit()
```

#037 Testar deep link com parâmetros de query string

Sênior

Deep links com parâmetros são usados em campanhas e notificações

```
# Deep link com parâmetros
driver.execute_script('mobile: deepLink', {
    'url': 'meusistema://produto/456?origem=campanha&cupom=QA10',
    'package': 'com.meusistema.app'
})

import time; time.sleep(1)

# Verificar que abriu o produto correto
titulo = driver.find_element('accessibility id', 'produto-titulo')
assert titulo.is_displayed()

# Verificar que cupom foi aplicado automaticamente
cupom_aplicado = driver.find_element('accessibility id', 'cupom-aplicado')
assert cupom_aplicado.is_displayed()
assert 'QA10' in cupom_aplicado.get_attribute('text')
```

#038 Testar App Links (verificação de domínio Android)

Sênior

App Links abrem o app diretamente — sem chooser dialog

```
import subprocess, time

# Abrir App Link (https em vez de scheme customizado)
subprocess.run([
    'adb', 'shell', 'am', 'start',
    '-a', 'android.intent.action.VIEW',
    '-d', 'https://www.meusistema.com/produto/789',
    '-c', 'android.intent.category.BROWSABLE'
])
time.sleep(2)

# Verificar que app abriu (não o browser)
current_app = driver.current_package
assert current_app == 'com.meusistema.app', \
    f'App Link não abriu o app: {current_app}'
```

#039 Implementar Page Object para telas mobile

Sênior

POM não é só para web — mobile se beneficia igualmente

```
# pages/TelaExtrato.py
class TelaExtrato:
    def __init__(self, driver):
        self.driver = driver
        self._titulo = ('accessibility id', 'titulo-extrato')
        self._filtro_data = ('accessibility id', 'filtro-periodo')
        self._lista_transacoes = ('accessibility id', 'lista-transacoes')
        self._btn_exportar = ('accessibility id', 'btn-exportar-pdf')

    def esperar_carregar(self):
        from selenium.webdriver.support.ui import WebDriverWait
        from selenium.webdriver.support import expected_conditions as EC
        WebDriverWait(self.driver, 10).until(
            EC.visibility_of_element_located(self._lista_transacoes)
        )
        return self

    def filtrar_periodo(self, periodo: str):
        self.driver.find_element(*self._filtro_data).click()
        self.driver.find_element('xpath', f"//*[@text='{periodo}']").click()
        return self

    def count_transacoes(self) -> int:
        return len(self.driver.find_elements(*self._lista_transacoes))
```

```
    def exportar_pdf(self):
        self.driver.find_element(*self._btn_exportar).click()
        return self
```

#040 Testar performance de startup com Perfetto/Systrace

Sênior

Métricas de performance mobile requerem ferramentas nativas

```
import subprocess, time, json

# Limpar cache para cold start real
subprocess.run(['adb', 'shell', 'am', 'force-stop', 'com.meusistema.app'])
subprocess.run(['adb', 'shell', 'am', 'kill', 'com.meusistema.app'])
time.sleep(1)

# Medir com am start -W (retorna tempo de startup)
result = subprocess.run([
    'adb', 'shell', 'am', 'start', '-W',
    '-n', 'com.meusistema.app/.MainActivity'
], capture_output=True, text=True)

output = result.stdout
print('Saída do am start:', output)

# Extrair ThisTime e TotalTime
for linha in output.split(' '):
    if 'ThisTime' in linha:
        this_time = int(linha.split(':')[1].strip())
        print(f'ThisTime (Activity): {this_time}ms')
        assert this_time < 1000, f'Activity lenta: {this_time}ms'
    if 'TotalTime' in linha:
        total_time = int(linha.split(':')[1].strip())
        print(f'TotalTime (Cold Start): {total_time}ms')
        assert total_time < 3000, f'Cold start lento: {total_time}ms'
```

#041 Testar comportamento do app com memória baixa

Sênior

Android mata apps em segundo plano quando memória é baixa

```
import subprocess, time
from appium.webdriver.applicationstate import ApplicationState

# Navegar para tela específica
driver.find_element('accessibility id', 'menu-transacoes').click()
time.sleep(1)

# Simular pressão de memória (envia TRIM_MEMORY)
subprocess.run([
    'adb', 'shell', 'am', 'send-trim-memory',
    'com.meusistema.app', 'RUNNING_CRITICAL'
])
time.sleep(2)

# App não deve ter crashado
estado = driver.query_app_state('com.meusistema.app')
assert estado != ApplicationState.NOT_RUNNING, 'App crashou sob pressão de memória'

# Se foi para background, reativar e verificar estado
if estado != ApplicationState.RUNNING_IN_FOREGROUND:
    driver.activate_app('com.meusistema.app')
    time.sleep(1)

# App deve estar funcional
dashboard = driver.find_element('accessibility id', 'dashboard')
```

```
assert dashboard.is_displayed() or driver.find_element('accessibility id',
'tela-login').is_displayed()
```

#042 Testar atualização OTA — fluxo de update do app

Sênior

In-app update é UX crítico — usuário não pode ficar preso

```
import time

# Simular que nova versão está disponível
# (via feature flag ou endpoint de versão da API)
import subprocess
subprocess.run([
    'adb', 'shell', 'am', 'broadcast',
    '-a', 'com.meusistema.APP_UPDATE_AVAILABLE',
    '--es', 'versao', '2.5.0',
    '--es', 'tipo', 'OPCIONAL'
])
time.sleep(2)

# Verificar banner de atualização
banner = driver.find_element('accessibility id', 'banner-atualizacao')
assert banner.is_displayed()

# Testar 'Agora não' — app deve continuar funcionando
driver.find_element('accessibility id', 'btn-atualizar-depois').click()
time.sleep(1)

# App deve estar utilizável
dashboard = driver.find_element('accessibility id', 'dashboard')
assert dashboard.is_displayed(), 'App bloqueou após recusar atualização opcional'
```

#043 Testar comportamento em modo escuro (Dark Mode) — Sênior

Dark Mode é padrão esperado — elementos devem ser legíveis

```
import subprocess, time

# Ativar Dark Mode via ADB
subprocess.run(['adb', 'shell', 'cmd', 'uimode', 'night', 'yes'])
time.sleep(1)

# Reiniciar activity para aplicar
driver.terminate_app('com.meusistema.app')
driver.activate_app('com.meusistema.app')
time.sleep(2)

# Verificar que elementos críticos ainda são visíveis
elementos = ['titulo-tela', 'btn-principal', 'label-saldo', 'menu-inferior']
for elem_id in elementos:
    elem = driver.find_element('accessibility id', elem_id)
    assert elem.is_displayed(), f'Elemento {elem_id} sumiu no dark mode'

# Screenshot para revisão visual
driver.save_screenshot('./evidencias/dark_mode.png')

# Restaurar light mode
subprocess.run(['adb', 'shell', 'cmd', 'uimode', 'night', 'no'])
```

#044 Testar que app não vaza dados via clipboard

Sênior

Clipboard é compartilhado entre apps — dados sensíveis não devem ir lá

```
import subprocess, time

# Navegar para tela com dados sensíveis
driver.find_element('accessibility id',
'campo-numero-cartao').send_keys('4111111111111111')
time.sleep(0.5)

# Tentar copiar o número do cartão (long press + copy)
campo = driver.find_element('accessibility id', 'campo-numero-cartao')
campo.click()

# Selecionar tudo e copiar
from selenium.webdriver.common.keys import Keys
campo.send_keys(Keys.CONTROL + 'a')
campo.send_keys(Keys.CONTROL + 'c')
time.sleep(0.5)

# Verificar clipboard via ADB
result = subprocess.run(
    ['adb', 'shell', 'service', 'call', 'clipboard', '2'],
    capture_output=True, text=True
)
clipboard_content = result.stdout
print('Conteúdo do clipboard:', clipboard_content[:50])

# Número do cartão NÃO deve estar no clipboard
assert '4111111111111111' not in clipboard_content, \
    'Número do cartão vazou para o clipboard!'
```

#045 Testar internacionalização — app em inglês e português

Sênior

i18n mobile: verificar strings em múltiplos idiomas

```
import subprocess, time

def mudar_idioma_device(codigo: str):
    subprocess.run([
        'adb', 'shell', 'am', 'start',
        '-n', 'com.android.settings/.LanguageSettings'
    ])
    # Em ambiente de CI, usar:
    subprocess.run([
        'adb', 'shell', 'settings', 'put', 'system',
        'system_locales', codigo
    ])
    time.sleep(2)

# Testar em PT-BR
mudar_idioma_device('pt-BR')
driver.terminate_app('com.meusistema.app')
driver.activate_app('com.meusistema.app')
time.sleep(2)

btn_ptbr = driver.find_element('accessibility id', 'btn-entrar')
texto_ptbr = btn_ptbr.get_attribute('text')
assert texto_ptbr in ['Entrar', 'ENTRAR'], f'Texto em PT-BR incorreto:
{texto_ptbr}'

# Testar em EN-US
```

```
mudar_idioma_device('en-US')

driver.terminate_app('com.meusistema.app')

driver.activate_app('com.meusistema.app')

time.sleep(2)


btn_en = driver.find_element('accessibility id', 'btn-entrar')

texto_en = btn_en.get_attribute('text')

assert texto_en in ['Sign In', 'Login', 'SIGN IN'], f'Texto em EN incorreto:
{texto_en}'
```

#046 Testar que app funciona corretamente em tablet (large screen)

Sênior

Apps de telefone em tablet: layout deve ser adaptado, não esticado

```
import subprocess, time


# Verificar tamanho de tela atual
tamanho = driver.get_window_size()
largura = tamanho['width']
altura = tamanho['height']
print(f'Resolução: {largura}x{altura}')


# Se for tablet (largura > 600dp), verificar layout de duas colunas
if largura >= 600:
    print('Dispositivo tablet detectado')
    # Layout de tablet deve usar dois painéis
    painel_esq = driver.find_element('accessibility id', 'painel-lista')
    painel_dir = driver.find_element('accessibility id', 'painel-detalhe')
    assert painel_esq.is_displayed(), 'Painel lista não visível em tablet'
    assert painel_dir.is_displayed(), 'Painel detalhe não visível em tablet'
else:
    print('Dispositivo telefone — layout de coluna única esperado')
    # Não deve ter dois painéis visíveis ao mesmo tempo
    try:
        painel_dir = driver.find_element('accessibility id', 'painel-detalhe')
        assert not painel_dir.is_displayed(), 'Painel detalhe visível em telefone — layout quebrado'
    except:
        pass # elemento não existe em telefone — OK
```

#047 Criar relatório HTML de execução mobile com screenshots **Sênior**

Evidências automáticas por screenshot em cada passo

```
# conftest.py — hook de screenshot automático em falha
import pytest, os
from datetime import datetime

@pytest.hookimpl(tryfirst=True, hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    rep = outcome.get_result()

    if rep.when == 'call' and rep.failed:
        driver = item.funcargs.get('driver') or
item.funcargs.get('driver_autenticado')
        if driver:
            os.makedirs('./evidencias', exist_ok=True)
            ts = datetime.now().strftime('%Y%m%d_%H%M%S')
            nome = f"./evidencias/FALHA_{item.name}_{ts}.png"
            driver.save_screenshot(nome)
            print(f'\nScreenshot salvo: {nome}')

# pytest.ini
# [pytest]
# addopts = -v --html=./relatorios/mobile-report.html --self-contained-html
# testpaths = tests/mobile
```

#048 Testar compatibilidade com Android 12+ (granular media permissions) — Sênior

■ *Android 12+ mudou permissões de mídia — pode quebrar apps antigos*

```
import subprocess, time

# Verificar versão do Android
result = subprocess.run(
    ['adb', 'shell', 'getprop', 'ro.build.version.sdk'],
    capture_output=True, text=True
)
sdk_version = int(result.stdout.strip())
print(f'SDK version: {sdk_version} (Android {sdk_version - 18})')

# Android 12 (SDK 31) ou superior: permissões granulares de mídia
if sdk_version >= 31:
    print('Android 12+ detectado — testando permissões granulares')
    # READ_MEDIA_IMAGES, READ_MEDIA_VIDEO, READ_MEDIA_AUDIO
    driver.find_element('accessibility id', 'btn-selecionar-foto').click()
    time.sleep(1)

    # Verificar prompt de permissão correto
    try:
        btn_permitir = driver.find_element('xpath',
            "//*[@resource-id='com.android.permissioncontroller:id/permission_allow
_button']")
        btn_permitir.click()
        print('■ Permissão granular solicitada corretamente')
    except:
        print('■■ Permissão não solicitada — verificar targetSdkVersion no
manifest')
```

#049 Testar analytics — eventos rastreados corretamente

Sênior

Analytics mal implementados custam decisões de negócio erradas

```
import subprocess, time, json

# Habilitar modo debug do Firebase Analytics via ADB
subprocess.run([
    'adb', 'shell', 'setprop',
    'debug.firebase.analytics.app', 'com.meusistema.app'
])

# Realizar ação rastreável
driver.find_element('accessibility id', 'btn-comprar').click()
time.sleep(2)

# Verificar evento no logcat
result = subprocess.run(
    ['adb', 'logcat', '-d', '-s', 'FA', # tag do Firebase Analytics
     '--format=brief'],
    capture_output=True, text=True, timeout=5
)

log = result.stdout
print('Firebase Analytics log:', log[:500])

# Verificar que evento de compra foi registrado
assert 'purchase' in log.lower() or 'begin_checkout' in log.lower(), \
    'Evento de compra não foi rastreado pelo Analytics'
```

50 Prompts para IA — QA Insider Collection

Todos os prompts usam o formato 'Aja como' para maximizar a qualidade do output. Testados com ChatGPT-4, Claude Sonnet e GitHub Copilot.

Categoria 1 — Geração de Casos de Teste

PROMPT #01 **Cenários de teste para endpoint REST**

```
Aja como um QA Sênior especialista em testes de API REST com 10 anos de experiência. Analise o endpoint abaixo e gere uma lista COMPLETA de cenários de teste cobrindo: fluxo feliz, validações de campo obrigatório, tipos inválidos, autenticação, autorização por perfil, casos de borda, performance básica e segurança (injeção). Use o formato: ID | Cenário | Pré-condição | Dados | Resultado Esperado | Prioridade. Endpoint: [COLE AQUI A ESPECIFICAÇÃO DO ENDPOINT]
```

→ *Substitua [COLE AQUI...] pela documentação real. Quanto mais contexto, melhor o output.*

PROMPT #02 **Cenários BDD para funcionalidade**

```
Aja como um QA Lead praticante de BDD em times ágeis. Baseado na user story abaixo, gere cenários Gherkin (Given/When/Then) cobrindo: caminho feliz, alternativo, exceções de negócio e casos de borda. Cada cenário deve ser independente, testável e com vocabulário de negócio (sem termos técnicos). User Story: [COLE A USER STORY AQUI]
```

→ *BDD funciona melhor quando os cenários usam linguagem de negócio, não de implementação.*

PROMPT #03 **Matriz de teste para regra de negócio complexa**

```
Aja como um QA Arquiteto especialista em design de testes. Analise a regra de negócio abaixo e crie uma matriz de teste usando técnica de partição de equivalência e análise de valor limite. Identifique: classes válidas, inválidas, limites críticos e combinações explosivas. Apresente em formato de tabela. Regra: [DESCREVA A REGRA DE NEGÓCIO]
```

→ *Ideal para regras com muitas condicionais — descontos, elegibilidade, cálculos.*

PROMPT #04 **Cenários de teste de regressão após bug fix**

```
Aja como um QA Sênior fazendo análise de impacto de uma correção de bug. Dado o bug descrito abaixo e a solução implementada, gere: (1) teste que comprova o bug foi corrigido, (2) testes de regressão das funcionalidades adjacentes que podem ter sido impactadas, (3) teste de regressão do mesmo bug em fluxos similares. Bug: [DESCREVA O BUG E A CORREÇÃO]
```

→ *Correções de bug frequentemente introduzem novos bugs — a regressão focada evita isso.*

PROMPT #05 **Casos de teste de segurança OWASP**

```
Aja como um QA de Segurança especialista em OWASP Top 10. Para a funcionalidade descrita abaixo, gere casos de teste de segurança cobrindo os riscos mais relevantes: injection, broken auth, XSS, IDOR, security misconfiguration. Para cada caso: descreva o vetor de ataque, o payload de teste e o comportamento esperado. Funcionalidade: [DESCREVA A FUNCIONALIDADE]
```

→ *QA não precisa ser pentester — precisa verificar que as proteções básicas existem.*

Categoria 2 — Geração de Código de Automação

PROMPT #06 **Converter roteiro manual em código Playwright**

```
Aja como um Engenheiro de Automação especialista em Playwright e TypeScript. Converta o roteiro de teste manual abaixo em código Playwright seguindo: Page Object Model, uso de data-testid como localizador preferencial, asserções explícitas em cada passo, comentários explicativos e tratamento de esperas assíncronas sem hard sleep. Roteiro: [COLE O ROTEIRO MANUAL]
```

→ *Forneça o roteiro completo com pré-condições e dados de teste para melhor output.*

PROMPT #07 **Criar Page Object Model para uma tela**

```
Aja como um QA Architect especialista em padrões de automação. Analise a estrutura HTML abaixo e crie um Page Object Model completo em Playwright/TypeScript: construtor com todos os localizadores, métodos que representam ações do usuário (não cliques isolados), método de verificação de estado da página e JSDoc nos métodos públicos. HTML: [COLE O HTML OU DESCREVA OS ELEMENTOS DA TELA]
```

→ *Bom POM = testes legíveis. Se o método se chama btnSubmit.click(), está errado. Deve ser formulario.submeter().*

PROMPT #08 — **Criar fixtures de dados de teste realistas**

```
Aja como um engenheiro de qualidade criando massa de dados para testes automatizados. Gere [N] objetos JSON com dados brasileiros realistas para: [DESCREVA O OBJETO]. Requisitos: CPFs válidos (com dígitos verificadores corretos), CEPs reais de cidades brasileiras, emails únicos no formato empresa.com, telefones com DDD válido, datas coerentes com a lógica de negócio.
```

→ *Dados irrealistas geram testes frágeis. Dados reais expõem bugs de validação.*

PROMPT #09 — **Implementar helpers de espera customizados**

```
Aja como um Engenheiro de Automação especialista em eliminar flaky tests. Implemente em Playwright/TypeScript um conjunto de helpers de espera para: (1) aguardar requisição de rede específica completar, (2) aguardar elemento ter texto específico, (3) aguardar condição arbitrária com polling customizado, (4) aguardar toast/snackbar aparecer E desaparecer. Cada helper deve ter timeout configurável e mensagem de erro descritiva.
```

→ *Waits customizados são a diferença entre suite confiável e suite que o time ignora.*

PROMPT #10 — **Criar script de setup e teardown de dados via API**

```
Aja como um QA Sênior implementando estratégia de dados de teste. Crie um módulo TypeScript para: (1) criar usuário de teste via POST /api/usuarios retornando o ID, (2) autenticar e retornar token JWT, (3) criar recursos de teste necessários para o suite, (4) limpar todos os recursos criados no teardown. Use padrão factory e garanta idempotência. Base URL: [COLE A URL DA API]
```

→ *Setup/teardown via API é muito mais rápido que via UI — use quando possível.*

Categoria 3 — Análise e Debugging

PROMPT #11 — **Analisar erro de teste e propor solução**

```
Aja como um especialista em Playwright/Cypress com ampla experiência em debugging de testes E2E flaky. Analise o erro abaixo e forneça: (1) diagnóstico da causa raiz, (2) solução imediata, (3) solução estrutural (para não repetir), (4) como reproduzir isoladamente para confirmar o fix. Erro: [COLE O ERRO COMPLETO COM STACK TRACE]
```

→ *Cole o erro COMPLETO — nunca apenas a última linha. Contexto é tudo para o diagnóstico.*

PROMPT #12 **Revisar código de automação como QA Architect**

```
Aja como um QA Architect sênior fazendo code review de automação. Analise o código abaixo e identifique: (1) violações de Page Object Model, (2) localizadores frágeis (sem data-testid, XPath absoluto), (3) hard sleeps e waits incorretos, (4) falta de asserções ou asserções insuficientes, (5) código duplicado que deveria ser extraído, (6) potenciais fontes de flakiness. Para cada problema, mostre o código problemático e a versão corrigida. Código: [COLE O CÓDIGO]
```

→ *Code review de testes é tão importante quanto code review de código de produção.*

PROMPT #13 **Analisar suite de testes e identificar gaps de cobertura**

```
Aja como um QA Lead fazendo auditoria de cobertura de testes. Analise os cenários de teste existentes abaixo e identifique: (1) fluxos não cobertos, (2) casos de borda ausentes, (3) cenários de erro/exceção faltando, (4) riscos de negócio sem cobertura de teste. Apresente em formato: Lacuna | Risco | Prioridade sugerida. Cenários existentes: [LISTE OS CENÁRIOS ATUAIS]
```

→ *O que você NÃO testa é tão importante quanto o que você testa.*

PROMPT #14 **Investigar flaky test recorrente**

```
Aja como um engenheiro de qualidade especialista em diagnóstico de flaky tests. O teste abaixo falha intermitentemente (~30% das execuções) em CI mas passa localmente. Analise: (1) causas possíveis por categoria (timing, dados, ambiente, dependência externa), (2) como instrumentar o teste para coletar mais dados sobre a falha, (3) solução definitiva para cada causa possível. Teste: [COLE O CÓDIGO DO TESTE] Erro observado: [COLE OS ERROS]
```

→ *Flaky em CI mas não local = geralmente timing, variável de ambiente ou isolamento de dados.*

PROMPT #15 **Calcular ROI de automação para apresentar ao time**

```
Aja como um QA Lead preparando justificativa de investimento em automação para gestão. Com base nos dados abaixo, calcule e apresente: (1) tempo atual gasto em regressão manual, (2) tempo estimado com automação, (3) break-even point do investimento, (4) métricas de qualidade esperadas (defect escape rate, coverage), (5) apresentação executiva em até 5 bullets. Dados: Regressão atual: [X horas] | Frequência: [N vezes/sprint] | Custo hora QA: R$ [X] | Time: [N QAs]
```

→ *Automação precisa de business case. Esse prompt gera o argumento para sua liderança.*

Categoria 4 — Estratégia e Planejamento

PROMPT #16

Criar estratégia de testes para nova feature

```
Aja como um QA Lead Sênior com experiência em times de produto. Baseado na especificação abaixo, crie uma estratégia de testes completa: (1) análise de riscos da feature, (2) pirâmide de testes recomendada (quantos testes por camada), (3) casos de teste de alto nível por categoria, (4) critérios de entrada e saída para liberação, (5) dependências e pré-requisitos para execução. Especificação: [COLE A SPEC/PRD]
```

→ *Estratégia antes de escrever um único teste. Sempre.*

PROMPT #17

Criar plano de automação para time sem automação

```
Aja como um QA Architect contratado para estruturar automação do zero em uma empresa com 5 QAs manuais, aplicação web + mobile, sem nenhum teste automatizado atual. Crie um plano de 90 dias: Semana 1-2: diagnóstico e escolha de stack. Mês 1: MVP de automação (smoke tests críticos). Mês 2: cobertura de regressão principal. Mês 3: CI/CD integrado e mobile básico. Para cada fase: entregáveis, critérios de sucesso e riscos.
```

→ *Plano de 90 dias é o mais pedido em entrevistas de QA Lead. Pratique com esse prompt.*

PROMPT #18

Definir métricas de qualidade para o time

```
Aja como um QA Manager implementando cultura de qualidade baseada em dados. Defina um dashboard de métricas de qualidade para um time de desenvolvimento ágil, incluindo: (1) métricas de processo (defect density, escape rate, lead time de bug fix), (2) métricas de automação (cobertura, tempo de execução, taxa de flakiness), (3) métricas de qualidade do produto (SLA de performance, disponibilidade), (4) como coletar cada métrica, (5) targets recomendados por maturidade do time.
```

→ *Métricas sem contexto são ruído. Esse prompt gera um framework completo e justificado.*

PROMPT #19

Criar onboarding de QA júnior em automação

```
Aja como um QA Lead criando plano de onboarding para um QA júnior sem experiência em automação que precisa estar produtivo em 60 dias. Stack do time: Playwright + TypeScript + GitHub Actions. Crie semana a semana: conceitos a aprender, exercícios práticos, entregáveis para validar aprendizado e como o lead deve dar suporte. Seja realista sobre o ritmo de aprendizado de um júnior.
```

→ *Onboarding estruturado economiza tempo do lead e reduz frustração do júnior.*

PROMPT #20 **Preparar pergunta técnica para entrevista de QA Sênior**

```
Aja como um Tech Lead conduzindo entrevista técnica para QA Sênior especialista em automação. Crie 5 questões técnicas progressivas (do conceito à aplicação prática) sobre o tema: [TEMA — ex: flaky tests, POM, CI/CD, estratégia de testes]. Para cada questão: o que você está avaliando, resposta esperada de um candidato sênior, follow-up questions para aprofundar e red flags que eliminam o candidato.
```

→ *Use para se preparar para dar ou receber entrevistas técnicas de QA.*

Categoria 5 — Mobile, Performance e Segurança

PROMPT #21 **Criar estratégia de testes mobile para lançamento**

```
Aja como um QA Lead especialista em qualidade mobile com experiência em lançamentos de apps financeiros na Play Store e App Store. Para o app descrito abaixo, crie uma estratégia de testes mobile completa: dispositivos prioritários para teste (versões Android/iOS, tamanhos de tela), funcionalidades específicas de mobile a testar (permissões, notificações, GPS, biometria), estratégia de testes em cloud (BrowserStack/Sauce Labs) vs real devices, critérios de liberação para produção. App: [DESCREVA O APP E SUAS FUNCIONALIDADES CRÍTICAS]
```

→ *Mobile tem dimensão adicional: hardware, SO, fabricante. Estratégia sem isso é incompleta.*

PROMPT #22 **Criar cenários de carga para API crítica**

```
Aja como um Engenheiro de Performance especialista em testes de carga com k6. Para a API descrita abaixo, crie: (1) script k6 completo com cenários de ramp-up, sustain e ramp-down, (2) thresholds de SLA (p95, p99, taxa de erro), (3) análise de resultados — o que monitorar durante o teste, (4) critérios de go/no-go para produção. API: [DESCREVA O ENDPOINT E O VOLUME ESPERADO EM PRODUÇÃO]
```

→ *Volume esperado em produção é o input mais crítico. Sem ele, os thresholds são chute.*

PROMPT #23 **Revisar aplicação para vulnerabilidades OWASP**

```
Aja como um QA de Segurança especialista em OWASP Top 10 2021. Revise a funcionalidade descrita abaixo e gere: um checklist de verificações de segurança por categoria de risco OWASP, payloads de teste específicos para essa funcionalidade, ferramentas recomendadas (OWASP ZAP, Burp Suite, etc.), e como documentar vulnerabilidades encontradas para o time de dev. Funcionalidade: [DESCREVA O FLUXO A TESTAR]
```

→ *QA de segurança não é pentest — é verificar que as proteções existem e funcionam.*

PROMPT #24 **Criar testes de acessibilidade WCAG**

```
Aja como um especialista em acessibilidade digital e QA de inclusão. Para a tela descrita abaixo, crie um checklist de verificações de acessibilidade baseado em WCAG 2.1 nível AA, cobrindo: contraste de cor, navegação por teclado, leitores de tela (aria-labels), foco visível, textos alternativos e reflow em 320px. Para cada item: como testar manualmente e como automatizar com axe-core. Tela: [DESCREVA A TELA]
```

→ *Acessibilidade é requisito legal em sistemas governamentais e financeiros no Brasil.*

PROMPT #25 **Criar estratégia de observabilidade para QA**

```
Aja como um QA Engineer especialista em observabilidade e monitoramento. Defina uma estratégia de observabilidade para suporte ao QA no produto descrito, incluindo: (1) logs estruturados essenciais para diagnóstico de bugs, (2) métricas de aplicação que QA deve monitorar, (3) alertas de anomalia que devem disparar criação automática de bug, (4) como correlacionar logs + métricas + traces para um bug específico, (5) ferramentas recomendadas (Datadog, Grafana, ELK, etc.). Produto: [DESCREVA A ARQUITETURA E O STACK]
```

→ *QA moderno usa observabilidade — não espera usuário reportar o bug.*

Categoria 6 — Carreira e Comunicação

PROMPT #26 **Preparar para entrevista técnica de QA Lead**

```
Aja como um Tech Lead de empresa de tecnologia preparando um candidato para entrevista de QA Lead Sênior. O candidato tem 8 anos de experiência, sendo 3 em automação. Simule uma entrevista técnica de 30 minutos cobrindo: estratégia de qualidade, arquitetura de automação, gestão de débito de qualidade, experiência com CI/CD e como você abordaria um time sem cultura de qualidade. Após cada resposta, dê feedback específico: o que foi bom, o que faltou.
```

→ *Simular entrevista com IA antes da real reduz ansiedade e identifica gaps.*

PROMPT #27 **Escrever proposta de melhoria de processo para gestão**

```
Aja como um QA Lead Sênior escrevendo uma proposta executiva para implementar testes automatizados em um time que ainda faz tudo manualmente. A proposta deve ser em linguagem de negócio (não técnica), incluindo: problema atual com dados (tempo, custo, risco), solução proposta com roadmap simplificado, investimento necessário (tempo da equipe, ferramentas), retorno esperado em 3/6/12 meses e próximos passos concretos. Contexto: [DESCREVA SEU TIME E A SITUAÇÃO ATUAL]
```

→ *Proposta bem escrita é o que separa QA que implementa mudanças de QA que só reclama.*

PROMPT #28 **Criar descrição de vagas para QA Sênior/Lead**

```
Aja como um Head de Engineering escrevendo descrição de vaga atraente para um QA Sênior especialista em automação. A vaga deve: descrever o impacto real do trabalho (não lista de tarefas), stack e contexto técnico honesto, diferenciais da empresa para o candidato, requisitos técnicos objetivos (separando must-have de nice-to-have) e o que NÃO é a vaga (ser honesto filtra candidatos ruins). Stack: [DESCREVA A STACK DO TIME]
```

→ *Boa vaga atrai candidato certo. Vaga genérica atrai 200 candidatos errados.*

PROMPT #29 **Criar blog post técnico sobre tema de QA**

```
Aja como um escritor técnico especialista em qualidade de software. Escreva um blog post de 800-1000 palavras sobre: [TEMA]. O post deve ser: voltado para profissionais de QA, com exemplos de código reais (não pseudo-código), opiniões bem fundamentadas (não apenas 'depende'), estrutura: problema → contexto → solução → armadilhas → conclusão prática. Tom: técnico mas acessível, como um colega experiente explicando.
```

→ *Conteúdo técnico de qualidade é o que constrói autoridade no mercado de QA.*

PROMPT #30 **Criar curriculum técnico para vaga de QA Sênior**

```
Aja como um especialista em RH técnico de empresas de produto digital. Revise o curriculum abaixo para uma vaga de QA Sênior/Lead em empresa de produto. Reescreva as experiências em formato de impacto (reduzi X, aumentei Y, implementei Z), identifique keywords técnicas ausentes que ATS vai buscar, sugira ordem de seções para maximizar leitura em 10 segundos, aponte o que remover (irrelevante para QA Sênior) e o que adicionar. CV atual: [COLE AS EXPERIÊNCIAS]
```

→ *ATS filtra antes do humano ler. Keywords certas = passar para a próxima fase.*

Prompts 031 a 050 — Referência Rápida

#31 **Gerar dados LGPD-compliant para testes**

```
Aja como DPO + QA Sênior. Gere dados de teste que: (1) não são dados reais de pessoas, (2) passam nas validações da aplicação, (3) cobrem todos os campos PII do sistema. Para cada campo PII: explique como anonimizar em logs e relatórios.
```

#32 **Criar plano de testes de migração de banco de dados**

```
Aja como QA especialista em migrações. Crie plano de testes para migração de [banco A] para [banco B] cobrindo: integridade, performance, rollback e validação pós-migração.
```

#33 Analisar relatório de cobertura de código e priorizar gaps

```
Aja como QA Lead. Analise o relatório de cobertura abaixo e priorize: quais gaps aumentam risco de bug, quais podem ser aceitos e quais testes adicionar primeiro.
```

#34 Criar checklist de release para mobile

```
Aja como QA Mobile Lead. Crie checklist de 50 itens para liberar app em produção: funcional, performance, segurança, acessibilidade, stores (Play/App Store).
```

#35 Escrever RCA (Root Cause Analysis) de bug crítico

```
Aja como QA Sênior documentando RCA. Dado o incidente abaixo, escreva RCA executiva: o que aconteceu, por que passou pelos testes, impacto, ação imediata e preventiva.
```

#36 Criar suite de testes de contrato com Pact

```
Aja como QA Architect especialista em consumer-driven contract testing. Implemente com Pact.js: consumer test, provider verification e pipeline de verificação.
```

#37 Revisar pipeline CI/CD e identificar gaps de qualidade

```
Aja como QA DevOps. Analise o pipeline abaixo e identifique: onde adicionar testes, quais gates de qualidade faltam e como não atrasar o deploy.
```

#38 Criar estratégia de testes para microsserviços

```
Aja como QA Architect em arquitetura de microsserviços. Defina estratégia de testes por camada: unitário, integração, contrato, E2E. Para cada camada: ferramenta, cobertura alvo e quem escreve.
```

#39 Analisar bugs recorrentes e propor prevenção

```
Aja como QA Lead fazendo análise de padrões. Analise o histórico de bugs abaixo, identifique padrões recorrentes e proponha: testes preventivos, melhorias de processo e indicadores de alerta precoce.
```

#40 Criar treinamento de qualidade para desenvolvedores

```
Aja como QA Lead criando cultura de qualidade. Monte treinamento de 4h para devs: mentalidade de teste, como escrever código testável, TDD básico e code review com foco em qualidade.
```

#41 Gerar mock de serviço externo para testes

```
Aja como QA Sênior implementando test doubles. Para a integração descrita,
implemente: mock completo com WireMock/MSW, cenários de falha e timeout, e como
alternar entre mock e real no CI/CD.
```

#42 Criar estratégia de testes para GraphQL

```
Aja como QA especialista em APIs GraphQL. Crie estratégia de testes cobrindo:
queries, mutations, subscriptions, validação de schema, N+1 queries e testes de
introspection desabilitada em produção.
```

#43 Implementar teste de chaos engineering básico

```
Aja como QA de Resiliência. Crie plano de chaos testing para [sistema]: falhas a
simular, impacto esperado, como monitorar durante o caos e critérios de recuperação
aceitável.
```

#44 Criar KPIs de qualidade para OKR do time

```
Aja como QA Manager definindo OKRs de qualidade. Para o time descrito, defina: 3
Objectives e 9 Key Results de qualidade mensuráveis, com baseline atual e target
trimestral.
```

#45 Analisar user journey e criar mapa de riscos

```
Aja como QA Sênior fazendo análise de risco baseada em usuário. Para o user journey
descrito, crie mapa de riscos: por etapa, probabilidade de falha, impacto e
cobertura de teste atual.
```

#46 Criar documentação técnica de framework de automação

```
Aja como Tech Writer + QA Architect. Documente o framework de automação: arquitetura
(com diagrama Mermaid), guia de contribuição, padrões obrigatórios e FAQ dos erros
mais comuns.
```

#47 Implementar notificação inteligente de falhas em CI

```
Aja como QA DevOps. Implemente sistema de notificação de falhas de teste em CI que:
(1) agrupa falhas similares, (2) identifica flaky vs real failure, (3) roteia para
canal correto no Slack com contexto completo.
```

#48 Criar estratégia de testes de internacionalização (i18n)

```
Aja como QA especialista em i18n. Para o sistema que suporta PT-BR, EN-US e ES, crie
estratégia de testes cobrindo: strings, datas, moedas, caracteres especiais e RTL se
aplicável.
```

#49 Analisar feature flag e criar matriz de testes

```
Aja como QA Sênior em time com feature flags. Para a feature descrita com [N flags],
crie: matriz de combinações a testar, como testar rollout gradual e o que monitorar
durante ativação em produção.
```

#50 Criar plano de testes de disaster recovery

```
Aja como QA de Resiliência + SRE. Crie plano de testes para DR do sistema descrito:
cenários de falha a simular (datacenter down, banco corrompido, etc.), RPO/RTO a
validar e como automatizar os testes de DR.
```

Checklist de Automação Profissional

Use este checklist ao iniciar um novo projeto, revisar um framework existente ou onboarding de um novo QA no time.

Estrutura do Projeto

- [ ] Pastas organizadas: tests/, pages/, fixtures/, helpers/, utils/
- [ ] playwright.config.js ou cypress.config.js na raiz
- [ ] .env.example com todas as variáveis de ambiente documentadas
- [ ] .gitignore configurado (node_modules, .auth, downloads, screenshots)
- [ ] README.md com: como instalar, como rodar e como contribuir
- [ ] package.json com scripts: test, test:headed, test:smoke, test:api

Estratégia de Testes

- [ ] Pirâmide de testes definida: % de unitários, integração e E2E
- [ ] Smoke suite separada com máximo 15 testes críticos
- [ ] Regressão completa separada da smoke
- [ ] Critérios de entrada/saída documentados
- [ ] Mapeamento de features críticas vs cobertura atual
- [ ] Plano de expansão da cobertura por sprint

Qualidade do Código de Teste

- [ ] Page Object Model implementado para todas as telas testadas
- [ ] Localizadores: 90%+ usando data-testid ou accessibility id
- [ ] Zero hard sleeps (time.sleep / waitForTimeout fixo) no código
- [ ] Asserções explícitas em cada ação crítica do teste

- [ ] Testes independentes: cada teste cria e limpa seus próprios dados
- [ ] Nomes de teste descritivos: deve_FAZER_quando_CONDIÇÃO
- [ ] Dados de teste externalizados em fixtures, não hardcoded

CI/CD

- [ ] Pipeline executando em cada Pull Request
- [ ] Pipeline bloqueando merge se testes falharem
- [ ] Variáveis sensíveis em secrets, nunca no código
- [ ] Relatório HTML publicado como artefato em cada run
- [ ] Notificação de falha no Slack/Teams com link para relatório
- [ ] Execução paralela configurada para reduzir tempo total
- [ ] Retry configurado: máximo 2 tentativas apenas em CI

Métricas e Monitoramento

- [ ] Taxa de flakiness monitorada (alerta se > 5%)
- [ ] Tempo de execução da suite monitorado por semana
- [ ] Cobertura de casos críticos acompanhada (alvo: 80%+)
- [ ] Defect escape rate (bugs encontrados em prod vs QA) registrado
- [ ] Dashboard de qualidade atualizado semanalmente

Segurança e Compliance

- [ ] Nenhuma credencial real no código ou no repositório
- [ ] Dados de usuários reais nunca usados em testes
- [ ] Testes de segurança básicos (auth, IDOR) incluídos na suite
- [ ] Ambientes de teste isolados de produção
- [ ] Logs de teste não expõem PII

Glossário Completo QA + Automação

150+ termos técnicos definidos de forma prática para QA e Automação.

A/B Testing	Técnica de comparação onde dois grupos de usuários recebem versões diferentes da aplicação para medir qual performa melhor. QA valida que os grupos são corretamente separados e que os dados são coletados sem interferência.
Accessibility ID	Localizador mobile no Appium que usa o atributo contentDescription (Android) ou accessibilityLabel (iOS) do elemento. É o localizador mais estável para automação mobile — deve ser o primeiro a usar.
Allure	Framework de relatório de testes com visualização rica: histórico, tendências, suítes, screenshots e steps. Integra com Playwright, JUnit, pytest e outros.
API Contract	Acordo formal entre produtor e consumidor de API sobre: endpoints disponíveis, formatos de request/response, status codes esperados e comportamento em falhas. Testes de contrato validam esse acordo automaticamente.
Appium	Framework open source para automação de apps mobile (iOS e Android). Usa o protocolo WebDriver, permitindo escrever testes com múltiplas linguagens (Python, Java, JS).
Assertion	Verificação explícita no código de teste. Ex: expect(status).toBe(200). Uma boa asserção tem mensagem descritiva de falha. Nunca escreva teste sem asserção.
BDD	Behavior-Driven Development — metodologia onde testes são escritos como comportamentos esperados em linguagem próxima ao negócio (Gherkin). Facilita comunicação entre QA, Dev e PO.
Black Box Testing	Teste sem conhecimento do código interno. O QA só conhece entradas e saídas esperadas. Diferente de White Box (acesso ao código) e Gray Box (acesso parcial).
Boundary Value Analysis	Técnica de design de teste que foca nos valores nos limites de partições (mínimo, máximo, mínimo-1, máximo+1). Ex: campo aceita 1-100 → testar 0, 1, 100, 101.
CI/CD	Continuous Integration/Continuous Delivery. CI = integração automática com testes a cada commit. CD = entrega automatizada para produção. Testes são o gate principal do pipeline.
Clean Code	Conjunto de práticas para código legível e manutenível: nomes descritivos, funções pequenas, sem duplicação. Aplica-se ao código de teste tanto quanto ao código de produção.

Code Coverage	Porcentagem do código de produção exercitada pelos testes. Alta cobertura não garante qualidade — um teste pode cobrir uma linha sem verificar nada. Métrica útil como indicador, não como objetivo absoluto.
Consumer-Driven Contract	Tipo de teste de contrato onde o consumidor da API define as expectativas. Ferramentas: Pact, Spring Cloud Contract. Evita breaking changes em microsserviços.
Cypress	Framework de automação web moderno, focado em testes E2E de aplicações JavaScript. Executa dentro do browser, tem Time Travel debugging e reload automático. Limitado a browsers baseados em Chromium + Firefox.
Data-Driven Testing	Abordagem onde o mesmo teste é executado com múltiplos conjuntos de dados. Reduz duplicação de código e aumenta cobertura com menos esforço.
Defect Density	Número de bugs por unidade de código (ex: bugs por 1000 linhas de código ou por story point). Métrica de qualidade de produto ao longo do tempo.
Defect Escape Rate	Porcentagem de bugs que chegaram a produção sem serem detectados pelo QA. Indica efetividade do processo de testes. Alvo ideal: < 5%.
E2E Test	End-to-End Test — teste que simula o fluxo completo do usuário, do front ao banco de dados. Mais lento e frágil que testes unitários, mas valida integrações reais.
Equivalence Partitioning	Técnica de design de teste que divide inputs em classes equivalentes. Apenas um representante de cada classe precisa ser testado. Ex: campo aceita A-Z e a-z → testar 'A' representa toda a classe maiúscula.
Exploratory Testing	Teste sem roteiro fixo, onde QA usa conhecimento do sistema para descobrir bugs não previstos. Complementar à automação — a IA nunca substitui a criatividade humana no exploratório.
Fixture	Dados ou estado necessário para executar um teste. Pode ser: arquivo JSON com dados, objeto criado via API antes do teste, ou estado do browser (login já realizado).
Flaky Test	Teste que passa e falha intermitentemente sem mudança no código. Causa: timing, dados compartilhados, dependência de ordem, ambiente instável. É pior que teste que sempre falha — destrói confiança na suite.
Gherkin	Linguagem de especificação do BDD. Palavras-chave: Given (pré-condição), When (ação), Then (resultado esperado), And, But. Executada por frameworks como Cucumber.
GitHub Actions	Plataforma de CI/CD integrada ao GitHub. Permite criar workflows YAML que executam testes a cada push/PR. Gratuito para repositórios públicos.
Hard Assert	Asserção que interrompe o teste imediatamente ao falhar. Comportamento padrão. Oposto de Soft Assert.

Headless Browser	Browser sem interface gráfica. Mais rápido para testes automatizados em CI. Playwright e Cypress suportam modo headless nativamente.
IDOR	Insecure Direct Object Reference — vulnerabilidade onde usuário acessa recurso de outro usuário manipulando IDs. Ex: GET /usuarios/123 retorna dados do usuário 123 para qualquer usuário autenticado.
Integration Test	Teste que verifica a integração entre dois ou mais componentes. Ex: frontend + API, API + banco de dados, dois microsserviços. Mais lento que unitário, mais rápido que E2E.
JMeter	Ferramenta Apache de testes de carga e performance. Suporta HTTP, JDBC, FTP, SOAP. Mais pesado que k6 mas mais completo para protocolos legados.
JSON Schema	Especificação formal da estrutura de um objeto JSON. Usado em testes de contrato de API para validar que responses seguem o formato esperado.
k6	Ferramenta moderna de teste de performance. Scripts em JavaScript, relatórios em tempo real, integração nativa com CI/CD. Mais leve e desenvolvedor-friendly que JMeter.
Locator	Referência a um elemento na página/tela. Playwright: page.locator('#id'). Appium: driver.find_element('accessibility id', 'nome'). Bons localizadores são estáveis e semânticos.
Mock	Objeto/serviço falso que simula comportamento de uma dependência real em testes. Diferente de stub (simples) e spy (registra chamadas). Usado para isolar o código sendo testado.
Mutation Testing	Técnica que introduz bugs artificiais (mutações) no código de produção para verificar se os testes detectam. Se os testes não capturam a mutação, são considerados fracos.
OWASP	Open Web Application Security Project. Mantém o Top 10 de vulnerabilidades mais críticas em aplicações web. QA deve conhecer e testar para todos os itens relevantes.
Page Object Model	Padrão de design onde cada página da aplicação é uma classe com seus elementos e ações encapsulados. Reduz duplicação, melhora manutenibilidade e torna testes legíveis.
Playwright	Framework de automação web moderno da Microsoft. Suporta Chromium, Firefox e WebKit. Multi-linguagem (JS/TS, Python, Java, C#). Tem API nativa para testes de browser e API HTTP.
QAOps	Integração de práticas de QA no pipeline DevOps. QA não é fase final, mas processo contínuo: qualidade em cada etapa do desenvolvimento, do refinamento ao deploy.
Regression Test	Teste que garante que funcionalidades existentes não foram quebradas por mudanças recentes. É o principal candidato à automação — executa repetidamente a cada ciclo.

REST	Representational State Transfer — estilo arquitetural para APIs. Usa HTTP (GET, POST, PUT, PATCH, DELETE), JSON como formato padrão e status codes para indicar resultado.
Robot Framework	Framework de automação open source baseado em Python com sintaxe de palavras-chave. Usado para web (SeleniumLibrary), API, banco de dados e mobile.
Sanity Test	Subset focado da regressão, executado após uma correção específica para verificar que o fix funcionou sem introduzir regressão imediata.
Selenium	Framework de automação web mais antigo e mais usado no mercado. Suporta múltiplas linguagens e browsers. Grid permite execução distribuída. Base de muitos frameworks corporativos.
Selenium Grid	Componente do Selenium que permite executar testes em múltiplas máquinas/browsers simultaneamente. Usado em pipelines de CI para paralelismo.
Shift-Left	Prática de mover atividades de qualidade para mais cedo no ciclo de desenvolvimento. QA envolvido desde o refinamento, não apenas no final do sprint.
Smoke Test	Suite de testes mínima que verifica se as funcionalidades mais críticas do sistema estão funcionando. Se smoke falhar, mais nenhum teste precisa rodar.
Soft Assert	Asserção que não interrompe o teste ao falhar — acumula todos os erros e reporta ao final. Útil quando você quer ver múltiplos problemas de uma vez.
SQL Injection	Ataque onde código SQL malicioso é inserido em campos de input. Testes de segurança devem verificar que a aplicação sanitiza inputs e usa queries parametrizadas.
Stub	Implementação simplificada de uma dependência que retorna respostas fixas. Mais simples que mock — não registra chamadas, só simula comportamento.
Test Double	Termo genérico para qualquer substituição de componente real em testes: mock, stub, fake, spy, dummy.
TestOps	Prática de aplicar princípios de DevOps à gestão de testes: automação de testes como código, métricas contínuas, colaboração entre QA, Dev e Ops.
TDD	Test-Driven Development — escrever o teste antes do código de produção. Ciclo: Red (teste falha) → Green (código passa) → Refactor (melhora código mantendo testes verdes).
Trace (Playwright)	Artefato de debug do Playwright que grava screenshots, network, DOM e ações em cada passo do teste. Visualizado com 'npx playwright show-trace'. Essencial para debug em CI.
UAT	User Acceptance Testing — testes realizados pelos usuários finais ou stakeholders para validar que o sistema atende aos requisitos de negócio antes do go-live.

UiAutomator2	Driver do Appium para automação Android. Usa a API UiAutomator do Android internamente. Mais estável que o driver legado UIAutomator.
Unit Test	Teste que verifica uma unidade isolada de código (função, método, classe). Rápido, barato, deve ser a base da pirâmide de testes. Responsabilidade primária do desenvolvedor.
Viewport	Área visível da janela do browser. Usado em testes de responsividade para simular diferentes tamanhos de tela (mobile, tablet, desktop).
WCAG	Web Content Accessibility Guidelines — diretrizes de acessibilidade para conteúdo web. Nível AA é o mais exigido em sistemas públicos e financeiros.
WebDriver	Protocolo padrão W3C para automação de browsers. Selenium, Playwright e Appium usam WebDriver internamente para se comunicar com browsers e apps.
XCUITest	Driver do Appium para automação iOS. Usa o framework XCTest da Apple. Necessário para automatizar apps nativos e híbridos em iOS.
XPath	Linguagem de consulta para navegação em XML/HTML. Usado como localizador quando id/css não está disponível. Evite XPath absoluto (/html/body/div[2]/...) — prefira relativo e semântico.
Zero-defect Policy	Meta de qualidade onde nenhum bug conhecido deve ir para produção. Requer processo de qualidade maduro, cobertura de testes alta e cultura de qualidade no time.

FIM DO EBOOK — PERCA O MEDO DA AUTOMAÇÃO

QA Insider | Lilian Borba | qa-insider-labs.base44.app

Todos os direitos reservados. Reprodução proibida sem autorização.